

第 31 章: Designing with Classes (用类设计)

原书:《Learning Python》(6th Edition), 作者 Mark Lutz 本章地位: 本书"类与 OOP"部分的实战设计篇。前面几章讲清了 class 的语法机制, 本章则回答"怎么用类"的问题——用类的组合方式(继承、组合、委托、工厂)来建模真实世界, 并深入探讨多重继承与 MRO 线性化、伪私有属性、绑定方法等设计级概念。这一章是读者从"会写类"走向"会设计类"的桥梁, 也是全书唯一系统讲解 OOP 设计模式与"优先组合而非继承"原则的章节。

31.1 Python 与 OOP (Python and OOP)

英文原文

Let's begin with a review—Python's implementation of OOP can be summarized by three ideas: Inheritance (继承) — Inheritance is based on attribute lookup in Python (in `X.name` expressions). Polymorphism (多态) — In `X.method`, the meaning of method depends on the type (class) of subject object `X`. Encapsulation (封装) — Methods and operators implement behavior, though data hiding is a convention by default. By now, you should have a good feel for what basic inhe

heritance is all about in Python, so we'll take it as a given here. As you've learned, it's the mechanism behind flexible code customization. We've also talked about Python's polymorphism a few times already. It flows from Python's lack of type declarations (per Chapter 6, the optional, unused, and paradoxical "type hinting" doesn't qualify). Because attributes are always resolved at runtime, objects that implement the same interfaces are automatically interchangeable; clients don't need to know what sorts of objects are implementing the methods they call. Newer here, encapsulation in Python means packaging—that is, hiding implementation details behind an object's interface. It does not mean enforced privacy, though that can be partly implemented with code, as you'll see in Chapter 39. Encapsulation is available and useful in Python nonetheless: it allows the implementation of an object's interface to be changed without impacting the users of that object.

中文翻译

让我们先做个回顾——Python 对 OOP 的实现可以用三个思想来概括：继承（Inheritance）——Python 中的继承基于属性查找（在 `X.name` 表达式中）。多态（Polymorphism）——在 `X.method` 中，`method` 的含义取决于主体对象 `X` 的类型（类）。封装（Encapsulation）——方法和运算符实现行为，不过默认情况下数据隐藏只是一种约定（convention），而非强制。到现在，你应该对 Python

中基本的继承是怎么回事已经很有感觉了，所以我们在这里把它当作已知前提。正如你所学的，继承正是灵活代码定制背后的机制。我们也已经好几次谈到 Python 的多态。它源于 Python 没有类型声明这一事实（按第 6 章的说法，可选、没用而又自相矛盾的"类型提示（type hinting）"不算数）。因为属性总是在运行时（runtime）才解析，实现了相同接口（interface）的对象可以自动互换；调用方（client）无需知道实现它所调用方法的究竟是哪种对象。相对新一点的是，Python 中的封装意味着打包（packaging）——也就是把实现细节隐藏在对象的接口背后。它并不意味着强制私有（privacy），尽管在某种程度上可以靠代码实现，正如你将在第 39 章看到的。尽管如此，封装在 Python 中依然可用且有用：它允许你改变对象接口的实现，而不影响该对象的使用者。

深度理解

·核心概念：Python 的 OOP = 继承（属性查找）+ 多态（运行时动态分发）+ 封装（接口隔离实现）。注意三个词都偏离了 C++/Java 教科书定义：Python 的"继承"只是"属性查找时沿类树向上搜索"，"多态"是"鸭子类型"而非类型签名重载，"封装"是"打包"而非私有性。

·底层实现：X.name 表达式的求值会在运行时触发沿 MRO（方法解析顺序）的属性搜索；X.method() 等价于先查找到方法函数，再把 X 绑定为第一个参数（self）。一切在运行时决定，这就是 Python 编译型语言所没有的动态性。

·设计原因：Lutz 强调"属性总是运行时解析"，是为了说明

接口互换性的价值——调用方只认接口，不认类型。这带来代码复用与未来兼容：今天传入的任何对象，只要明天还实现同样的方法，代码就仍然正确。

·实际问题：封装"是约定而非强制"意味着你可以修改任何对象的任何属性（如 `obj.dict`）。这在大团队里容易造成"偷偷改别人的属性"的隐患，所以需要靠命名约定（如单下划线 `X`）和伪私有属性来管理。

·初学者误区：很多从 C++/Java 转来的学习者会问"Python 的属性为什么没有 `private/protected` 关键字？"——答案是：Python 选择"信任程序员"（guarding by convention），用 `X` 约定和 `X` 改写机制代替语言级强制；强制私有反而会破坏它的灵活性。生活类比：C++ 的私有成员像保险柜，Python 的属性像抽屉上的标签——标着"别动"，但真想打开也拦不住，靠的是团队默契。

31.2 多态意味着接口，而非调用签名 (Polymorphism Means Interfaces, Not Call Signatures)

英文原文

Some OOP languages also define polymorphism to mean overloading functions based on the type signatures of their arguments—the number passed and/or their types. Because there are no real type declarations in Python, this concept doesn't apply; as we've seen

n, polymorphism in Python is based on object interfaces, not types. If you're pining for your C++ days, you can try to overload methods by their argument lists, like this: `python class C: def meth(self, x): ... def meth(self, x, y, z): ...` Such code will run without error, but because the `def` simply assigns an object to a name in the class's scope, the last definition of the method function is the only one that will be retained. Put another way, it's just as if you say `X = 1` and then `X = 2`; at the end, `X` will be 2. Hence, there can be only one definition of a method name. Per Chapter 29, this includes `__init__` constructors, which are only special because of when they are run. If call-signature dispatch is truly required, you can always code type-based selections using the type-testing ideas we met in Chapters 4 and 9, or the argument-list tools introduced in Chapter 18: `python class C: def meth(self, args): if len(args) == 1: # Branch on number arguments ... elif type(args[0]) == int: # Branch on argument types (or isinstance()) ...` You normally shouldn't do this, though—it's not the "Python way." As explained in Chapter 16, you should write your code to expect only an object interface, not a specific object type. That way, it will be useful for a broader category of types and applications, both now and in the future: `python class C: def meth(self, x): x.operation() # Assume x does the right thing` It's also generally considered better to use distinct method names for distinct operations rather than relyin

g on call signatures (no matter what language you code in). But enough review. Although Python's object model is straightforward, much of the art in OOP is in the way we combine classes to achieve a program's goals. The next section begins a tour of some of the ways larger programs use classes to their advantage.

中文翻译

有些 OOP 语言还把多态定义为：根据参数的类型签名（传入参数的数量和/或类型）来重载（overload）函数。由于 Python 没有真正的类型声明，这个概念并不适用；正如我们所见，Python 中的多态基于对象接口，而非类型。如果你怀念 C++ 的日子，可以尝试按参数列表重载方法，像这样：
`python class C: def meth(self, x): ... def meth(self, x, y, z): ...` 这样的代码运行不会报错，但因为 `def` 只是把一个对象赋值给类作用域里的一个名字，所以方法函数的最后一个定义是唯一会被保留的。换句话说，这就像你先说 `X = 1` 再说 `X = 2`；最终 `X` 会是 2。因此，一个方法名只能有一个定义。按第 29 章的说法，这包括 `init` 构造器——它只是因为何时运行才显得特殊。如果确实需要按调用签名分发，你总可以用第 4 章和第 9 章介绍过的类型测试思路，或第 18 章引入的参数列表工具来编码基于类型的选择：
`python class C: def meth(self, args): if len(args) == 1: # 按参数数量分支... elif type(arg[0]) == int: # 按参数类型分支（或用 isinstance()） ...` 不过你通常不应该这么做——这不是“Python 之道”。正如第 16 章所解释的，你应该把代码写成只期望一个对象接口，而不是某个特定对象类型。这样，无论现在还是将来，你的代码都能适用于更广泛的类型与

应用：python class C: def meth(self, x): x.operation()
假设 x 会做正确的事通常人们也认为：对不同的操作使用不同的方法名，比依赖调用签名更好（无论你用哪种语言编程）。不过回顾到此为止。虽然 Python 的对象模型直截了当，但 OOP 的艺术很大程度上在于我们如何组合类来实现程序的目标。下一节就开始游览大型程序利用类的各种方式。

深度理解

- 核心概念：多态 = 接口 (interface) 决定行为，而非类型 (type)。“重载”在 Python 中不存在——同一个名字的 def 只会留下最后一个定义。这是“动态类型 + 名称绑定”的直接推论。
- 底层实现：Python 中 def 是可执行语句，它把函数对象绑定 (bind) 到类作用域的命名空间 (dict) 中。第二次 def meth(...) 只是把新函数对象覆盖到同一个键上，旧对象被垃圾回收。所以“重载”在语法层就被设计掉了。
- 设计原因：类型签名重载服务于静态类型语言的编译期分派需求；Python 没有编译期类型检查，一切在运行时用属性查找解决，重载机制毫无必要。Lutz 借此贯彻全书主线：“期望接口，而不是类型”。
- 实际问题：如果你真的需要“按参数数量或类型走不同逻辑”，惯用做法是 args + isinstance()，或干脆拆成 meth1/meth2 两个方法。真正的 Python 式做法是依赖鸭子类型 (duck typing)，只调用对象提供的方法。
- 初学者误区：初学者常以为“同名不同参数的方法可以共

存"（照搬 Java）。后果是后定义的 def 悄悄覆盖前者，且无任何报错——这正是动态语言"宽容"的陷阱。另外，类型提示（type hint）不是类型声明，Python 运行时依然完全忽略它。

代码分析

```
class C:
    def meth(self, x):
        ...                                # —

    def meth(self, x, y, z):
        ...                                # —

class C2:
    def meth(self, *args):                 # *args
        if len(args) == 1:                 #
            ...
        elif type(args[0]) == int:         # isinstance...
            ...

class C3:
    def meth(self, x):
        x.operation()                     # x
```

·解释器如何处理：当解释器执行 class C 语句体时，它在一个全新的命名空间里逐条执行语句。第一个 def 创建函数对象并存入 C.dict['meth']; 第二个 def 创建新函数对象并覆盖同一键。类定义结束时，dict 里只剩最后一个函数。这跟 X = 1; X = 2 完全同理——"方法重载"在字节码层面根本不存在，因为名称查找先于一切。

·设计思想：args + 条件分支可以模拟重载，但作者明确说"通常不应该"——正确姿势是 C3 的接口风格：调用 x.

operation(), 让运行时去查找。好处是未来任何带 operation 方法的对象都能直接使用, 无需修改 meth。

31.3 OOP 与继承: "Is-a" (是一个) 关系 (OOP and Inheritance: "Is-a" Relationships)

英文原文

We've explored the mechanics of inheritance in depth already; let's turn to an example of how it can be used to model real-world relationships. From a programmer's point of view, inheritance is kicked off by attribute qualifications, which trigger searches for names in instances, their classes, and then any superclasses. From a designer's point of view, inheritance is a way to specify set membership: a class defines a set of properties that may be inherited and customized by more specific sets (i.e., subclasses).

To illustrate, let's put that pizza-making robot we talked about at the start of this part of the book to work. Suppose we've decided to explore alternative career paths and open a pizza restaurant (not bad, as career paths go). One of the first things we'll need to do is hire employees to serve customers, prepare the food, and so on. Being engineers at heart, we've decided to build a robot to make the pizzas, but being politically and cybernetically correct, we've also decided to make our robot a full-fledged employee with a sala

ry.

Our pizza shop team can be simulated by the four classes in Example 31-1, `employees.py`. The most general class, `Employee`, is a takeoff on Chapter 28's demo. It provides common behavior such as bumping up salaries (`giveRaise`) and printing (`repr`). There are two kinds of employees, and so there are two subclasses of `Employee`—`Chef` and `Server`. Both override the inherited `work` method to print more specific messages. Finally, our pizza robot is modeled by an even more specific class—`PizzaRobot` is a kind of `Chef`, which is a kind of `Employee`. In OOP terms, we call these relationships "is-a" links: a robot is a chef, which is an employee.

Example 31-1. `employees.py`

```
python class Employee:
    def init(self, name, salary=0):
        self.name = name
        self.salary = salary
    def giveRaise(self, percent):
        self.salary += self.salary * percent
    def work(self):
        print(self.name, 'does stuff')
    def repr(self):
        return (f'<{self.class.name}: '
                f'fname="{self.name}", salary={self.salary:,.2f}>')
```

```
class Chef(Employee):
    def init(self, name):
        Employee.init(self, name, 50000)
```

```
    def work(self):
        print(self.name, 'makes food')
class Server(Employee):
    def init(self, name):
        Employee.init(self, name, 40000)
```

```
def work(self): print(self.name,'interfaces with custom  
er') class PizzaRobot(Chef): def init(self, name): Chef.i  
nit(self, name)
```

```
def work(self): print(self.name,'makes pizza') if name  
=='main': pat = PizzaRobot('pat') # Make a robot na  
med pat print(pat) # Run inherited repr pat.work() #  
Run type-specific action pat.giveRaise(0.20) # Give pa  
t a 20% raise
```

```
print(pat); print() for klass in Employee, Chef, Server,  
PizzaRobot: object = klass(klass.name) object.work()
```

When we run the self-test code included in this module, we create a pizza-making robot named pat, which inherits names from three classes: PizzaRobot, Chef, and Employee. For instance, printing and giving a raise to pat runs the Employee class's repr and giveRaise methods two levels up, respectively, simply because that's where the inheritance search finds these methods:

```
$ python3 employees.py <PizzaRobot: name="pat",  
salary=50,000.00> pat makes pizza <PizzaRobot: na  
me="pat", salary=60,000.00> Employee does stuff Ch  
ef makes food Server interfaces with customer PizzaR  
obot makes pizza
```

In a class hierarchy like this, you can usually make instances of any of the classes, not just the ones at the bottom. For instance, the for loop in this module's self-test code creates instances of all four classes; each r

esponds differently when asked to work because the work method is different in each. `pat the robot`, for example, gets work from the most specific (i.e., lowest) `PizzaRobot` class.

Of course, these classes just simulate real-world objects; `work` prints a message for the time being, but it could be expanded to do real work later (see Python's interfaces to devices such as serial ports, Arduino boards, and the Raspberry Pi if you're taking this section much too literally!).

NOTE — The super re-reminder: Notice how Example 31-1 uses explicit class calls to run superclass constructors; this is how `PizzaRobot` salaries are set. Per the sidebar "The super Alternative", these could also use the `super().init(...)` form explored in the next chapter. As you'll find, this call avoids having to pass `self` along in single-inheritance trees like those here, but it's much more complex in the multiple-inheritance contexts you'll meet ahead. If you're anxious to see what this looks like now, see the alternative `employees-super.py` in the `examples` package.

中文翻译

我们已经深入探讨过继承的机制；现在来看一个用它建模真实世界关系的例子。从程序员的角度看，继承由属性限定（attribute qualification）触发——它引发对实例、实例的类、再到所有超类中名字的搜索。从设计者的角度看，继承是指定集合成员关系（set membership）的一种方式：—

个类定义了一组属性，这些属性可以被更具体的集合（即子类）继承和定制。为了说明，让我们把本书这一部分开头提到的那个做披萨的机器人投入使用。假设我们决定探索一条备选职业道路：开一家披萨餐厅（作为职业道路来说还不错）。我们要做的第一件事就是雇佣员工来服务顾客、准备食物等等。身为工程师的我们，决定造一个机器人来做披萨；但为了政治上和机器人学上（cybernetically）的正确性，我们也决定让我们的机器人成为一名拿薪水的正式员工。我们的披萨店团队可以用示例 31-1（文件 employees.py）中的四个类来模拟。最通用的类 Employee 是第 28 章那个 demo 的翻版，它提供加薪（giveRaise）和打印（repr）等通用行为。员工有两种，因此 Employee 有两个子类——Chef 和 Server。两者都覆盖继承来的 work 方法，以打印更具体的消息。最后，我们的披萨机器人由一个更具体的类建模——PizzaRobot 是一种 Chef，而 Chef 是一种 Employee。用 OOP 术语说，我们称这些关系为“is-a”（是一个）链：机器人是一个厨师，厨师是一个员工。示例 31-1. employees.py

```
python class Employee:
    def init(self, name, salary=0):
        self.name = name
        self.salary = salary
    def giveRaise(self, percent):
        self.salary += self.salary * percent
    def work(self):
        print(self.name, 'does stuff')
    def repr(self):
        return (f'<{self.class.name}:f' name="{self.name}", salary={self.salary:,.2f}>')
class Chef(Employee):
    def init(self, name):
        Employee.init(self, name, 50000)
    def work(self):
        print(self.name, 'makes food')
class Server(Employee):
    def init(self, name):
        Employee.init(self, name, 40000)
    def work(self):
        print(self.name, 'interfaces with customer')
class PizzaRobot(Chef):
    def i
```

```

nit(self, name): Chef.init(self, name) def work(self): pri
nt(self.name,'makes pizza') if name == 'main': pat = Pi
zzaRobot('pat') # 创建一个名叫 pat 的机器人print(pat)
# 运行继承来的 repr pat.work() # 运行类型专属的动作pa
t.giveRaise(0.20) # 给 pat 加薪 20% print(pat); print() f
or klass in Employee, Chef, Server, PizzaRobot: object
= klass(klass.name) # 依次实例化四个类object.work()

```

当我们运行这个模块自带的自测代码时，会创建一个名叫 `pat` 的做披萨机器人，它从三个类继承名字：`PizzaRobot`、`Chef` 和 `Employee`。例如，打印 `pat` 和给 `pat` 加薪，分别运行的是 `Employee` 类向上两级的 `repr` 和 `giveRaise` 方法——只因继承搜索恰好在那里找到这些方法：

```

$ python 3 employees.py <PizzaRobot: name="pat", salary=50,000.00>
pat makes pizza <PizzaRobot: name="pat", salary=60,000.00>
Employee does stuff Chef makes food Server interfaces with customer
PizzaRobot makes pizza

```

在这样的类层次结构中，你通常可以实例化其中任何一个类，而不只是最底层的那些。例如，本模块自测代码中的 `for` 循环创建了全部四个类的实例；每个实例被要求 `work` 时响应都不同，因为每个类里的 `work` 方法都不一样。例如，机器人 `pat` 的 `work` 来自最具体（即最低）的 `PizzaRobot` 类。当然，这些类只是模拟真实世界的对象；`work` 目前只是打印一条消息，但以后可以扩展去做真正的工作（如果你把这部分内容理解得太字面化了，可以去看看 Python 与串口、Arduino 板、树莓派等设备的接口！）。注意——`super` 的再次提醒：注意示例 31-1 使用显式的类调用（`Employee.init(self, ...)`）来运行超类构造器；`PizzaRobot` 的薪水就是这样设置的。按照侧边栏“The super Altern

ative"的说法，这些也可以改用下一章要探讨的 `super().init(...)` 形式。你会发现，这个调用在像这里这样的单继承树中免去了手动传递 `self`，但在你即将遇到的多重继承场景中要复杂得多。如果你急着想看它长什么样，请看示例包中的替代版 `employees-super.py`。

深度理解

·核心概念：“is-a”关系 = 集合成员关系。PizzaRobot 是一种 Chef，Chef 是一种 Employee。继承让子类自动拥有父类的全部行为，只覆盖自己不同的部分（这里只覆盖了 `work` 和构造器）。设计者视角：继承 = “类的集合可以继承并定制更具体集合的属性”。

·底层实现：`pat.work()` 会触发属性查找：先在 `pat` 实例的 `dict` 找 `work`（没有），再到 `PizzaRobot.dict`（找到，直接用）。`pat.giveRaise()` 则一路查到 `Employee.dict` 才找到。`repr` 里的 `self.class.name` 是动态的——实例的 `class` 指向创建它的类，所以所有子类打印时都显示自己的类名。

·设计原因：把通用逻辑（加薪、格式化打印）放在最高层基类，把差异逻辑（做什么工作、起薪多少）下放到各子类，实现了“写一遍、到处复用”。这正是继承的核心价值：代码复用 + 定制点分离。

·实际问题：注意构造器调用链——`Chef.init` 里要显式调用 `Employee.init(self, name, 50000)`，把子类特有的默认薪水传进去。忘记调用父类构造器是继承里最常见的 bug，会导致 `name/salary` 属性缺失 (`AttributeError`)

·初学者误区：一是以为 PizzaRobot 实例会拥有三个类各自独立的副本数据——实际只有一个实例 dict，属性是共享的、按需查找的；二是以为子类构造器"自动"调用父类构造器——Python 不会自动调用，必须显式调用；三是混淆 repr（调试显示）与 str（用户友好显示），print 优先用 str，两者都缺省时回退到默认的 <类名 object at 0x...>。

代码分析

·解释器如何处理：执行 class Chef(Employee) 时，解释器创建 Chef 类对象并记录 bases = (Employee,)。调用 PizzaRobot('pat') 时，解释器先找到 PizzaRobot.init 执行；其中 Chef.init(self, name) 是"经类取方法"，返回普通函数，所以必须显式传 self——这正是本示例用显式类调用而非 super() 的原因（self 始终指向 pat 这个唯一实例）。pat.work() 查找过程：pat.dict → PizzaRobot.dict（命中）。pat.giveRaise(0.20) 查找过程：pat.dict → PizzaRobot.dict → Chef.dict → Employee.dict（命中）。加薪后 salary 从 50000 变成 60000，f'{...;,.2f}' 格式化输出千分位。

·设计思想：类 = 集合；子类 = 更具体的集合。机器人（机器人是厨师是员工）构成一条清晰的 is-a 链；只要链上每一级只定义自己的差异点，整棵树的行为就自然正确。

31.4 OOP 与组合: "Has-a" (有一个) 关系 (OOP and Composition: "Has-a" Relationships)

英文原文

The notion of composition was introduced in Chapter s 26 and 28. From a programmer's perspective, composition involves embedding other objects in a container object and activating them to implement container methods. To a designer, composition is another way to represent relationships in a problem domain. But, rather than set membership, composition has to do with components—parts of a whole.

Composition also reflects the relationships between parts, called "has-a" relationships. Some OOP design texts refer to composition as aggregation or distinguish between the two terms by using aggregation to describe a weaker dependency between container and contained. In this text, a "composition" simply refers to a collection of embedded objects. The composite class generally provides an interface all its own and implements it by directing the embedded objects.

Now that we've implemented our employees, let's put them in the pizza shop and let them get busy. Our pizza shop is a composite object: it has an oven, and it has employees like servers and chefs. When a custo

mer enters and places an order, the components of the shop spring into action—the server takes the order, the chef makes the pizza, and so on. Example 31-2—file `pizzashop.py`—simulates all the objects and relationships in this scenario.

Example 31-2. `pizzashop.py`

```
python from employees import PizzaRobot, Server #
From Example 31-1 class Customer: def init(self, nam
e): self.name = name def order(self, server): print(self.
name,'orders from', server) def pay(self, server): print
(self.name,'pays for item to', server) class Oven: def b
ake(self): print('oven bakes') class PizzaShop: def init(
self): self.server = Server('Jan') # Embed other objects
self.chef = PizzaRobot('Pat') # A robot named Pat self
.oven = Oven() def order(self, name): customer = Cus
tomer(name) # Activate other objects customer.order
(self.server) # Customer orders from server

self.chef.work() self.oven.bake() customer.pay(self.ser
ver)

if name == 'main': scene = PizzaShop() # Make the co
mposite scene.order('Sue') # Simulate Sue's order
print('...') scene.order('Bob') # Simulate Bob's order
```

The `PizzaShop` class is a container and controller; its constructor makes and embeds instances of the employee classes we wrote in the prior section, as well as an `Oven` class defined here. When this module's self-

test code calls the PizzaShop's order method, the embedded objects are asked to carry out their actions in turn.

Notice that we make a new Customer object for each order, and we pass on the embedded Server object to Customer methods; customers come and go, but the server is part of the pizza shop composite. Also notice that employees are still involved in an inheritance relationship; composition and inheritance are complementary tools.

When we run this module, our pizza shop handles two orders—one from Sue and then one from Bob (overlapping orders with the async coroutines of Chapter 20 is explicitly out of scope here):

```
$ python3 pizzashop.py Sue orders from <Server: name="Jan", salary=40,000.00> Pat makes pizza oven bakes Sue pays for item to <Server: name="Jan", salary=40,000.00>
```

```
... Bob orders from <Server: name="Jan", salary=40,000.00> Pat makes pizza oven bakes Bob pays for item to <Server: name="Jan", salary=40,000.00>
```

Again, this is mostly just a toy simulation, but the objects and interactions are representative of composites at work. As a rule of thumb, classes can represent just about any objects and relationships you can express in a sentence; just replace nouns with classes (e.g., Oven) and verbs with methods (e.g., bake), and you'

ll have a first cut at a design.

中文翻译

组合 (composition) 的概念在第 26 章和第 28 章引入过。从程序员的角度看, 组合就是把其他对象嵌入一个容器对象中, 并激活它们来实现容器的方法。对设计者而言, 组合是表示问题域中关系的另一种方式。但与集合成员关系不同, 组合关乎组件——整体的一部分。组合也反映了部分之间的关系, 称为 "has-a" (有一个) 关系。有些 OOP 设计书籍把组合称作聚合 (aggregation), 或者把两者区分开, 用 "聚合" 描述容器与被含对象之间更弱的依赖。在本书中, "组合" 简单指嵌入对象的集合。组合类 (composite class) 通常提供一套完全属于自己的接口, 并通过指挥 (directing) 被嵌入的对象来实现它。既然我们已经实现了员工, 那就把他们放进披萨店, 让他们忙起来吧。我们的披萨店是一个组合对象: 它有一台烤箱, 也有服务生和厨师这样的员工。顾客进来点单时, 店里的各个组件立刻行动起来——服务生接单, 厨师做披萨, 等等。示例 31-2 (文件 pizzas hop.py) 模拟了这个场景中的所有对象和关系。示例 31-2. pizzashop.py

```
python from employees import PizzaRobot, Server # 来自示例 31-1
class Customer:
    def init(self, name):
        self.name = name
    def order(self, server):
        print(self.name, 'orders from', server)
    def pay(self, server):
        print(self.name, 'pays for item to', server)
class Oven:
    def bake(self):
        print('oven bakes')
class PizzaShop:
    def init(self):
        self.server = Server('Jan') # 嵌入其他对象
        self.chef = PizzaRobot('Pat') # 一个名叫 Pat 的机器人
        self.oven = Oven()
    def order(self, name):
        customer = Cu
```

```
stomer(name) # 激活其他对象customer.order(self.serv
er) # 顾客向服务生下订单self.chef.work() self.oven.bake
() customer.pay(self.server) if name == 'main': scene
= PizzaShop() # 创建这个组合体scene.order('Sue') # 模
拟 Sue 的订单print('...') scene.order('Bob') # 模拟 Bob
的订单 PizzaShop 类是一个容器兼控制器；它的构造器创
建并嵌入了上一节写的员工类的实例，以及在这里定义的 O
ven 类。当这个模块的自测代码调用 PizzaShop 的 order
方法时，被嵌入的对象依次被要求执行自己的动作。注意，
每个订单我们都新建一个 Customer 对象，并把嵌入的 Se
rver 对象传给 Customer 的方法；顾客来来去去，而服务
生是披萨店这个组合体的一部分。还要注意，员工之间仍然
处于继承关系中；组合与继承是互补的工具。运行这个模块
时，我们的披萨店处理两笔订单——先是一笔 Sue 的，再
是一笔 Bob 的（用第 20 章的 async 协程处理并发订单显
然超出本节范围）：$ python3 pizzashop.py Sue order
s from <Server: name="Jan", salary=40,000.00> Pat
makes pizza oven bakes Sue pays for item to <Server
: name="Jan", salary=40,000.00> ... Bob orders from
<Server: name="Jan", salary=40,000.00> Pat makes p
izza oven bakes Bob pays for item to <Server: name=
"Jan", salary=40,000.00> 再说一次，这基本上只是个玩
具模拟，但这些对象与交互很有代表性，展示了组合体如何
运作。一个经验法则：你能用一句话表达的任何对象和关系
，都可以用类来表示——把名词换成类（如 Oven），把动
词换成方法（如 bake），你就有了一个设计初稿。
```

深度理解

·核心概念：组合 = "has-a" (有一个) 关系。PizzaShop 有一个服务生、有一个厨师、有一个烤箱。与继承的"是"不同，组合强调的是"部分-整体"：整体提供自己的接口，内部指挥部分干活。

·底层实现：PizzaShop.init 在创建实例时把 Server、PizzaRobot、Oven 三个对象存进 self.dict (self.server、self.chef、self.oven) 。order() 方法内部逐个调用嵌入对象的方法——这是典型的"接口编排" (orchestration) ：控制器只管调度，具体实现各组件自己负责。

·设计原因：为什么不用继承？因为"披萨店是一个厨师"在语义上不成立。继承表达的是类别归属，组合表达的是拥有关系；用错关系会导致荒谬的类树（让 PizzaShop 继承 Chef 会让店铺拥有"做食物"的能力，却无法拥有多个不同员工）。设计原则：is-a 用继承，has-a 用组合。

·实际问题：注意 Customer 是每个订单临时创建的（顾客来来去去），而 server/chef/oven 是店铺的常驻组件——这体现了对象的生命周期管理：组合体中嵌入对象的生命周期由容器掌控。真实项目（如 Django 的 ORM 模型、游戏引擎的组件系统）都是这个模式的大规模应用。

·初学者误区：一是"万物皆可继承"——初学者倾向于为了复用代码乱用继承，结果类树深不可测；本章后续的混入 (mix-in) 与工厂示例会展示组合/委托才是更稳的设计。二是忘了组合与继承可以共存：Server、PizzaRobot 之间仍是继承关系，而店铺与它们之间是组合关系。

代码分析

·解释器如何处理：`scene.order('Sue')` 的执行路径：① 调用 `PizzaShop.order`（绑定方法，`self` 自动绑定 `scene`）；② 新建 `Customer` 实例；③ `customer.order(self.server)`——注意这里传入的是 `scene.server` 这个实例，`Customer.order` 打印时会触发 `Server` 的 `repr`（继承自 `Employee`），于是输出里出现格式化的 `<Server: name="Jan", salary=40,000.00>`；④ `self.chef.work()` 触发 `PizzaRobot.work`（继承链查找）；⑤ `self.oven.bake()`。整个过程中 `PizzaShop` 只负责把消息传递给组件，组件各自执行。

·设计思想：控制器模式（controller pattern）。`PizzaShop` 对外只暴露 `order` 一个接口，内部细节（谁接单、谁烤、谁收钱）完全封装。这正体现了本章开头说的“封装 = 打包”：调用方不需要知道店铺内部有哪些组件。

31.5 再访流处理器（Stream Processors Revisited）

英文原文

For a composition example that may be a bit more tangible than pizza-making robots, recall the generic data-stream processor function we partially coded in the introduction to OOP in Chapter 26, repeated here for ease:

```
python def processor(reader, converter, writer): while
True: data = reader.read() if not data: break data = c
onverter(data) writer.write(data)
```

Rather than using a simple function here, we might code this as a class that uses composition to do its work in order to provide more structure and support inheritance. Example 31-3, file `streams.py`, demonstrates one way to code the class (it also mutates one method name, `readline`, because we're actually going to run this code here).

Example 31-3. `streams.py`

```
python class Processor: def init(self, reader, writer): s
elf.reader = reader self.writer = writer def process(sel
f): while True: data = self.reader.readline() if not data:
break data = self.converter(data) self.writer.write(dat
a) def converter(self, data): assert False, 'converter mu
st be defined' # Or raise exception
```

This class defines a `converter` method that it expects subclasses to fill in; it's an example of the abstract superclass model we outlined in Chapter 29 (again, more on `assert` in Part VII—it simply raises an exception if its test is false).

Coded this way, `reader` and `writer` objects are embedded within the class instance (composition), and we supply the conversion logic in a subclass rather than passing in a `converter` function (inheritance). The file in Example 31-4, `converters.py`, shows how.

Example 31-4. converters.py

```
python from streams import Processor class Uppercase(Processor): def converter(self, data): return data.upper() if name == 'main': import sys obj = Uppercase(open('trihack.txt'), sys.stdout) obj.process()
```

Here, the Uppercase class inherits the stream-processing loop logic of process (and anything else that may be coded in its superclasses). It needs to define only what is unique about it—the data conversion logic. When this file is run, it makes and runs an instance that reads from the file trihack.txt in the current directory and writes the uppercase equivalent of that file to the stdout stream (which usually means the window you're working in):

```
$ cat trihack.txt hack Hack HACK!
```

```
$ python3 converters.py HACK HACK HACK!
```

To process different sorts of streams, pass in different sorts of objects to the class construction call. Here, we use an output file instead of a stream:

```
$ python3 >> import converters >> scan = converters.Uppercase(open('trihack.txt'), open('trihackup.txt', 'w')) >> scan.process()
```

```
$ cat trihackup.txt HACK HACK HACK!
```

But, as suggested earlier, we could also pass in arbitrary objects coded as classes that define the required input and output method interfaces. Here's a simple

example that passes in a writer class that wraps up the text inside HTML tags—lines are read from a file, run through uppercase conversion, and then printed with HTML tags:

```
$ python3 >> from converters import Uppercase >>
class HTMLize:
    def write(self, line): print(f'<PRE>{line.rstrip()}</PRE>')
>> Uppercase(open('trihack.txt'), HTMLize()).process()
<PRE>HACK</PRE> <PRE>HACK</PRE> <PRE>HACK!</PRE>
```

If you trace through this example's control flow, you'll see that we get both uppercase conversion (by inheritance) and HTML formatting (by composition), even though the core processing logic in the original `Processor` superclass knows nothing about either step.

The processing code only cares that writers have a `write` method and that a method named `converter` is defined; it doesn't care what those methods do when they are called. Such polymorphism and encapsulation of logic are behind much of the power of classes in Python.

As is, the `Processor` superclass only provides a file-scanning loop. In more realistic work, we might extend it to support additional programming tools for its subclasses and, in the process, turn it into a full-blown application framework. Coding such a tool once in a superclass enables you to reuse it in all your program

s. Even in this simple example, because so much is packaged and inherited with classes, all we had to code was the HTML formatting step; the rest was free.

For another example of composition at work, see exercise 9 in "Test Your Knowledge: Part VI Exercises" and its solution in Appendix B; it's similar to the pizza shop example. We've focused on inheritance in this book because that is the main tool that the Python language itself provides for OOP.

But, in practice, composition may be used as much as inheritance as a way to structure classes, especially in larger systems. As we've seen, inheritance and composition are often complementary (and sometimes alternative) techniques. Because composition is a design issue outside the scope of the Python language and this book, though, we'll defer to other resources for more on this topic.

中文翻译

对于一个可能比做披萨的机器人更“落地”的组合示例，回想一下我们在第 26 章 OOP 导论里部分实现过的通用数据流处理函数，这里为了方便再列一遍：`python def processor(reader, converter, writer): while True: data = reader.read() if not data: break data = converter(data) writer.write(data)` 与其在这里用一个简单函数，我们不妨把它写成用组合完成工作的类，以便提供更多结构并支持继承。示例 31-3（文件 `streams.py`）演示了编码这个类的一种方式（它还改了一个方法名 `readline`，因为我们要真的运行这

里的代码)。示例 31-3. streams.py python class Processor: def init(self, reader, writer): self.reader = reader self.writer = writer def process(self): while True: data = self.reader.readline() if not data: break data = self.converter(data) self.writer.write(data) def converter(self, data): assert False, 'converter must be defined' # 或抛异常这个类定义了一个 converter 方法，期待子类来填充；这是我们在第 29 章概述过的抽象超类（abstract superclass）模型的例子（再说一次，assert 的细节在第 VII 部分——如果它的测试为假，就抛异常）。这样编码后，reader 和 writer 对象被嵌入类实例内部（组合），而转换逻辑则由子类提供，而不是传入一个 converter 函数（继承）。示例 31-4 的文件 converters.py 展示了如何做。示例 31-4. converters.py python from streams import Processor class Uppercase(Processor): def converter(self, data): return data.upper() if name == 'main': import sys obj = Uppercase(open('trihack.txt'), sys.stdout) obj.process() 这里的 Uppercase 类继承了 process 的流处理循环逻辑（以及超类中可能编码的任何其他东西）。它只需要定义自己特有的部分——数据转换逻辑。运行这个文件时，它创建并运行一个实例：从当前目录的 trihack.txt 文件读取，把该文件的大写版本写到 stdout 流（通常指你正在使用的那个窗口）：\$ cat trihack.txt hack Hack HACK! \$ python3 converters.py HACK HACK HACK! 要处理不同类型的流，就给类构造调用传入不同类型的对象。这里我们用一个输出文件代替流：\$ python3 >>> import converters >>> scan = converters.Uppercase(open('trihack.txt'), open('trihackup.txt','w')) >>> scan.process()

\$ cat trihackup.txt HACK HACK HACK! 而且，正如前面提到的，我们还可以传入任意用类编码的、定义了所需输入输出方法接口的对象。下面这个简单例子传入一个 writer 类，把文本包进 HTML 标签里——从文件读行、转成大写、再带 HTML 标签打印：\$ python3 >>> from converters import Uppercase >>> class HTMLize: def write(self, line): print(f'<PRE>{line.rstrip()}</PRE>') >>> Uppercase(open('trihack.txt'), HTMLize()).process() <PRE>HACK</PRE> <PRE>HACK</PRE> <PRE>HACK!</PRE> 如果你顺着这个例子的控制流走一遍，会发现我们同时得到了大写转换（靠继承）和 HTML 格式化（靠组合），而原始 Processor 超类中的核心处理逻辑对这两步一无所知。处理代码只关心 writer 有没有 write 方法、有没有一个叫 converter 的方法被定义；它不关心这些方法被调用时做什么。这种多态与逻辑封装，正是 Python 中类的力量源泉。就目前而言，Processor 超类只提供了一个文件扫描循环。在更现实的工作中，我们可能会扩展它，为子类提供额外的编程工具，并在此过程中把它变成一个完整的应用框架（framework）。在超类中编写一次这样的工具，就能在你所有的程序中复用它。即使在这个简单例子里，因为有这么多东西被打包并由类继承，我们只需要编码 HTML 格式化这一步，其余全是免费得来的。组合在实战中的另一个例子，见"Test Your Knowledge: Part VI Exercises"中的练习 9 及其附录 B 的解答；它与披萨店的例子类似。本书之所以聚焦继承，是因为那是 Python 语言本身为 OOP 提供的主要工具。但实际上，作为组织类的方式，组合的使用频率可能和继承一样高，尤其是在大型系统中。正如我们所见，继承和组合常常是互补的（有时也是二选一的）技术

。不过，因为组合是一个超出 Python 语言和本书范围的设计问题，更深入的内容我们要留待其他资源。

深度理解

·核心概念：这是全书最重要的设计样板之一——抽象超类 + 模板方法 (template method) 模式。Processor.process 定义了"循环：读→转换→写"的骨架（算法步骤），把可变的"转换"步骤留成钩子 (hook)，由子类填充。这就是开闭原则（对扩展开放、对修改关闭）的教科书示范。

·底层实现：process 中的 self.converter(data) 是动态分派——解释器沿 self 的类型 (Uppercase) 查找 converter。Uppercase 的构造器 Processor.init 把 reader/writer 存进实例。注意 HTMLize 不是 Processor 的子类，只是一个恰好定义了 write 方法的普通类——这就是鸭子类型：process 只要求 writer 有 write 方法，从不检查类型。

·设计原因：把"函数参数注入"升级为"类 + 继承 + 嵌入"，换来三点：① 处理循环只写一次；② 数据流对象 (reader/writer) 可随意组合；③ 未来可扩展框架。函数版本 (processor(reader, converter, writer)) 把转换逻辑当作参数，类版本把它当作方法覆盖——两者结果相同，但类版本更容易承载状态和额外工具。

·实际问题：converter 里的 assert False 是"抽象方法"的廉价实现——如果子类忘了覆盖，运行到转换步骤就会立即抛 AssertionError，而不是静默出错或报"未实现"。真

实项目中常用 `raise NotImplementedError` 或 `abc.ABC + @abstractmethod` (第 29 章会提到)。本书用 `assert` 只是因为还没讲到异常与 `abc` 模块。

·初学者误区：① 以为“类必须有明确的类型关系才能协作”——实际上 `HTMLize` 与 `Processor` 毫无继承关系也能合作，这正是 Python 与 Java/C++ 的本质差异；② 把 `init` 与“创建对象”混为一谈——`init` 只是初始化钩子，真正分配内存的是 `new`；③ 以为组合只能嵌套类定义——这里嵌入的是任意对象（文件、`sys.stdout`、`HTMLize` 实例），组合的“胶水”是接口而非类型。

代码分析

```
class Processor:
    def __init__(self, reader, writer):
        self.reader = reader          # reader
        self.writer = writer          # writer
    def process(self):
        while True:
            data = self.reader.readline() # readline
            if not data: break            # EOF
            data = self.converter(data)   #
            self.writer.write(data)      # write
    def converter(self, data):
        assert False, 'converter must be defined' #

class Uppercase(Processor):
    def converter(self, data):
        return data.upper()            #
```

·解释器如何处理：`Uppercase(open('trihack.txt'), sys.stdout)` 分两步走：① 调用 `Uppercase.init`——但 `Uppercase` 没定义 `init`，于是沿 MRO 找到 `Processor.init` 执行

，把文件对象与 `sys.stdout` 存入实例；② `obj.process()` 运行时，`self.converter` 的查找链是 `obj.dict` → `Uppercase.dict`（命中）——注意 `self` 是 `Uppercase` 实例，所以永远命中子类版本。文件对象的 `readline()` 返回行字符串；到达文件末尾返回""（空字符串为假值），循环终止。

·设计思想：模板方法 + 组合双管齐下：骨架由超类固定（继承），具体数据源/数据汇由外部注入（组合），变换逻辑由子类定制（多态）。三者 in `Processor` 一个类里自然融合——这也是 Python 中“少即是多”设计的典型。

31.6 为什么你会关心：类与持久化 (WHY YOU WILL CARE: Classes and Persistence)

英文原文

We've explored Python's `pickle` and `shelve` object persistence earlier in this part of the book because it works especially well with class instances. In fact, these tools are often compelling enough to motivate the use of classes in general—by pickling or shelving a class instance, we store both data and logic.

For example, besides allowing us to simulate real-world interactions, the pizza shop classes developed in this chapter could also be used as the basis of a restaurant database. Pickling instances of such classes to a file makes them persistent across Python program

executions:

```
>> from pizzashop import PizzaShop >> shop = PizzaShop() >> shop.chef <PizzaRobot: name="Pat", salary=50,000.00> >> import pickle >> pickle.dump(shop, open('shopfile.pkl','wb'))
```

This stores an entire composite shop object in a file all at once. To bring it back later in another session or program, a single step suffices as well. Objects restored this way retain both state and behavior:

```
>> import pickle >> shop = pickle.load(open('shopfile.pkl','rb')) >> shop.chef <PizzaRobot: name="Pat", salary=50,000.00> >> shop.order('sue') sue orders from <Server: name="Jan", salary=40,000.00> Pat makes pizza oven bakes sue pays for item to <Server: name="Jan", salary=40,000.00>
```

This is just a prototype as is, but we might extend the shop to keep track of inventory, revenue, and so on—saving it to its file after changes would retain its updated state. See the standard-library manual and related coverage in Chapters 9, 28, and 37 for more on pickles and shelves.

中文翻译

我们在本书这一部分前面已经探索过 Python 的 pickle 和 shelf 对象持久化，因为它们与类实例配合得特别好。事实上，这些工具常常足以成为使用类的一般理由——通过 pickling 或 shelving 一个类实例，我们同时存下了数据和逻辑。

辑。例如，除了让我们能模拟现实世界的交互之外，本章开发的披萨店类还可以用作餐厅数据库的基础。把这类类的实例 pickle 到文件里，可以让它们在 Python 程序多次执行之间保持持久：

```
>>> from pizzashop import PizzaShop
>>> shop = PizzaShop()
>>> shop.chef <PizzaRobot: name="Pat", salary=50,000.00>
>>> import pickle
>>> pickle.dump(shop, open('shopfile.pkl','wb'))
```

这一步就把整个组合体 shop 对象一次性存进了文件。之后在另一个会话或程序里把它取回来，也只需一步。这样恢复出来的对象同时保留了状态和行为：

```
>>> import pickle
>>> shop = pickle.load(open('shopfile.pkl','rb'))
>>> shop.chef <PizzaRobot: name="Pat", salary=50,000.00>
>>> shop.order('sue') sue orders from <Server: name="Jan", salary=40,000.00>
Pat makes pizza oven bakes sue pays for item to <Server: name="Jan", salary=40,000.00>
```

就目前而言这只是一个原型，但可以扩展店铺来记录库存、营收等等——每次变更后保存到文件，就能保留更新后的状态。更多关于 pickle 和 shelve 的内容，见标准库手册以及第 9、28、37 章的相关讲解。

深度理解

·核心概念：持久化（persistence）= 让对象在程序运行结束后仍然存在。pickle.dump 把整个对象图（店铺→服务员→厨师→烤箱……）序列化成字节流写进文件；pickle.load 还原。因为实例化类时“数据 + 行为”绑定在一起，还原出来的对象不仅带着数据，还能继续调用方法。

·底层实现：pickle 并不真正序列化代码——它序列化的是实例数据 + 类名的引用（模块路径与类名），加载时重

新导入类，再把数据填充到新实例里。所以要求类所在的模块必须可导入，且类定义不能有大改。

·设计原因：把"模拟"升级为"真实系统"的关键一步：披萨店从玩具变成数据库只需加 pickle。Lutz 借此说明：类的价值不只是组织代码，还在于与语言生态（标准库）协同。

·实际问题：序列化的对象图会递归包含所有嵌入对象（组合体的优势在此体现——一次 dump 全保存）。真实项目注意：① 不要 pickle 不受信任的数据（存在反序列化攻击风险）；② 类结构变更后旧 pickle 文件可能无法加载；③ 现代项目通常改用 JSON/数据库，但 pickle 仍是 Python 内建的快速方案。

·初学者误区：以为 pickle 会把"函数/类定义"也存进文件（其实只存引用）；以为 pickle.load 是万能还原（版本不兼容、模块路径改变都会失败）；不知道 shelve 是"以键访问的 pickle 仓库"，适合存多个命名对象。

31.7 OOP 与委托: "Like-a" (像一个) 关系 (OOP and Delegation: "Like-a" Relationships)

英文原文

Besides inheritance and composition, object-oriented programmers often speak of delegation, which usually implies controller objects that embed other objects

to which they pass off operation requests. The controllers can take care of administrative activities, such as logging or validating accesses, adding extra steps to interface components, or monitoring active instances. In a sense, delegation is a special form of composition, with a single embedded object managed by a proxy (sometimes called a wrapper) class that retains most or all of the embedded object's interface. The notion of proxies sometimes applies to other mechanisms, too, such as function calls; in delegation, we're concerned with proxies for all of an object's behavior, including method calls and other operations. This concept was introduced by example in Chapter 28, and in Python is often implemented with the `getattr` method hook we studied in Chapter 30. Because this operator-overloading method intercepts accesses to nonexistent attributes, a wrapper class can use `getattr` to route arbitrary accesses to a wrapped object. Because this method allows attribute requests to be routed generically, the wrapper class retains the interface of the wrapped object and may add additional operations of its own. By way of review, consider the file `trace.py` in Example 31-5.

```
python class Wrapper:
    def init(self, object):
        self.wrapped = object
    # Save object
    def getattr(self, attrname):
        print('Trace:' + attrname)
    # Trace fetch
    return getattr(self.wrapped, attrname)
    # Delegate fetch
Recall from Chapter 30 that getattr gets the attribute name as a s
```

tring. This code makes use of the `getattr` built-in function to fetch an attribute from the wrapped object by name string—`getattr(X, N)` is like `X.N`, except that `N` is an expression that evaluates to a string at runtime, not a variable. In fact, `getattr(X, N)` is similar to `X.dict[N]`, but the former also performs an inheritance search, like `X.N`, while the latter does not (see Chapters 23 and 29 for more on the `dict` attribute). You can use the approach of this module's wrapper class to manage access to any object with attributes—lists, dictionaries, and even classes and instances. Here, the `Wrapper` class simply prints a trace message on each attribute access and delegates the attribute request to the embedded wrapped object:

```
>>> from trace import Wrapper >>> x = Wrapper([
1, 2, 3]) # Wrap a list >>> x.append(4) # Delegate to
list method Trace: append

>>> x.wrapped # Print my member [1, 2, 3, 4]

>>> x = Wrapper({'a': 1, 'b': 2}) # Wrap a dictionary >
>> list(x.keys()) # Delegate to dictionary method Trace: keys ['a', 'b']
```

The net effect is to augment the entire interface of the wrapped object with additional code in the `Wrapper` class. We can use this to log our method calls, route method calls to extra or custom logic, adapt a class to a new interface, and so on. In the next chapter, we'll revive the notions of wrapped objects and delegate

d operations as one way to extend built-in types. If you are interested in the delegation design pattern, also watch for the discussions in Chapters 32 and 39 of function decorators, a strongly related concept designed to augment a specific function or method call rather than the entire interface of an object, as well as class decorators, which serve as a way to automatically add such delegation-based wrappers to all instances of a class. NOTE — Delegation reminder: As noted in the sidebar "Delegating Built-ins—or Not", general proxies like the Wrapper example here cannot directly intercept and delegate calls to operator-overloading methods run by built-in operations. The list's `add`, for instance, is not caught and fails:

```
>>> [1, 2, 3] + [4, 5] [1, 2, 3, 4, 5]
>>> [1, 2, 3].add([4, 5]) [1, 2, 3, 4, 5]
>>> Wrapper([1, 2, 3]) + [4, 5] TypeError: unsupported operand type(s) for +: 'Wrapper' and 'list'
```

Explicit-name attribute fetches are routed to `getattr`, but built-in operations differ in ways that impact some delegation-based tools. We'll return to this issue and see it live in Chapters 38 and 39, in the context of managed attributes and decorators. For now, keep in mind that delegation proxies may need to redefine operator-overloading methods by code, tools, or subclasses if those methods are used by embedded objects and should be routed to them.

中文翻译

除了继承和组合，面向对象程序员还经常谈论委托（delegation），它通常指控制器对象嵌入其他对象，并把操作请求转交给它们。控制器可以负责管理性活动，比如记录日志、校验访问、给接口组件增加额外步骤，或监控活动实例。从某种意义上说，委托是组合的一种特殊形式：由一个代理（proxy，有时也叫 wrapper 包装类）类管理单个嵌入对象，代理保留被嵌入对象的大部分或全部接口。“代理”的概念有时也适用于其他机制，比如函数调用；而在委托中，我们关心的是对对象所有行为的代理，包括方法调用和其他操作。这个概念在第 28 章通过例子介绍过，在 Python 中通常用我们第 30 章学过的 `getattr` 方法钩子实现。因为这个运算符重载方法会拦截对不存在属性的访问，包装类就能用 `getattr` 把任意的属性访问路由到被包装对象。因为这个方法能让属性请求被通用地路由，包装类便保留了被包装对象的接口，还可以添加自己的额外操作。作为回顾，请看示例 31-5 的文件 `trace.py`。示例 31-5. `trace.py`

```
python class Wrapper:
def init(self, object):
self.wrapped = object
# 保存对象
def getattr(self, attrname):
print('Trace:' + attrname)
# 追踪属性获取
return getattr(self.wrapped, attrname)
# 委托属性获取
```

回顾第 30 章，`getattr` 拿到的是以字符串形式给出的属性名。这段代码用内置函数 `getattr` 按名字字符串从被包装对象取属性——`getattr(X, N)` 就像 `X.N`，只不过 `N` 是运行时求值为字符串的表达式，而不是变量。事实上，`getattr(X, N)` 类似 `X.dict[N]`，但前者像 `X.N` 一样会执行继承搜索，而后者不会（关于 `dict` 属性的更多内容见第 23、29 章）。你可以用这个模块包装类的方法

来管理对任何带属性的对象的访问——列表、字典，甚至类和实例。这里的 Wrapper 类只是在每次属性访问时打印一条追踪消息，然后把属性请求委托给嵌入的 wrapped 对象：

```
>>> from trace import Wrapper >>> x = Wrapper([1, 2, 3]) # 包装一个列表>>> x.append(4) # 委托给列表方法Trace: append >>> x.wrapped # 打印我的成员[1, 2, 3, 4] >>> x = Wrapper({'a': 1, 'b': 2}) # 包装一个字典>>> list(x.keys()) # 委托给字典方法Trace: keys ['a', 'b']
```

净效果是：用 Wrapper 类里的附加代码增强了被包装对象的整个接口。我们可以用它来记录方法调用日志、把方法调用路由到额外或自定义的逻辑、把某个类适配到新接口，等等。下一章我们会重新拾起被包装对象和委托操作的概念，把它作为扩展内置类型的一种方式。如果你对委托设计模式感兴趣，还要留意第 32 章和第 39 章关于函数装饰器 (function decorators) 的讨论——这是一个强相关的概念，用于增强某个特定的函数或方法调用，而不是对象的整个接口；以及类装饰器 (class decorators)，它能把这种基于委托的包装自动添加到类的所有实例上。注意——委托提醒：正如侧边栏“Delegating Built-ins—or Not”（委托内置操作——或者不）指出的，像这里的 Wrapper 这样的通用代理，无法直接拦截并委托由内置操作运行的运算符重载方法。比如列表的 add 就不会被捕获，因而失败：

```
>>> [1, 2, 3] + [4, 5] [1, 2, 3, 4, 5] >>> [1, 2, 3].add([4, 5]) [1, 2, 3, 4, 5] >>> Wrapper([1, 2, 3]) + [4, 5] TypeError: unsupported operand type(s) for +: 'Wrapper' and 'list'
```

显式名字的属性获取会被路由到 getattr，但内置操作的行为方式不同，这会影响到一些基于委托的工具。我们会在第 38、39 章以受管属性和装饰器为背景回到这个问题并现场

演示。目前请记住：如果被嵌入对象使用了运算符重载方法且需要被路由到它们，委托代理可能需要通过代码、工具或超类重新定义这些方法。

深度理解

·核心概念：委托 = "Like-a"（像一个）关系。代理对象"长得像"被包装对象（保留其接口），但偷偷在每次访问前后插入自己的逻辑。这是代理模式/装饰器模式的 Python 形态，也是 AOP（面向切面编程，如日志、鉴权、监控）的朴素实现。

·底层实现：Wrapper 的 `x.append(4)` 查找过程：① `x.dict` 没有 `append`；② `Wrapper.dict` 没有 `append`；③ 触发 `getattr('append')`；④ 打印追踪消息，`getattr(self.wrapped, 'append')` 从真正的列表取到绑定方法并返回；⑤ 调用者随即调用它。注意：`x.wrapped` 本身存在于 `Wrapper.dict` 之外（在实例 `dict` 里），所以能正常取到，不会陷入递归——命名要小心：若包装类也有同名方法，则直接命中普通方法，`getattr` 根本不会被调用。

·设计原因：不改动被包装对象的代码，就能给它的所有方法加上额外行为（日志、计数、鉴权）。相比继承，委托不需要知道被包装对象有哪些方法，天然适配任意对象（列表、字典、文件.....）。

·实际问题：`getattr(X, N)` 会执行完整继承搜索，所以比 `X.dict[N]` 更接近 `X.N` 的语义；`X.dict[N]` 只查实例自己的字典。注意 `getattr` 的"内置操作陷阱"：`x + [4,5]` 不会走 `getattr`（CPython 对内置操作直接按特殊方法名查找类

型槽位)，这是代理模式最著名的坑，第 38 章会展开。

·初学者误区：① 把 `getattr` 和 `getattrattribute` 混淆——前者只在属性找不到时触发，后者每次访问都触发（更慢、更危险）；② 以为 `getattr` 能拦下 `print(x)`、`x + y` 这类内置操作——拦不住；③ 在 `getattr` 里不小心引用不存在的属性会无限递归——务必只委托给 `self.wrapped`。

代码分析

```
class Wrapper:
    def __init__(self, object):
        self.wrapped = object          #
    def __getattr__(self, attrname):
        print('Trace: ' + attrname)    #
        return getattr(self.wrapped, attrname)  #
```

·解释器如何处理：`x.append(4)` 时，Python 先在 `x` 实例的 dict 与 `Wrapper` 类及其父类中查找 `append`——全部未命中，于是调用 `Wrapper.getattr('append')`。函数体里 `getattr(self.wrapped, 'append')` 对真正的列表执行属性查找（列表实例 + `list` 类），返回绑定方法；这个绑定方法再被调用，4 被追加。整个调用链：`Wrapper.getattr` → `getattr` → 真正的 `list.append`，中间只多了一道“打印 + 转发”。

·设计思想：`getattr` 是 Python 最强大的钩子之一——它让“动态接口”成为可能：包装对象未来新增的任何方法，代理都无需改动即可透传。这是“鸭子类型”哲学在系统设计层面的延伸。

31.8 伪私有类属性 (Pseudoprivate Class Attributes)

英文原文

Besides larger structuring goals, class designs often must address name usage too. In Chapter 28's case study, for example, we noted that methods defined within a general tool class might be modified by subclasses if exposed, and noted the trade-offs of this policy—while it supports method customization and direct calls, it's also open to accidental replacements. In Part V, we learned that every name assigned at the top level of a module file is exported. By default, the same holds for classes—data hiding is a convention, and clients may fetch or change attributes in any class or instance to which they have a reference. All attributes are all "public" and "virtual," in C++ terms: they're accessible everywhere and are looked up dynamically at runtime. In fact, it's even possible to change or delete a class's method at runtime, though this is rarely done in practical programs. As a scripting language, Python is about enabling, not restricting. All that being said, Python today does support the notion of name "mangling" (i.e., expansion) to associate some names with their classes. Mangled names are sometimes misleadingly called "private attributes," but really this is just a way to localize a name to the class that creat

ed it—name mangling does not prevent access by code outside the class. This feature is mostly intended to avoid namespace collisions in instances, not to restrict access to names in general; mangled names are therefore better called "pseudoprivate" than "private." Pseudoprivate names are an advanced and entirely optional feature, and you probably won't find them very useful until you start writing general tools or larger class hierarchies for use in multiprogrammer projects. In fact, they are not always used even when they probably should be—more commonly, Python programmers code internal names with a single underscore (e.g., `_X`), which is just an informal convention to let you know that a name shouldn't generally be changed, but this means nothing to Python itself. Because you may see this feature in other people's code, though, you need to be somewhat aware of it, even if you don't use it yourself. And once you learn its advantages and contexts of use, you may find this feature to be more useful in your own code than some programmers realize.

中文翻译

除了更大规模的结构目标，类设计往往还必须处理名称使用问题。例如在第 28 章的案例研究中，我们提到通用工具类中定义的方法如果暴露出来，可能会被子类修改，并指出了这种策略的权衡——它既支持方法定制和直接调用，也容易发生意外替换。在第 V 部分我们学过，模块文件顶层赋

值的每个名字都会被导出。默认情况下，类也是如此——数据隐藏只是一种约定，客户可以获取或修改他们能拿到引用的任何类或实例的属性。用 C++ 的术语说，所有属性都是"公有"且"虚"的：它们到处可访问，在运行时被动态查找。事实上，你甚至可以在运行时改变或删除类的方法，虽然实际程序中很少这样做。作为一种脚本语言，Python 讲究"赋能" (enabling)，而不是"限制" (restricting)。话虽如此，今天的 Python 确实支持名称改写 (name mangling，即扩展) 的概念，把某些名字与它们的类关联起来。被改写的名字有时被误导性地称为"私有属性"，但实际上这只是一个把名字本地化 (localize) 到创建它的类的方式——名称改写并不阻止类外代码的访问。这个特性的主要目的是避免实例中的命名空间冲突，而不是普遍限制名称访问；因此被改写的名字与其叫"私有"，不如叫"伪私有" (pseudoprivate)。伪私有名字是一个高级且完全可选的特性，在你开始为多人项目编写通用工具或更大的类层次之前，你大概不会觉得它有多大用处。事实上，即使在"应该用"的时候，它也不总被使用——更常见的是，Python 程序员用单下划线 (如 X) 来编写内部名字，这只是一个非正式约定，提示你这个名字一般不应被改动，但对 Python 本身毫无意义。不过，因为你可能会在别人的代码里见到这个特性，即使你自己不用，也需要对它有所了解。而且一旦你了解了它的优点和使用场景，你可能会发现它比某些程序员意识到的更有用。

深度理解

·核心概念：Python 没有真正的私有属性，只有伪私有：以双下划线开头且不以双下划线结尾的名字 (如 X) 会被"

改写"成类名X，从而把名字本地化到定义它的类。这只防"意外"不防"恶意"——`l.C1X = 77` 照样能访问。

·底层实现：改写发生在编译期：解释器把类语句体内的 X 标识符（包括 `self.X` 里的属性名）在字节码层面改写成 `ClassNameX`。所以实例 dict 里存储的键是 `C1X` 而非 `X`。这是纯文本级的机械替换，不涉及任何运行时检查。

·设计原因：Python 实例只有一个共享的 dict，多类混合（尤其多重继承）时，不同类的同名 `self.X` 赋值会互相覆盖。改写给每个类"划了地盘"，避免名称冲突。这是"约定优于强制"哲学的折中：既不加限制，又减少事故。

·实际问题：适用场景：① 通用工具类/mixin 的内部方法名（避免与客户子类撞名）；② 多重继承中多个类给同一实例赋值同名属性；③ 防止子类意外重定义内部方法。不适用场景：真正的安全/隐私需求（做不到）。

·初学者误区：① 以为 X 是私有，访问会报错——不会，改个名而已；② 以为 X 适用于"模块级"名字——只在 `class` 语句内生效；③ 把 X（双下划线首尾都有，如 `__init__`，是特殊方法，完全不改写）和 `_X`（只开头双下划线）搞混。

31.9 名称改写概述 (Name Mangling Overview)

英文原文

Here's how name mangling works: within a class statement only, any names that start with two underscor

es but do not end with two underscores are automatically expanded to include the name of the enclosing class at their front. For instance, a name like `X` within a class named `Hack` is changed to `HackX` automatically: the original name is prefixed with a single underscore and the enclosing class's name. Because the modified name contains the name of the enclosing class, it's generally unique; it won't clash with similar names created by other classes in a hierarchy. Name mangling happens only for names that appear inside a class statement's code and then only for names that begin with two leading underscores. It works for every name preceded with double underscores, though—both class attributes (including method names) and instance attribute names assigned to `self`. For example, in a class named `Hack`, a method named `meth` is mangled to `Hackmeth`, and an instance attribute reference `self.X` is transformed to `self.HackX`. Despite the mangling, as long as the class uses the double-underscore version everywhere it refers to the name, all its references will still work. Because more than one class may add attributes to an instance, though, this mangling helps avoid clashes—but we need to move on to an example to see how.

中文翻译

名称改写是这样工作的：仅在 `class` 语句内部，任何以两个下划线开头但不以两个下划线结尾的名字，都会被自动扩展

，在前面加上所在类的名字。例如，名为 Hack 的类内部的名字 X 会被自动改成 HackX：原始名字前面被加上一个下划线和所在类的名字。因为修改后的名字包含所在类的名字，它通常是唯一的；不会与类层次中其他类创建的相似名字冲突。名称改写只作用于出现在 class 语句代码内部的名字，而且只作用于以两个下划线开头的名字。但它对所有带双下划线前缀的名字都有效——包括类属性（含方法名）和赋给 self 的实例属性名。例如，在名为 Hack 的类里，名为 meth 的方法会被改写成 Hackmeth，实例属性引用 self.X 会被转换成 self.HackX。尽管被改写，只要类在引用该名字的每个地方都用双下划线版本，它所有的引用就仍然正常。不过，因为可能有多个类向同一个实例添加属性，这种改写有助于避免冲突——我们需要看个例子来理解具体如何避免。

深度理解

·核心概念：改写的规则一句话：X（类内）→ ClassNameX。两个必要条件：① 在 class 语句内；② 以双下划线开头、不以双下划线结尾。init、str 这类“魔法方法”（首尾双下划线）不受影响。

·底层实现：这是编译器层的名字变换。注意它发生在源码词法分析之后、字节码生成之前：所有对该名字的引用都会被一致替换，因此类内部自治；而类外部代码用原名引用自然找不到（因为实际名字已变）。它不读取运行时值，不检查可见性——纯粹的机械改写。

·设计原因：实例 dict 是共享的“公共内存”，不同类各自“声明拥有”的属性名（如两个类都想用 X）在没有机制隔离

时必然冲突；改写让每个类的属性名带上自己的类名前缀，相当于给共享黑板上的每条记录盖上归属章。

·实际问题：改写也让 `dir`、工具类（如本章后面 `ListInstance` 的 `attrnames`）等方法名不会与客户类的属性冲突；但跨类调用被改写方法时要小心（必须用 `self.ClassNameX` 这种全名）。

·初学者误区：以为 `X` 对所有以开头的名字生效（对 `X` 不生效）；以为改写发生在运行时（实际是编译期）；以为 `X`（单下划线）也有某种“半私有”语义（对 Python 无意义，纯约定）。

31.10 为何使用伪私有属性？（Why Use Pseudoprivate Attributes?）

英文原文

One of the main issues that the pseudoprivate attribute feature is meant to alleviate has to do with the way instance attributes are stored. In Python, instance attributes normally wind up in the single instance object at the bottom of the class tree and are shared by class-level method functions the instance is passed into. This is different from the C++ model, where each class gets its own space for data members it defines. Within a class's method in Python, whenever a method assigns to a self attribute (e.g., `self.attr = value`), it changes or creates an attribute in the instance (recall

that inheritance searches happen only on reference, not on assignment). Because this is true even if multiple classes in a hierarchy assign to the same attribute, collisions are possible. For example, suppose that when a programmer codes a class, it is assumed that the class owns the attribute name `X` in the instance. In this class's methods, the name is set and later fetched:

```
python class C1: def meth1(self): self.X = 88 # I assume X is mine
def meth2(self): print(self.X)
Suppose further that another programmer, working in isolation, makes the same assumption in another class:
```

```
python class C2: def metha(self): self.X = 99 # Me too
def methb(self): print(self.X)
Both of these classes work by themselves. The problem arises if the two classes are ever mixed together in the same class tree, using the multiple inheritance we'll expand on ahead:
```

```
python class C3(C1, C2): ... I = C3() # But only 1 X in I!
```

Now, the value that each class gets back when it says `self.X` will depend on which class assigned it last. Because all assignments to `self.X` refer to the same single instance, there is only one `X` attribute—I.X—no matter how many classes use that attribute name. This isn't a problem if it's expected, and indeed, this is how classes normally communicate—the instance is shared memory. To guarantee that an attribute belongs to the class that uses it, though, prefix the name with `d`

double underscores everywhere it is used in the class, as in Example 31-6, `pseudoprivate.py`. Example 31-6. `pseudoprivate.py`

```
python class C1: def meth1(self): self.X = 88 # Now X
is mine def meth2(self): print(self.X) # Becomes C1X i
n I class C2: def metha(self): self.X = 99 # Me too def
methb(self): print(self.X) # Becomes C2X in I class C3(
C1, C2): pass I = C3() # Two X names in I I.meth1(); I.
metha() # Set names

print(I.dict) # Actual storage I.meth2(); I.methb() # Fet
ch names
```

When thus prefixed, the X attributes will be expanded to include the names of their classes before being added to the instance. If you run a `dir` call on I or inspect its namespace dictionary after the attributes have been assigned, you'll see the expanded names, C1X and C2X, but not X. Because the expansion makes the names more unique within the instance, the classes' coders can be fairly safe in assuming that they truly own any names that they prefix with two underscores:

```
$ python3 pseudoprivate.py {'C1X': 88,'C2X': 99} 88 9
9
```

This trick can avoid potential name collisions in the instance, but note that it does not amount to true privacy. If you know the name of the enclosing class, you can still access either of these attributes anywhere you have a reference to the instance by using the full

y expanded name (e.g., `I.C1X = 77`). Moreover, names could still collide if unknowing programmers use the expanded naming pattern explicitly (unlikely, but not impossible). On the other hand, this feature makes it much less likely that you will accidentally step on a class's names. Pseudoprivate attributes are also useful in larger frameworks or tools, both to avoid introducing new method names that might accidentally hide definitions elsewhere in the class tree and to reduce the chance of internal methods being replaced by names defined lower in the tree. If a method is intended for use only within a class that may be mixed into other classes, the double underscore prefix virtually ensures that the method won't interfere with other names in the tree, especially in multiple-inheritance scenarios:

```
python class Super: def method(self): ... # A real application method
class Tool: def method(self): ... # Becomes Tool.method
def other(self): self.method() # Use my internal method
class Sub1(Tool, Super): ... def actions(self): self.method() # Runs Super.method as expected
class Sub2(Tool): def init(self): self.method = 99 # Doesn't break Tool.method
def method(self): ... # Ditto
```

We met multiple inheritance briefly in Chapter 26 and will explore it in more detail later in this chapter. Recall that superclasses are searched according to their left-to-right order in class header lines. Here, this means `Sub1` prefers `Tool` attributes to those in `Super`.

Although in this example we could force Python to pick the application class's methods first by switching the order of the superclasses listed in the Sub1 class header, pseudoprivate attributes resolve the issue altogether. Pseudoprivate names also prevent subclasses from accidentally redefining the internal method's names, as in Sub2. Again, this feature tends to be of use primarily for larger multiprogrammer projects and then only for selected names. Don't be tempted to clutter your code unnecessarily; only use this feature for names that truly need to be controlled by a single class. Although useful in some general class-based tools, for simpler programs, it's probably overkill. For more examples that make use of the X naming feature, see the `lister.py` mix-in classes introduced later in this chapter's multiple inheritance section, as well as the later class decorators mentioned in the following note.

NOTE — Private matters: If you're interested in more binding forms of privacy, you may want to review the emulation of private instance attributes coded in "Attribute Access: `getattr` and `setattr`" in Chapter 30 and watch for the broader Private class decorator we'll build with delegation in Chapter 39. Although it's possible to add name-access controls in Python classes, this is rarely done in practice—even for large systems that solve real-world problems. Go figure?

中文翻译

伪私有属性特性要缓解的主要问题之一，与实例属性的存储方式有关。在 Python 中，实例属性通常会进入类树底部那唯一的实例对象里，并由该实例传入的类级方法函数共享。这与 C++ 模型不同——C++ 中每个类对自己定义的数据成员都有自己的独立空间。在 Python 的类方法内部，只要方法给 self 属性赋值（如 `self.attr = value`），它就会在实例里修改或创建属性（回忆一下：继承搜索只在引用时发生，不在赋值时发生）。因为即使类层次中多个类给同一个属性赋值，情况也是如此，冲突就不可避免。例如，假设某个程序员编码一个类时，认为这个类“拥有”实例中的属性名 X。在这个类的方法里，这个名字被设置、之后被获取：`python class C1: def meth1(self): self.X = 88 # 我认为 X 是我的``def meth2(self): print(self.X)` 再假设另一位程序员在完全隔离的情况下，在另一个类里做了同样的假设：`python class C2: def metha(self): self.X = 99 # 我也是``def methb(self): print(self.X)` 这两个类各自运行都没问题。问题出现在两个类被混进同一个类树时——也就是用我们稍后要展开的多重继承：`python class C3(C1, C2): ... I = C3()` # 但 I 里只有一个 X！这时，每个类说 `self.X` 时取回的值，取决于哪个类最后给它赋值。因为对 `self.X` 的所有赋值都指向同一个实例，无论多少个类使用这个属性名，都只有一个 X 属性——`I.X`。如果这是预期的行为，那就不是问题；事实上，这本来就是类之间正常通信的方式——实例就是共享内存。但是，要保证某个属性归属于使用它的类，就在该类里使用该名字的所有地方加上双下划线前缀，如示例 31-6 (`pseudoprivate.py`) 所示。示例 31-6. `pseudopri`

vate.py python class C1: def meth1(self): self.X = 88 # 现在 X 是我的了 def meth2(self): print(self.X) # 在 I 里变成 C1X class C2: def metha(self): self.X = 99 # 我也是 def methb(self): print(self.X) # 在 I 里变成 C2X class C3(C1, C2): pass I = C3() # I 里有两个 X 名字 I.meth1(); I.metha() # 设置名字 print(I.dict) # 实际存储内容 I.meth2(); I.methb() # 取回名字加了前缀之后, X 属性在加入实例之前会被扩展, 包含各自类的名字。属性赋值后, 如果你对 I 运行 dir 或检查它的命名空间字典, 你会看到扩展后的名字 C1X 和 C2X, 而不是 X。因为扩展让名字在实例内更唯一, 类的编写者可以相当放心: 他们用双下划线加前缀的任何名字, 真的归自己所有: \$ python3 pseudoprivate.py {'C1X': 88, 'C2X': 99} 88 99 这个技巧可以避免实例中潜在的名字冲突, 但要注意它不等于真正的私有。如果你知道所在类的名字, 只要手里有实例的引用, 你就可以用完整扩展名在任何地方访问这两个属性 (如 I.C1X = 77)。此外, 如果不知情的程序员显式使用扩展命名模式, 名字仍可能冲突 (不太可能, 但并非不可能)。另一方面, 这个特性大大降低了你意外踩到某个类名字的概率。伪私有属性在较大的框架或工具中也很有用: 既可以避免引入可能意外隐藏类树中其他地方定义的新方法名, 也可以降低内部方法被树中更低层定义的名字替换的风险。如果一个方法只打算在可能混入其他类的类内部使用, 双下划线前缀几乎能确保它不会与树中其他名字相互干扰, 尤其是在多重继承场景中: python class Super: def method(self): ... # 真正的应用方法 class Tool: def method(self): ... # 变成 Toolmethod def other(self): self.method() # 使用我的内部方法 class Sub1(Tool, Super): ... def actions(self): self.method() # 按预期

运行 `Super.method` class `Sub2(Tool):` `def init(self): self`
`.method = 99` # 不会破坏 `Tool.method` `def method(self): ...` # 同理我们在第 26 章简单见过多重继承，稍后会在本章更详细地探讨。回顾一下：超类按 `class` 头行中从左到右的顺序被搜索。在这里，这意味着 `Sub1` 优先使用 `Tool` 的属性而非 `Super` 的。虽然在本例中我们可以通过交换 `Sub1` 类头中列出的超类顺序，强迫 Python 先选应用类的方法，但伪私有属性把这个问题彻底解决了。伪私有名字也防止子类意外重定义内部方法名，就像 `Sub2` 那样。再说一次，这个特性主要对大型多人项目有用，而且只用于选定的名字。不要忍不住把代码弄乱；只为真正需要被单个类控制的名字使用这个特性。虽然它在某些通用类工具中 useful，但对更简单的程序来说，可能有点小题大做。更多使用 `X` 命名特性的例子，见本章稍后多重继承一节介绍的 `lister.py` 混入类，以及下面这个注意里提到的后续类装饰器。注意——私有问题（Private matters）：如果你对更强制的私有形式感兴趣，可以回顾第 30 章“Attribute Access: `getattr` and `setattr`”中编码的私有实例属性模拟，并留意我们将在第 39 章用委托构建的更通用的 `Private` 类装饰器。虽然在 Python 类中添加名字访问控制是可能的，但实践中很少这么做——即使是解决真实世界问题的大型系统也不例外。奇不奇怪？

深度理解

·核心概念：实例是共享内存——所有类的方法操作的是同一个实例 `dict`。这是特性不是 bug：类之间借此通信。但当多个类想“独占”同名属性时，就产生了冲突；`X` 改写为每个类开辟私有命名空间。

·底层实现：`self.X = 88` 在编译时被改写为 `self.C1X = 88`；`print(self.X)` 改写为 `print(self.C1X)`。因此 C1 与 C2 虽然都写 X，实际写入实例的键分别是 C1X 与 C2X——`l.dict` 输出证实了这一点。继承搜索在赋值时完全不发生（赋值总是直接进实例 dict），这正是问题根源。

·设计原因：C++ 模型下每个类有独立数据成员空间，天然隔离；Python 把全部状态放在一个实例里，靠名字区分。伪私有是 Python 在“不引入语言级私有”前提下，给共享内存模型装的“保险丝”。

·实际问题：本章后面 `ListInstance` 等通用混入类都用 `__attnames` 这类伪私有名，正是为了避免工具类的方法与客户类属性撞名。多人协作的框架代码里，这个特性几乎是“标准配置”。

·初学者误区：① 认为 X 提供了安全边界——没有，`l.C1X = 77` 仍可写；② 误以为“单下划线 = 私有”——X 对解释器毫无意义，纯提示；③ 在类外或模块层使用 X——不改写，也没有私有语义；④ 用“私有”去隐藏需要跨类调用的方法，反而把自己锁死。

代码分析

```
class C1:
    def meth1(self): self.__X = 88          # : self._C1__X = 88
    def meth2(self): print(self.__X)        # : print(self._C1__X)

class C2:
    def metha(self): self.__X = 99          # : self._C2__X = 99
    def methb(self): print(self.__X)        # : print(self._C2__X)
```

```

class C3(C1, C2): pass                                #
I = C3()                                              # X
I.meth1(); I.metha()                                # _C1__X _C2__X
print(I.__dict__)                                    # {'_C1__X': 88, '_C2__X...'
I.meth2(); I.methb()                                # 88 99

```

·解释器如何处理：CPython 在解析 `class C1` 语句体时，对每个“双下划线开头、非双下划线结尾”的标识符做词法改写。因此 `meth1` 的字节码里出现的是常量 `'C1X'`，执行 `I.meth1()` 时，`self.dict['C1X'] = 88` 直接写入实例字典。`C2` 同理写入 `'C2X'`。两者井水不犯河水，打印互不干扰——而如果用普通 `X`，后者会把前者覆盖（谁最后赋值谁生效）。

·设计思想：伪私有 = “命名空间隔离”而非“访问控制”。它解决的是碰撞问题（collision），不是保密问题（security）。作者反复强调这一点，就是防止读者被“私有”一词误导。

31.11 方法对象：绑定与否 (Method Objects : Bound or Not)

英文原文

Methods in general, and bound methods in particular, simplify the implementation of many design goals in Python. We met bound methods briefly while studying call in Chapter 30. The full story, which we'll flesh out here, turns out to be more general and flexible than

an you might expect. In Chapter 19, we learned how functions can be processed as normal objects. Methods are a kind of object too, and can be used generically in much the same way as other objects—they can be assigned to names, passed to functions, stored in data structures, and so on—and like simple functions, modules, and classes, qualify as first-class objects. Because a class's methods can be accessed from an instance or a class, though, they come in two flavors: Bound methods (绑定方法) : when a method is referenced through an instance — Accessing a function attribute of a class by qualifying an instance returns a bound method object. This object automatically packages the instance with the function as a pair. When a bound method is later called, the instance is automatically passed to the function's self argument. Plain functions (普通函数) : when a method is referenced through a class — Accessing a function attribute of a class by qualifying a class returns a plain function object. To call this function later, you must provide an instance object explicitly to the self argument—if the function expects one. Both kinds of methods are full-fledged objects; they can be transferred around a program at will, just like strings and numbers. Bound methods simply remember an instance, but plain functions do not. This is why we've had to pass in an instance explicitly when calling superclass methods from subclass methods in previous examples (including this chapter

's employees.py in Example 31-1); technically, such calls produce plain functions along the way. When calling a bound method object, though, Python provides the instance argument for us—the instance that was used to create the bound method object. This means that bound method objects are usually interchangeable with simple function objects and makes them especially useful for interfaces originally written for functions. NOTE — Blast from the past: Python once required all methods in a class to have an instance argument and termed methods fetched directly from a class unbound methods—implying that they required an instance argument when later called. Today, method functions in a class are really just plain functions; their only special quality is that they are bound to an instance when fetched through the instance. Unbound methods, however, are dead; long live plain functions!

中文翻译

方法一般而言、绑定方法 (bound method) 尤其而言，简化了 Python 中许多设计目标的实现。我们在第 30 章研究 call 时简单接触过绑定方法。这里我们要讲完整的来龙去脉——它比你可能预期的更通用、更灵活。在第 19 章我们学过，函数可以当作普通对象来处理。方法也是一种对象，可以和其他对象一样被通用地使用——可以赋给名字、传给函数、存进数据结构等等——而且和简单函数、模块、类一样，够格称为一等对象 (first-class object)。不过，因为类的方法既可以通过实例访问、也可以通过类访问，

它有两种形态：绑定方法 (Bound methods)：通过实例引用方法时——通过限定实例来访问类的函数属性，返回的是绑定方法对象。这个对象自动把实例与函数打包成一对。之后调用绑定方法时，实例被自动传给函数的 `self` 参数。普通函数 (Plain functions)：通过类引用方法时——通过限定类来访问类的函数属性，返回的是普通函数对象。之后要调用这个函数，你必须显式地把实例对象传给 `self` 参数——如果函数期望的话。两种方法都是完完整整的对象；它们可以像字符串和数字一样在程序里随意传递。绑定方法只是记住了一个实例，而普通函数不记得。这就是为什么在之前的例子里（包括本章示例 31-1 的 `employees.py`），从子类方法调用超类方法时必须显式传入实例；严格说，这样的调用沿途产生的是普通函数。而调用绑定方法对象时，Python 替我们提供了实例参数——就是当初创建该绑定方法对象的那个实例。这意味着绑定方法对象通常可以与简单函数对象互换使用，这让它们对原本为函数编写的接口格外有用。注意——来自过去的回声 (Blast from the past)：Python 曾要求类中的所有方法都必须有一个实例参数，并把直接从类取出的方法称为非绑定方法 (unbound method) ——暗示它们之后被调用时需要实例参数。如今，类中的方法函数真的就只是普通函数；它们唯一特殊的地方是：通过实例获取时会被绑定到该实例。然而，非绑定方法已死；普通函数万岁！

深度理解

·核心概念：同一个 `def` 定义的方法，通过实例取是绑定方法（自动记住实例），通过类取是普通函数（需要手动传 `self`）。这是理解“为什么 `Employee.init(self, name)` 要

显式传 `self` 的钥匙。

·底层实现：CPython 里，`instance.method` 这种属性访问返回一个 `method`（绑定方法）对象，内部由两个指针构成：`self`（实例）和 `func`（函数）。调用时，解释器把 `self` 自动作为第一个实参传给 `func`。`class.method` 则只是函数对象本身（函数也是描述符，`class` 访问时没有触发绑定）。

·设计原因：绑定方法本质是“闭包式的参数预填”——把实例预先填进第一个参数。这让方法可以当作无参（`self-less`）函数到处传递，例如 GUI 事件回调、`map/sorted` 的 `key` 函数等一切“接口期望函数”的地方。

·实际问题：回调场景中 `self.handler` 比 `lambda: self.handler()` 更简洁且不泄漏——绑定方法已记住 `self`。而 `Hack.doit` 这类“类取方法”调用时必须手传实例，漏传会得到 `TypeError: missing 1 required positional argument: 'self'`。

·初学者误区：① 以为方法必须通过实例调用——类也能调用，只是要手传 `self`；② 写无 `self` 的方法（如 `def doit3(message)`），通过实例调用会报“多传了一个参数”；③ 不理解 `self.method` 和 `method(self, ...)` 是同一回事，在超类调用、回调、装饰器里反复踩坑。

31.12 绑定方法实战 (Bound Methods in Action)

英文原文

To illustrate in simple terms, suppose we define the following class in the REPL of our choosing:

```
>>> class Hack: def doit(self, message): print(message)
Now, in normal operation, we make an instance and call its method in a single step to print the passed-in argument:
```

```
>>> inst = Hack() >>> inst.doit('hello') # Typical method calls hello
```

Really, though, a bound method object is generated along the way—just before the method call's parentheses. In fact, we can fetch a bound method without actually calling it. An `object.name` expression evaluates to an object as all expressions do. In the following, it returns a bound method object that packages the instance (`inst`) with the method function (`Hack.doit`). The net effect is that we can assign this bound method pair to another name and then call it as though it were a simple self-less function:

```
>>> inst = Hack() >>> meth = inst.doit # Bound method object: instance+function
>>> meth('hola') # Same effect as inst.doit('...') hola
```

In fact, if you know where to look, you can see the instance/function pair that the bound method packages:

```
>>> meth <bound method Hack.doit of <main.Hack
object at 0x108f2e8a0>> >>> meth.self <main.Hack
object at 0x108f2e8a0> >>> meth.func <function Ha
ck.doit at 0x108f37ce0>
```

On the other hand, if we qualify the class to get to `doit`, we get back a plain function object with no associated instance. To call this type of method, we usually must pass in an instance as the leftmost argument—there isn't one in the expression otherwise, and the method in this demo expects it:

```
>>> inst = Hack() >>> meth = Hack.doit # Plain function: requires self
>>> meth(inst,'ciao') # Pass in instance (if the method expects one)
ciao
```

This time, though, the method is substantially more mundane because it's a plain function:

```
>>> meth <function Hack.doit at 0x108f37ce0>
```

By extension, the same rules apply within a class's method if we reference `self` attributes that name functions in a class. A `self.method` expression is a bound method object because `self` is an instance object, though a `class.method` is a simple function that may require a `self` when run:

```
>>> class Hack2(Hack):
def doit2(self):
    meth = self.doit # Bound method object: instance+function
    meth('bonjour') # Looks like a simple function
    meth = Hack.doit
    meth(self,'privit') # Plain function: requires self
```



```
>>> Hack2().doit2() bonjour privit
```

Most of the time, you call methods immediately after fetching them with attribute qualification, so you don't notice the method objects generated along the way. But if you start writing code that calls objects generically, you need to be careful to treat nonbound methods specially—they normally require an explicit instance object to be passed in. Implied by this model, classes can also code methods that do not require a self instance, and are called normally when fetched from their class; apart from inheritance, this is similar to a function in a module file:

```
>>> class Hack3: def doit3(message): # A self-less method (function)
print(message) >>> Hack3.doit3('guten tag') # No "self" is passed
guten tag
```

This falls down, though, if we try to call such a self-less method function through an instance: because the bound method made along the way packages and passes an instance, the call sends one too many arguments:

```
>>> x = Hack3() >>> x.doit3('namaste')
TypeError: Hack3.doit3() takes 1 positional argument but 2 were given
```

In other words, you can code self-less functions in a class, and can call them normally through a class without an instance. To call them through an instance, too, though, you'll have to stay tuned for the next chapter.

er's coverage of static and class methods—methods marked specially to suppress an automatic instance argument in all contexts. Also in the next chapter, you'll see that the super built-in binds an instance with a method, too, but is substantially more convoluted than the bound method pairs we've met here. Finally, to demo how flexible bound methods can be, the following stores four of them in a list and calls them generically, with normal call expressions:

```
>>> class Number:
    def init(self, base):
        self.base = base
    def double(self):
        return self.base * 2
    def triple(self):
        return self.base * 3
>>> x, y, z = Number(2), Number(3), Number(4) # Class instance objects
>>> x.double() # Normal immediate calls
4

>>> acts = [x.double, y.double, z.double, z.triple] # List of bound methods
>>> for act in acts: # Calls are deferred
    print(act(), end=" ") # Call as though functions
4 6 8 12
```

In the end, calls to act here run methods in the class to process instances—both saved previously in bound methods.

中文翻译

为了简单说明，假设我们在自己选择的 REPL 里定义下面这个类：

```
>>> class Hack:
    def doit(self, message):
        print(message)
```

现在，正常操作下，我们创建一个实例并一步调用它的方法来打印传入的参数：

```
>>> inst = Hack()
>>>
```

inst.doit('hello') # 典型的方法调用hello 但事实上，就在方法调用的小括号之前，一个绑定方法对象已经被生成。其实我们可以只取出绑定方法而不真正调用它。object.name 表达式和所有表达式一样会求值为一个对象。下面它就返回一个绑定方法对象，把实例 (inst) 和方法函数 (Hack.doit) 打包在一起。净效果是：我们可以把这个绑定方法对赋给另一个名字，然后像调用一个简单的无 self 函数那样调用它：

```
>>> inst = Hack()
>>> meth = inst.doit # 绑定方法对象：实例 + 函数
>>> meth('hola') # 效果等同于 inst.doit('...') hola
```

事实上，如果你知道往哪儿看，就能看到绑定方法打包的那对“实例/函数”：

```
>>> meth <bound method Hack.doit of <main.Hack object at 0x108f2e8a0>
>>> meth.self <main.Hack object at 0x108f2e8a0>
>>> meth.func <function Hack.doit at 0x108f37ce0>
```

另一方面，如果我们限定类去取 doit，得到的是没有任何关联实例的普通函数对象。要调用这类方法，我们通常必须把实例作为最左边的参数传入——否则表达式里根本没有实例，而这个演示里的方法又确实需要它：

```
>>> inst = Hack()
>>> meth = Hack.doit # 普通函数：需要 self
>>> meth(inst,'ciao') # 传入实例（如果方法期望一个）ciao
```

不过这一次，这个方法就平淡多了，因为它是个普通函数：

```
>>> meth <function Hack.doit at 0x108f37ce0>
```

推而广之，在类的方法内部，如果我们引用命名了类中函数的 self 属性，同样规则也适用。self.method 表达式是绑定方法对象，因为 self 就是一个实例对象；而 class.method 是简单函数，运行时可能需要 self：

```
>>> class Hack2(Hack):
def doit2(self):
    meth = self.doit # 绑定方法对象：实例 + 函数
    meth('bonjour') # 看起来像简单函数meth
```

= Hack.doit meth(self,'privit') # 普通函数：需要 self >
>> Hack2().doit2() bonjour privit 大多数时候，你通过属性限定取到方法后立刻调用，所以注意不到沿途生成的方法对象。但如果你开始编写通用地调用对象的代码，就需要小心地把非绑定方法区别对待——它们通常需要显式传入实例对象。这个模型还暗示，类里也可以编写不需要 self 实例的方法，从它们的类上取用时可以正常调用；除了继承，这类似模块文件里的函数：>>> class Hack3: def doit3(message): # 一个无 self 的方法（函数） print(message) >>> Hack3.doit3('guten tag') # 没有传 "self" guten tag 但是，如果我们试图通过实例调用这种无 self 的方法函数，就会失败：因为沿途生成的绑定方法会打包并传入一个实例，调用就会多出一个参数：>>> x = Hack3() >>> x.doit3('namaste') TypeError: Hack3.doit3() takes 1 positional argument but 2 were given 换句话说，你可以在类里编写无 self 的函数，并通过类正常调用、无需实例。但若想也通过实例调用它们，你得等到下一章学习静态方法（static method）和类方法（class method）——它们是专门标记的、在所有上下文里都抑制自动实例参数的方法。下一章你还会看到，super 内建函数也会把实例与方法绑定，但它比我们这里遇到的绑定方法对复杂得多。最后，为了演示绑定方法有多灵活，下面把四个绑定方法存进列表，用普通的调用表达式通用地调用它们：>>> class Number: def init(self, base): self.base = base def double(self): return self.base 2 def triple(self): return self.base 3 >>> x, y, z = Number(2), Number(3), Number(4) # 类实例对象 >>> x.double() # 正常的即时调用 4 >>> acts = [x.double, y.double, z.double, z.triple] # 绑定方法列

表>>> for act in acts: # 调用被延迟print(act(), end='')
像函数一样调用4 6 8 12 最终，这里对 act 的调用运行了类中的方法去处理实例——两者都早已保存在绑定方法里了。

深度理解

·核心概念：绑定方法 = （实例，函数）二元组，是延迟调用的载体：先捕获"谁 + 做什么"，稍后再执行。这使方法可以当作普通函数对象存储、传递、回调。

·底层实现：CPython 中类里的函数是描述符（descriptor）。inst.doit 触发 doit.get(inst, Hack)，返回绑定方法（内部记 self 和 func）；Hack.doit 触发 doit.get(None, Hack)，返回原函数。绑定方法调用时，meth('hola') 被解释为 meth.func(meth.self, 'hola')——这就是"自动传 self"的本质。

·设计原因：一等对象（函数、方法、类、模块可随意传递）是 Python 回调式编程（GUI、事件、异步）的基石。若无绑定方法，事件系统就得用闭包或全局变量携带实例状态，繁琐且易错。

·实际问题：回调场景（如 tkinter 的 command=self.handler）：事件系统调用时不传任何参数，绑定方法恰好补齐了 self。而 Hack3.doit3 这种无 self 方法通过实例调用会报"2 个参数给了 1 个形参"——下一章的 @static method 解决此问题。

·初学者误区：① 混淆 inst.doit（绑定方法对象）与 inst.doit()（调用结果）——前者不执行函数体；② 以为 Hac

k.doit(inst, ...) 是错误写法——这是合法且常见的（超类调用就是这种模式）；③ 不知道 meth.self/meth.func 可以拆包检查绑定方法。

代码分析

```
class Number:
    def __init__(self, base):
        self.base = base
    def double(self):
        return self.base * 2
    def triple(self):
        return self.base * 3

x, y, z = Number(2), Number(3), Number(4)      #
acts = [x.double, y.double, z.double, z.triple] #
for act in acts:                                #
    print(act(), end=' ')                       # 4 6 8 12
```

·解释器如何处理：x.double 在列表构造时就被求值：实例 x 与 double 函数被包成绑定方法存入列表——此刻没有任何函数体被执行。for 循环里 act() 才真正调用：解释器把 act.self（即 x 或 y 或 z）自动作为第一个参数传入 act.func。于是同一段循环代码处理了三个不同实例的数据，输出 4 6 8 12。

·设计思想：把“调用”与“调用时机”解耦。这种“取方法、存起来、稍后调用”的模式，在事件回调、命令模式（Command pattern）、任务队列里随处可见——也是函数式风格（一等函数）与 OOP 风格的接口转换枢纽。

31.13 为什么你会关心：绑定方法回调 (WHY YOU WILL CARE: Bound Method Callbacks)

英文原文

Because bound methods automatically pair an instance with a class's method function, you can use them anywhere a simple function is expected. One of the most common places you'll see this idea put to work is in code that registers methods to handle event callbacks run by GUI interfaces, like Python's tkinter standard-library module. As review, here's the simple case: `python def handler(): ...use globals or closure scopes for state... widget = Button(text='Tap', command=handler)` With tkinter, to register a handler for button click events, we usually pass a callable object that takes no arguments to the `command` keyword argument. Function names (and lambdas) work here, and so do class-level methods—though they must be bound methods if they expect an instance when called: `python class MyGui: def handler(self): ...use self.attr for state... def makewidgets(self): b = Button(text='Tap', command=self.handler)` Here, the event handler is `self.handler`—a bound method object that remembers both `self` and `MyGui.handler`. Because `self` will refer to the original instance when `handler` is later invoked on events, the method will have access to instance attrib

utes that can retain state between events, as well as class-level methods. With simple functions, state normally must be retained in global variables or enclosing function scopes (a.k.a. closures) instead. See also the discussion of call operator overloading in Chapter 30 for another way to make classes compatible with function-based APIs, as well as lambda in Chapter 19 for another tool often used in callback roles. As noted in the former of these, you don't need to wrap a bound method in a lambda unless extra arguments must be added to the call; because the bound method in the preceding example already defers the call (there are no parentheses to trigger one!), adding a lambda here would otherwise be pointless.

中文翻译

因为绑定方法自动把实例和类的方法函数配对，你可以在任何期望简单函数的地方使用它们。你最容易看到这个想法发挥作用的地方之一，是注册方法处理 GUI 界面运行的事件回调的代码，比如 Python 标准库的 tkinter 模块。作为回顾，先看简单的情况：python def handler(): ...用全局变量或闭包作用域保存状态... widget = Button(text='Tap', command=handler) 使用 tkinter 时，要为按钮点击事件注册处理器，我们通常把不带参数的可调用对象传给 command 关键字参数。函数名（和 lambda）在这里都能用，类级方法也行——不过如果它们被调用时需要一个实例，就必须是绑定方法：python class MyGui: def handler(self): ...用 self.attr 保存状态... def makewidgets(self): b

= Button(text='Tap', command=self.handler) 这里的事件处理器是 self.handler——一个同时记住了 self 和 MyGui.handler 的绑定方法对象。因为事件发生时调用 handler 时, self 会指向最初的实例, 所以方法既能访问实例属性 (这些属性可以在多个事件之间保留状态), 也能访问类级方法。而用简单函数时, 状态通常只能保存在全局变量或外层函数作用域 (即闭包, closure) 里。另外可以看看第 30 章关于 call 运算符重载的讨论——那是让类兼容基于函数的 API 的另一种方式; 还有第 19 章的 lambda——它是回调角色中另一个常用工具。正如前者所述, 除非调用时需要添加额外参数, 否则没必要把绑定方法包在 lambda 里; 因为前面例子中的绑定方法已经延迟了调用 (没有括号触发调用!), 在这里再加 lambda 纯属多此一举。

深度理解

·核心概念: 事件回调 = "框架在某个时刻调用你注册的 callable 对象"。绑定方法让"实例状态"随回调自然携带, 免去全局变量。

·底层实现: command=self.handler 只是把绑定方法对象存进 Button 控件内部, 不做调用。用户点击时, tkinter 内部执行 command()——无参调用——绑定方法自动补上 self。

·设计原因: GUI 的每个事件对应"哪个窗口、哪个方法、什么状态", 绑定方法把三者打包成一个对象, 正是为这种"接口期望无参函数"的场景量身定制。相比之下, lambda: self.handler(x) 仅在需要传额外参数时才必要。

·实际问题：这是面向对象 GUI (tkinter、PyQt 等) 的标准套路：界面上每个控件的 `command` 都指向 `self.某方法`，界面状态全部放实例属性。若用普通函数，状态只能退到全局变量或闭包——可维护性差。

·初学者误区：① 写 `command=self.handler()` 带括号——这会在注册时立刻执行方法，回调时什么都不发生；② 用 `lambda` 多包一层——多余；③ 以为 `command` 只接受函数——任何可调用对象（绑定方法、带 `call` 的实例）都行。

31.14 类也是对象：通用对象工厂 (Classes Are Objects: Generic Object Factories)

英文原文

Sometimes, class-based designs require objects to be created in response to conditions that can't be predicted when a program is written. The factory design pattern allows such a deferred approach. Due in large part to Python's flexibility, factories can take multiple forms, some of which don't seem special at all. Because classes are also first-class objects (in the Chapter 19 sense), it's easy to pass them around a program, store them in data structures, and so on. You can also pass classes to functions that generate arbitrary kinds of objects; such functions are sometimes called factories in OOP design circles. Factories can be a major

undertaking in a statically typed language such as C++ but are almost trivial to implement in Python. For example, the call syntax we studied in Chapter 18 can call any class with any number of positional or keyword constructor arguments in one step to generate any sort of instance. Example 31-7 demos the underlying code. Example 31-7. factory.py

```
python def factory(aClass, pargs, kargs): # Varargs tuple, dict
    return aClass(pargs, kargs) # Call aClass class
Hack: def doit(self, message): print(message)
class Person:
    def init(self, name, job=None):
        self.name = name
        self.job = job
object1 = factory(Hack) # Make a Hack object
object2 = factory(Person, 'Sue', 'dev') # Make a Person object
object3 = factory(Person, name='Bob') # Ditto, with keywords and default
This code's factory is passed a class object, along with zero or more arguments for the class's constructor. When called, it uses star syntax to collect and unpack arguments and calls the class to return an instance. Really, factory can invoke any callable object, including functions, classes, and methods, but we're using it for classes here. The rest of the example simply defines two classes and generates instances of both by passing them to the factory function. And that's the only factory function you may ever need to write in Python; it works for any class and any constructor arguments. If you run this example live, your objects will look like this:
```

```
>>> from factory import >>> object1.doit(99) 99
>>> object2.name, object2.job ('Sue','dev')
>>> object3.name, object3.job ('Bob', None)
```

By now, you should know that everything is a first-class object in Python—including classes, which are usually just compiler input in languages like C++. It's natural to pass them around this way. As mentioned at the start of this part of the book, though, only objects derived from classes do full OOP in Python.

中文翻译

有时候，基于类的设计需要在程序编写时无法预知的条件下创建对象。工厂（factory）设计模式就允许这种延迟（deferred）方案。很大程度上得益于 Python 的灵活性，工厂可以有多种形态，其中一些看起来根本没什么特别的。因为类也是一等对象（第 19 章意义上的），把它们在程序里传来传去、存进数据结构等都轻而易举。你也可以把类传给生成任意类型对象的函数；这种函数在 OOP 设计圈子里有时被称为工厂。在 C++ 这样的静态类型语言里，工厂可能是个大工程，但在 Python 里几乎微不足道。例如，我们第 18 章学过的调用语法，可以用任意数量的位置参数或关键字构造参数，一步调用任意类，生成任意类型的实例。示例 31-7 演示了底层代码。示例 31-7. factory.py

```
python def
factory(aClass, pargs, kargs): # 变长参数元组、字典
    return aClass(pargs, kargs) # 调用 aClass
class Hack:
    def doit(self, message):
        print(message)
class Person:
    def init(self, name, job=None):
        self.name = name
        self.job =
```

job object1 = factory(Hack) # 创建一个 Hack 对象
object2 = factory(Person, 'Sue', 'dev') # 创建一个 Person 对象
object3 = factory(Person, name='Bob') # 同上，用关键字和默认值
这段代码的 factory 接收一个类对象，以及零个或多个传给该类构造器的参数。被调用时，它用星号语法收集并解包参数，然后调用类返回一个实例。实际上，factory 可以调用任何可调用对象，包括函数、类和方法，只不过我们这里用来调类。示例其余部分只是定义了两个类，并把它们传给 factory 函数来生成两者的实例。这大概就是你在 Python 里唯一需要写的工厂函数了；它对任何类和任何构造参数都通用。如果你实际运行这个例子，你的对象看起来会是这样：

```
>>> from factory import *
>>> object1.doit(99)
99
>>> object2.name, object2.job
('Sue', 'dev')
>>> object3.name, object3.job
('Bob', None)
```

到现在你应该知道，Python 里一切都是一等对象——包括类。而在 C++ 这样的语言里，类通常只是编译器输入。像这样传来传去是很自然的。不过，正如本书这一部分开头提到的，Python 中只有从类派生出来的对象才做完整的 OOP。

深度理解

·核心概念：工厂模式 = "把'创建什么'的决定权推迟到运行时"。工厂函数接收类对象作为参数，返回该类的实例——创建逻辑与使用逻辑解耦。

·底层实现：aClass(pargs, kargs) 是通用调用语法：pargs 把位置参数收集成元组再展开传入，kargs 同理处理关键字参数。aClass 是类时，调用类对象 = 调用其元类 (type) 的 call → 分配内存 (new) → 运行 init。类能当参

数传递，因为类本身是 type 的实例。

·设计原因：C++ 需要虚构造器、工厂类、注册表等一整套机制才能实现"运行时决定创建哪个类"；Python 把类当作对象传递，一行 `return aClass(...)` 搞定。这是动态语言的降维打击。

·实际问题：真实场景：配置文件里写字符串 `"streamtypes.FileReader"`，程序用 `getattr` 取类再 factory 实例化——类在写代码时可能还不存在（第三方插件）。`getattr(module, name)` 是"按字符串取类"的关键工具。

·初学者误区：① 以为"工厂"必须是类（如 Java 的 `Factory` 类）——Python 里一个函数足矣；② 忘记 `args, kwargs` 的收集/展开两阶段语义；③ 以为类实例化只有 `init`——实际上 `type.call` → `new` → `init` 三步。

代码分析

```
def factory(aClass, *pargs, **kargs):    # /
    return aClass(*pargs, **kargs)      #

class Person:
    def __init__(self, name, job=None): #
        self.name = name
        self.job = job

object2 = factory(Person, 'Sue', 'dev') #
object3 = factory(Person, name='Bob')   # job None
```

·解释器如何处理：调用 `factory(Person, 'Sue', 'dev')` 时，`aClass=Person`、`pargs=('Sue', 'dev')`、`kargs={}`；`return Person(pargs, kargs)` 把元组展开为 `Person('Sue', 'dev')`。调用 `factory(Person, name='Bob')` 时 `kargs={'n`

ame':'Bob'}}, 展开为 Person(name='Bob'), job 落到默认值 None。两条路径最终都生成 Person 实例。

·设计思想：一个通用工厂通吃"任何类 + 任何参数"。作者称之为"你唯一需要写的工厂函数"——因为它把 Python 的动态特性（一等类、变长参数、运行时调用）浓缩成三行。

31.15 为什么需要工厂？ (Why Factories?)

英文原文

So, what good is the factory function (besides providing an excuse to illustrate first-class class objects in this book)? Unfortunately, it's difficult to show applications of this design pattern without listing much more code than we have space for here. In general, though, such a factory might allow code to be insulated from the details of dynamically configured object construction. For instance, recall the Processor class presented as a composition demo earlier in Example 31-3. It accepts reader and writer objects for processing arbitrary data streams. The original abstract version of this example in Chapter 26 manually passed in instances of specialized classes like FileWriter and SocketReader to customize the data streams being processed; later, we passed in hardcoded file, stream, and formatter objects. In a more dynamic scenario, external devices such as configuration files or GUIs might be used

to configure the streams. In such a dynamic world, we might not be able to hardcode the creation of stream interface objects in our scripts but might instead create them at runtime according to the contents of a configuration file. Such a file might simply give the string name of a stream class to be imported from a module, plus an optional constructor call argument. Factory-style functions or code might come in handy here because they would allow us to fetch and pass in classes that are not hardcoded in our program ahead of time. Indeed, those classes might not even have existed at all when we wrote our code. Hypothetically:

```
python classname = ...parse from config file... classarg = ...parse from config file... import streamtypes # Customizable code  
aclass = getattr(streamtypes, classname) # Fetch from module  
reader = factory(aclass, classarg) # Or aclass(classarg)  
processor(reader, ...)
```

Here, the `getattr` built-in is again used to fetch a module attribute given a string name (it's like saying `obj.attr`, but `attr` is a string). This code snippet doesn't strictly need `factory`—it could make an instance with just a `class(classarg)`. A separate function, though, may prove more useful when extra work is required at instance creation time, such as caching objects for reuse. However they are coded, factories are almost trivial with Python's dynamic typing and universal first-class object model.

中文翻译

那么 factory 函数到底有什么用（除了给本书提供一个展示一等类对象的借口）？遗憾的是，要展示这个设计模式的应用而不列出一大堆超出我们篇幅的代码，是很困难的。不过总的来说，这样的工厂可以让代码与动态配置的对象构造细节绝缘（insulated）。例如，回想一下示例 31-3 中作为组合演示的 Processor 类。它接受 reader 和 writer 对象来处理任意数据流。第 26 章这个例子的原始抽象版本，是手动传入 FileWriter、SocketReader 等专门类的实例来定制被处理的数据流；后来我们又传入了硬编码的文件、流和格式化对象。在更动态的场景中，可以用配置文件或 GUI 之类的外部设备来配置数据流。在这样一个动态的世界里，我们可能无法在脚本里硬编码流接口对象的创建，而要根据配置文件的内容在运行时创建它们。这样的文件可能只是给出要从某个模块导入的流类的字符串名字，外加一个可选的构造调用参数。工厂式函数或代码在这里就会派上用场，因为它们让我们能取出并传入那些没有预先硬编码在程序里的类。事实上，我们写代码的时候，那些类可能根本还不存在。假设性地：python classname = ...从配置文件解析... classarg = ...从配置文件解析... import streamtypes # 可定制的代码aclass = getattr(streamtypes, classname) # 从模块中取出reader = factory(aclass, classarg) # 或 aclass(classarg) processor(reader, ...) 这里又用到了内置函数 getattr，按字符串名字取模块属性（就像写 obj.attr，只是 attr 是字符串）。这段代码其实并不严格要求 factory——只用 aclass(classarg) 也能创建实例。不过，当实例创建时需要额外工作（比如缓存对象以便复用）时，单独的函数

可能更有用。不管怎么编码，在 Python 的动态类型和通用一等对象模型下，工厂几乎微不足道。

深度理解

·核心概念：工厂的实战价值 = 插件化/可配置化：程序运行时才知道要创建哪个类。配置文件（或数据库、GUI）提供类名字符串，程序用 `getattr` 动态取类并实例化。

·底层实现：`import streamtypes` 执行模块获得模块对象；`getattr(streamtypes, classname)` 在模块命名空间里按字符串键查找，返回类对象；随后 `factory` 调用它。整条链路没有一处使用编译期类型，全靠运行时名称解析。

·设计原因：把“哪些类可用”从代码里剥离出来，交给部署者配置——这正是依赖注入（`dependency injection`）、策略模式、插件架构的核心。写代码时类可能不存在（第三方扩展），程序无需修改即可适配。

·实际问题：`getattr` 是“字符串 → 属性/类”的桥梁；`importlib.importmodule`（第 31 章稍后 `testmixin.py` 里会用到）是“字符串 → 模块”的桥梁。两者组合可实现完整反射式插件加载。缺点：拼写错误只能在运行时暴露（属性不存在抛 `AttributeError`）。

·初学者误区：① 以为 `getattr(obj, 'name')` 只能取属性——取类、取函数、取方法都行；② 直接 `obj['name']` 取属性——那是字典语法，对象要 `getattr`；③ 以为工厂模式必须引入抽象工厂/工厂方法等重量级结构——Python 中函数工厂就够了。

31.16 多重继承与 MRO (Multiple Inheritance and the MRO)

英文原文

Our last design pattern is one of the most useful and will serve as a subject for more realistic examples to wrap up this chapter. As a bonus, the code we'll write here may be useful tools. Most of our examples so far have used single inheritance—class trees in which each class has just one superclass. This suffices for simple hierarchies and enables customization. In our pizza shop demo of Example 31-1, for example, each worker belonged to just one category and inherited names from only one branch of the class tree. Abstractly:

```
python class B: ... class A(B): ... I = A() # Use attributes from I, A, and B - in that order
```

Many class-based designs, however, call for combining disparate sets of methods. As we've seen, in a class statement, more than one superclass can be listed in parentheses in the header line. When you do this, you leverage multiple inheritance—the class and its instances inherit names from all the listed superclasses:

```
python class C: ... class B: ... class A(B, C): ... I = A() # Use attributes from I, A, B, and C - in that order
```

In general, multiple inheritance is good for modeling objects that belong to more than one set. For instance, a person may be an engineer, a writer, a musician, and so on and inherit proper

ties from all such sets. With multiple inheritance, objects obtain the union of the behavior in all their superclasses. As you'll see ahead, multiple inheritance also allows classes to function as general packages of mixable attributes known as mix-in classes. Although it's a useful tool, multiple inheritance adds another dimension to attribute inheritance: Single inheritance searches only depth first—from instance, to class, and to each superclass from lowest to highest. This order is determined by the class from which an instance is made and the superclass listed in parentheses in each class statement's header (which is mirrored by class objects' bases attributes). Multiple inheritance also searches left to right—according to the order of classes listed in parentheses in class headers. This is a nested component, pursued only after branches further to the left have reached the top of the tree and have been exhausted of their own right-branch candidates. We call the combination of these two DFLR, for depth first, and then left to right. This search suffices when an attribute name shows up in just one branch of a class tree, which is a typical case. When the same name appears in multiple branches, though, DFLR alone is subpar: a lower (and hence more specialized) subclass to the right isn't able to redefine a name in one of its higher (and hence more general) superclasses reached through a branch to the left. Because of this, multiple inheritance requires a slightly different search order k

known as MRO, for method resolution order (though it's used for all attributes, not just methods). In brief, MRO order works the same as DFLR in typical trees but proceeds across by tree levels before moving up in a more breadth-first fashion when multiple classes in a tree share a common superclass, forming what's called a diamond pattern—after the tree's square-on-its-corner shape. This MRO search order is run for all class trees but differs from DFLR when multiple-inheritance diamonds are present. It's designed to visit a common superclass just once, and after all its subclasses. While user-defined classes don't often form diamonds in Python, the built-in object class automatically added above all root (topmost) classes makes every multiple-inheritance tree a diamond; without MRO, object's defaults may hide user-defined versions. To illustrate what diamonds are all about and how MRO search runs across before up in this one somewhat atypical case, let's turn to some code.

中文翻译

我们的最后一个设计模式是最有用的模式之一，也将作为本章收尾的更现实示例的主题。额外的好处是，我们在这里写的代码可能成为有用的工具。到目前为止，我们的大多数示例都用了单继承（single inheritance）——每个类只有一个超类的类树。这对简单层次结构够用，也能支持定制。比如在示例 31-1 的披萨店 demo 中，每个员工只属于一个类别，只从类树的一个分支继承名字。抽象地说：python

class B: ... class A(B): ... I = A() # 依次使用 I、A、B 中的属性然而，许多基于类的设计需要组合不同来源的方法集合。正如我们所见，在 class 语句中，头行的括号里可以列出多个超类。这样做时，你就用上了多重继承（multiple inheritance）——类和它的实例从所有列出的超类继承名字：

```
python class C: ... class B: ... class A(B, C): ... I = A() #
```

依次使用 I、A、B、C 中的属性一般来说，多重继承适合建模属于多个集合的对象。比如，一个人可以既是工程师、又是作家、还是音乐家等等，并从所有这些集合继承属性。有了多重继承，对象获得所有超类行为的并集（union）。你稍后会看到，多重继承还允许类充当可混合属性的通用包，也就是所谓的混入类（mix-in class）。虽然这是个有用的工具，但多重继承给属性继承增加了一个新的维度：单继承只做深度优先（depth first）搜索——从实例到类，再到每个超类，由低到高。这个顺序由实例从哪个类创建、以及每个 class 语句头行括号里列出的超类决定（这也反映在类对象的 bases 属性上）。多重继承还要从左到右（left to right）搜索——按 class 头行括号里列出类的顺序。这是一个嵌套组件，只有在更左侧的分支到达树顶、且自身右侧分支候选全部耗尽之后，才被继续搜索。我们把这两者的组合称为 DFLR（depth first, then left to right，先深度优先、后从左到右）。当属性名只出现在类树的一个分支时，这种搜索就够了——这是典型情况。但当同一个名字出现在多个分支时，单靠 DFLR 就有点不给力：右侧一个更低（因而更专门）的子类，无法重定义经左侧分支到达的、更靠上（因而更通用）的超类中的名字。因此，多重继承需要一种略有不同的搜索顺序，称为 MRO（method resolution order，方法解析顺序；虽然它用于所有属性，不只是方法）

。简言之，MRO 在典型树中与 DFLR 行为相同，但当树中多个类共享一个共同的超类、形成所谓的菱形（diamond）模式时（因树的“角朝下的方形”形状而得名），MRO 会先横向按树层级推进，再向上走，更接近广度优先。这个 MRO 搜索顺序对所有类树都生效，但在存在多重继承菱形时与 DFLR 不同。它被设计成：每个共同的超类只被访问一次，而且是在它的所有子类之后。虽然用户自定义类在 Python 中不常形成菱形，但自动加在所有根（最顶层）类之上的内置 object 类，让每一棵多重继承树都成为菱形；没有 MRO 的话，object 的默认行为可能会藏匿用户自定义的版本。为了说明菱形到底是什么、MRO 在这种稍微非典型的情况下如何“先横后上”地搜索，让我们来看代码。

深度理解

·核心概念：DFLR 是朴素的“深度优先再从左到右”；MRO 是 Python 真正使用的顺序，专为菱形（多个类共享共同超类）优化：共同超类只访问一次，且在其全部子类之后。由于所有根类之上都隐式挂着 object，Python 中每一棵多重继承树其实都是菱形。

·底层实现：MRO 是类创建时由 C3 线性化算法计算出的一个元组，存在类.mro 里；属性查找就是按这个元组逐个类搜 dict。object 是每棵树的收尾节点，所以用户类的定义永远优先于 object 的默认实现（如 str、repr）。

·设计原因：为什么 DFLR 不够？看经典例子：A(B, C)、B(D)、C(D)，若 C 重定义了 D 中的 attr2，DFLR 会在搜到 D 后直接返回 D 的版本，C 的覆盖无效——这违反了“更专门的类优先”原则。MRO 让 C 在 D 之前被搜，修复

了这个问题。

·实际问题：多重继承最实用场景就是混入 (mix-in)：把通用能力 (打印、序列化、日志) 做成无状态小类，用 `class MyApp(MyBase, ListInstance)` 自由组合。代价是搜索顺序复杂化——`super()` 在多重继承下行为隐晦，本书建议显式类名调用。这也是“优先组合而非继承”原则最重要的反对理由之一。

·初学者误区：① 以为属性查找是“每个类只查一次”的树状遍历——实际是沿 mro 线性序列逐个查；② 以为 bases 就是查找顺序——它只记录直接父类，mro 才是完整顺序；③ 以为 MRO 只在有菱形时生效——它总是生效，只是无菱形时恰好与 DFLR 一致；④ 用 C++ 的虚继承心智去理解 Python 多重继承——机制完全不同。

31.17 多重继承如何工作 (How Multiple Inheritance Works)

英文原文

We'll get more formal about MROs in the next section, but it's easy to demo live. Let's start with the simple and typical case. In the following, class A inherits from both B and C using multiple inheritance, and B and C are root (topmost) classes that define unique attributes. When we make an instance I of A and ask for its attributes, inheritance searches I, A, B, and C—and in that order. Hence, attr1 is located in B, and attr2 in

C (if you're working along in a REPL, type a blank line after each class, omitted here for brevity):

```
>>> class C: attr2 = 'C2'>>> class B: attr1 = 'B1'>>>
class A(B, C): pass>>> I = A()>>> I.attr1, I.attr2 ('B1'
, 'C2')
```

As you might already expect, B wins if it has the same name as C due to the left-to-right component of the search, and lower classes like A still beat their supers as before:

```
>>> class C: attr2 = 'C2'>>> class B: attr2 = 'B2'; attr
1 = 'B1'>>> class A(B, C): attr1 = 'A1'>>> I = A()>>>
I.attr1, I.attr2 ('A1', 'B2')
```

Next, let's add a higher superclass above B: in the following, A inherits from both B and C as before, but B also inherits from D, and C is a root class with no supers. Per the DFLR order (depth first and then left to right), attr2 is found in D by virtue of searching I, A, B, and D—and before C is even checked:

```
>>> class D: attr2 = 'D2'>>> class C: attr2 = 'C2'>>>
class B(D): attr1 = 'B1'>>> class A(B, C): pass>>> I =
A()>>> I.attr1, I.attr2 ('B1', 'D2')
```

Watch what happens, though, if B and C both inherit from D, and C has the same attribute as D—per the MRO, the lower C class's attribute wins and effectively replaces that attribute in D:

```
>>> class D: attr2 = 'D2'>>> class C(D): attr2 = 'C2'>
>> class B(D): attr1 = 'B1'>>> class A(B, C): pass >>>
l = A() >>> l.attr1, l.attr2 ('B1','C2')
```

This last case is what is meant by a diamond pattern of inheritance: multiple classes, B and C, both inherit from the same superclass. When this happens, the lower class's version of an attribute is used instead of a same-named version in a superclass—even if that superclass could be reached first by depth alone. Importantly, this differs only in diamonds: nondiamonds still choose higher superclasses over lower classes with the same attribute name as in the third example in this section. The MRO allows a class to override attributes in a higher class from which it inherits; in the last example, C can replace names in D even if D is first per DFLR. To summarize: in nondiamonds, the search proceeds all the way to the top, then backs up and starts searching to the right (DFLR); in diamonds, the search checks classes to the right before climbing up to a common superclass (MRO). Strictly speaking, all four examples in this section are implicit diamonds because root classes inherit from the built-in object class automatically. We've seen that this class provides defaults for some methods, such as print displays. Because of the MRO, the object built-in's defaults never hide methods in user-defined classes below it in a class tree. To see how object is ruled out this way, let's get a bit more precise on the MRO's order.

中文翻译

下一节我们会更正式地讲 MRO，但现在很容易现场演示。先从简单而典型的情况开始。下面，类 A 用多重继承同时继承 B 和 C，而 B、C 是定义了各自独特属性的根（最顶层）类。当我们创建 A 的实例 I 并请求它的属性时，继承搜索按 I、A、B、C 的顺序进行。因此，attr1 在 B 中找到，attr2 在 C 中找到（如果你在 REPL 里跟着做，每个 class 后面要敲一个空行，这里为了简洁省略了）：

```
>>> class C: attr2 = 'C2'>>> class B: attr1 = 'B1'>>> class A(B, C): pass>>> I = A()>>> I.attr1, I.attr2 ('B1','C2')
```

正如你可能已经预料到的，由于搜索从左到右的分量，如果 B 和 C 有相同的名字，B 胜出；而 A 这样更低的类依然像以前一样胜过其超类：

```
>>> class C: attr2 = 'C2'>>> class B: attr2 = 'B2'; attr1 = 'B1'>>> class A(B, C): attr1 = 'A1'>>> I = A()>>> I.attr1, I.attr2 ('A1','B2')
```

接下来，在 B 之上再加一个更高的超类：下面 A 还像以前一样继承 B 和 C，但 B 也继承 D，而 C 是没有超类的根类。按 DFLR 顺序（先深度、后从左到右），attr2 在搜索 I、A、B、D 时在 D 中被找到——甚至在 C 还没被检查之前：

```
>>> class D: attr2 = 'D2'>>> class C: attr2 = 'C2'>>> class B(D): attr1 = 'B1'>>> class A(B, C): pass>>> I = A()>>> I.attr1, I.attr2 ('B1','D2')
```

不过请注意，如果 B 和 C 都继承自 D，而 C 有和 D 相同的属性，会发生什么——按 MRO，更低的 C 类的属性胜出，并有效地替换了 D 中的那个属性：

```
>>> class D: attr2 = 'D2'>>> class C(D): attr2 = 'C2'>>> class B(D): attr1 = 'B1'>>> class A(B, C): pass>>> I = A()>>> I.attr1, I.attr2 ('B1','C2')
```

最后一个

例子正是菱形继承模式的含义：多个类（B 和 C）都继承自同一个超类。发生这种情况时，更低类的属性版本会取代超类中同名的版本——即使那个超类仅凭深度就能先被到达。重要的是，这种差别只出现在菱形中：非菱形仍然像本节第三个例子那样，选择更高的超类而不是具有同名属性的更低类。MRO 允许类覆盖它继承自更高类中的属性；在最后一个例子里，即使按 DFLR D 在前，C 也能替换 D 中的名字。总结：在非菱形中，搜索一路走到树顶，然后回退，开始向右搜索（DFLR）；在菱形中，搜索先检查右侧的类，再向上爬向共同超类（MRO）。严格来说，本节全部四个例子都是隐式菱形，因为根类都自动继承自内置 object 类。我们已经见过这个类为某些方法提供默认实现，比如打印显示。因为 MRO 的存在，内置 object 的默认实现永远不会藏匿类树中它下面的用户自定义类的方法。想看清 object 是如何这样被排除的，让我们更精确地看看 MRO 的顺序。

深度理解

- 核心概念：四个递进示例演示了查找顺序的演化：① 同名冲突 → 左父类优先；② 同名冲突 → 左胜、低胜；③ 非菱形 → DFLR 一路深挖（D 在 C 之前）；④ 菱形 → MRO 让右侧更低的 C 优先于共同的 D。
- 底层实现：前三个示例中 DFLR 与 MRO 结果一致（唯一的 object 重复被去重，不改变结果）；第四例真正展现了 MRO 的威力： $A.mro = (A, B, C, D, object)$ 。注意 D 只出现一次，且在 C 之后。
- 设计原因：菱形里"右侧子类想覆盖共同超类"是合理需求

(C(D) 重定义 D 的属性，就是普通覆盖)，DFLR 会让这种覆盖失效，MRO 修正了它。同时"共同超类只访问一次"避免了重复初始化和重复方法调用。

·实际问题：理解这个例子，才能读懂后面的 mapattrs.py 和 super() 的行为；也是排查"为什么调用的不是我期望的方法"这类 bug 的关键工具——类.mro 一查便知。

·初学者误区：① 以为搜索顺序 = bases 顺序——只对直接父类成立；② 以为"左侧优先"永远成立——在菱形里右侧更低的类可能反超共同超类；③ 忽略隐式 object 造成的"隐性菱形"。

代码分析

```
class D: attr2 = 'D2'          #
class C(D): attr2 = 'C2'      # C  attr2
class B(D): attr1 = 'B1'      # B  D
class A(B, C): pass           # BC  D

I = A()
I.attr1, I.attr2              # ('B1', 'C2')
```

·解释器如何处理：创建 A 时，Python 用 C3 线性化计算 A.mro：先 DFLR 得 [A, B, D, C, D, object] (D 出现两次)，再去掉重复只留最右一次，得 (A, B, C, D, object)。I.attr2 沿此序列查找：I.dict 无 → A.dict 无 → B.dict 无 → C.dict 命中'C2'——D 的'D2' 永远轮不到。

·设计思想：MRO 用"线性化"把复杂的树形继承压成一条确定的查找线，让"哪个类赢"完全可预测、可计算 (mro 属性可见)。这是 Python 多重继承与传统语言最大的不同：它不靠运行时算法做树遍历，而是预先算好一条线。

31.18 MRO 如何工作 (How the MRO Works)

英文原文

Formally speaking, the MRO inheritance search order works as if all classes are listed per the DFLR, and then all but the rightmost duplicate of each class is removed. In more detail, it's computed as follows: 1. List all the classes from which an instance inherits using the DFLR lookup order, and include a class multiple times if it's visited more than once. 2. Scan the resulting list for duplicate classes, removing all but the last (rightmost) occurrence of duplicates in the list. The resulting MRO sequence for a given class includes the class, its superclasses, and all higher superclasses up to and including the implicit or explicit object root class above the tops of the tree. It's ordered such that each class appears before its parents, and multiple parents retain the order in which they appear in the class header. Because common parents in diamonds appear only at the position of their last visitation in the MRO, lower classes are searched first when the MRO list is used later by attribute inheritance (making it more breadth-first than depth-first in diamonds only), and each class is included and thus visited just once, no matter how many classes lead to it. Consider the nondia

mond class tree of Example 31-8, whose shape is sketched as "ASCII art" in its comments. Example 31-8. mronondiamond.py

```
python class E: attr = 'E' # D E class D: attr = 'D' # | | class C(E): attr = 'C' # B C class B(D): pass # \ / class A(B, C): pass # A # | X = A() # X print(X.attr) # D To compute the MRO inheritance search order through this tree, Python first enumerates all classes accessible per DFLR and then removes duplicates (remember that the built-in object is implicitly above all root classes):
```

DFLR => [X, A, B, D, object, C, E, object] MRO => [X, A, B, D, C, E, object] In this tree, the net result for both DFLR and MRO ordering is the same: the output is "D" by taking attr from class D at the top left. Technically, the MRO differs because the built-in object appears just once at the end after its duplicates are removed; this is irrelevant to our classes because they don't redefine an object default (like print strings). When a user-defined diamond is coded in Example 31-9, however, the MRO's difference is more striking. Example 31-9. mrodiamond.py

```
python class D: attr = 'D' # D class C(D): attr = 'C' # / \ class B(D): pass # B C class A(B, C): pass # \ / # A X = A() # | print(X.attr) # C X For this tree, the common user-defined class D is reached twice by DFLR. Because the MRO keeps only the last (rightmost) D, the lower C's definition of attr now wins over D's. Hence, the o
```

output is now "C" instead of "D":

DFLR => [X, A, B, D, object, C, D, object] MRO => [X, A, B, C, D, object] In fact, you can view the MRO of any class with its built-in mro attribute. This tuple gives the inheritance search order followed for instances of the class (after the instance itself). It's set to the result of the mro class method at class creation time (which can technically be customized for roles too obscure to cover here). It's also a lot to look at unless we select class names as in the following, which inspects classes in the nondiamond tree of Example 31-8:

```
>>> from mronondiamond import D
>>> [c.name for c in A.mro] ['A','B','D','C','E','object']
>>> [c.name for c in C.mro] ['C','E','object']
```

And here's the case for the diamond class tree in Example 31-9:

```
>>> from mrodiamond import C
>>> [c.name for c in A.mro] ['A','B','C','D','object']
>>> [c.name for c in C.mro] ['C','D','object']
```

The full mro is the classes used by inheritance at a given class, and bases is just supers there:

```
>>> A.mro (<class'mrodiamond.A'>, <class'mrodiamond.B'>, <class'mrodiamond.C'>,
<class'mrodiamond.D'>, <class'object'>) >>> X.class
.mro (<class'mrodiamond.A'>, <class'mrodiamond.B'
```



```
>, <class'mrodiamond.C'>,
<class'mrodiamond.D'>, <class'object'>)>>> A.bases
(<class'mrodiamond.B'>, <class'mrodiamond.C'>)
>>> D.bases (<class'object'>,)

```

Experiment with mro on your own for more fidelity. Especially if you're unsure how the MRO handles a given class tree, this attribute can be consulted to see how inheritance will truly search.

中文翻译

严格地说，MRO 继承搜索顺序的工作方式，就仿佛所有类先按 DFLR 列出，然后删除每个类的重复出现（只保留最右侧的那次）。更详细地说，它是这样计算的：1. 用 DFLR 查找顺序列出实例继承的所有类；如果一个类被访问多次，就列多次。2. 扫描结果列表找出重复的类，删除除最后（最右侧）一次以外的所有出现。给定类的最终 MRO 序列，包含该类本身、它的超类、以及直到并包括树顶之上隐式或显式 object 根类的所有更高超类。它的排序保证：每个类都出现在它的父类之前，多个父类保留它们在 class 头行中出现的顺序。因为菱形中的共同父类在 MRO 中只出现在它最后一次被访问的位置，所以当 MRO 列表稍后被属性继承使用时，更低的类会被先搜索（这使搜索只在菱形中比深度优先更接近广度优先），而且每个类只被包含（从而只被访问）一次，无论有多少个类通向它。考虑示例 31-8 的非菱形类树，其形状在注释里画成了"ASCII 艺术图"。示例 31-8. mronondiamond.py

```
python class E: attr='E' # D E
class D: attr='D' # || class C(E): attr='C' # B C class B

```

(D): pass # \ / class A(B, C): pass # A # | X = A() # X print(X.attr) # D 要计算穿过这棵树的 MRO 继承搜索顺序, Python 先用 DFLR 枚举所有可达的类, 然后删除重复 (记住: 内置 object 隐式地处于所有根类之上): DFLR => [X, A, B, D, object, C, E, object] MRO => [X, A, B, D, C, E, object] 在这棵树中, DFLR 和 MRO 排序的净结果相同: 从左上角的 D 类取 attr, 输出是 "D"。严格说 MRO 确实不同——内置 object 在删除重复后只在末尾出现一次; 这对我们的类无关紧要, 因为它们没有重定义 object 的默认行为 (比如打印字符串)。然而, 当示例 31-9 中编码了用户自定义菱形时, MRO 的差异就非常明显了。示例 31-9. mrodiamond.py

```
python class D: attr = 'D' # D class C(D): attr = 'C' # / \ class B(D): pass # B C class A(B, C): pass # \ / # A X = A() # | print(X.attr) # C X
```

对这棵树来说, 用户定义的共同类 D 会被 DFLR 到达两次。因为 MRO 只保留最后一个 (最右侧的) D, 更低的 C 对 attr 的定义现在压过了 D 的。因此, 输出现在是 "C" 而不是 "D": DFLR => [X, A, B, D, object, C, D, object] MRO => [X, A, B, C, D, object] 事实上, 你可以用内置的 mro 属性查看任何类的 MRO。这个元组给出了该类实例 (实例本身之后) 遵循的继承搜索顺序。它在类创建时被设为 mro 类方法的结果 (技术上可以为某些过于冷僻的角色定制, 这里不展开)。直接看它有点费眼, 除非像下面这样只挑出类名——这里检查的是示例 31-8 非菱形树中的类:

```
>>> from mronondiamond import D >>> [c.name for c in A.mro] ['A','B','D','C','E','object'] >>> [c.name for c in C.mro] ['C','E','object']
```

下面是示例 31-9 菱形类树的情况:

```
>>> from mrodiamond import C >>> [c.name for c i
```

n A.mro] ['A','B','C','D','object'] >>> [c.name for c in C.mro] ['C','D','object'] 完整的 mro 是给定类继承所使用的类序列，而 bases 只是那里的直接超类：>>> A.mro (<class'mrodiamond.A'>, <class'mrodiamond.B'>, <class'mrodiamond.C'>, <class'mrodiamond.D'>, <class'object'>) >>> X.class.mro (<class'mrodiamond.A'>, <class'mrodiamond.B'>, <class'mrodiamond.C'>, <class'mrodiamond.D'>, <class'object'>) >>> A.bases (<class'mrodiamond.B'>, <class'mrodiamond.C'>) >>> D.bases (<class'object'>,) 请自己动手实验 mro 以获得更精确的感觉。特别是当你不确定 MRO 如何处理某个类树时，可以查看这个属性，看看继承到底会怎么搜。

深度理解

·核心概念：MRO 的官方算法（简化版）就两步：① 按 DFLR 列出所有可达类（允许重复）；② 删除重复，只留最右侧那次。结果是一条"子类在前、父类在后、父类按头行顺序排列、共同超类只出现一次"的线性序列。

·底层实现：真正实现是 C3 线性化算法（本书略去了数学细节），它比"DFLR+去重"更严格，但直觉等价。mro 元组在类创建时（type.new 阶段）计算并冻结；属性查找（X.attr、getattr、super）全部基于它。bases 只是直接父类元组。

·设计原因："只留最右侧重复"巧妙地把"更专门优先"编码进线性序：菱形里 C 比 D 靠前（C 是 D 的子类），因为 D 的重复位置被保留到了最右（最后被访问）。于是子类天然覆盖共同超类。

·实际问题：调试属性“来源不明”时，`[c.name for c in Cls.mro]` 一行命令就是真相。`super()` 在多重继承下也按 `mro` 推进——第 32 章会展开。

·初学者误区：① 以为 `mro` 是运行时动态计算的——类定义完就固定了；② 以为 `MRO` 顺序可以随意改（改不了，除非自定义元类的 `mro()`，属于冷僻技巧）；③ 把 `mro` 和 `bases` 混用——后者不含间接父类。

代码分析

```
class E: attr = 'E'          # ASCII art
class D: attr = 'D'
class C(E): attr = 'C'
class B(D): pass
class A(B, C): pass          # BC

X = A()
print(X.attr)                # D
```

```
class D: attr = 'D'          # BC D
class C(D): attr = 'C'
class B(D): pass
class A(B, C): pass

X = A()
print(X.attr)                # CMRO C D
```

·解释器如何处理：非菱形树的 `MRO` 计算：`DFLR` 枚举 `[X, A, B, D, object, C, E, object]` → 去重（保留最右）→ `[X, A, B, D, C, E, object]`。`X.attr` 沿此线搜索：B 分支一路到 D 命中'D'。菱形树的 `MRO`：`DFLR` `[X, A, B, D, object, C, D, object]` → 去重 → `[X, A, B, C, D, object]`。`X.attr` 在 `B.dict` 没有、`C.dict` 命中'C'——D 被压后。注意

两种树里 object 都只出现一次且永远在末尾：这就是"用户类永远优先于内置默认"的保证。

·设计思想："用线性序编码继承语义"是 Python 的独创设计：相比 C++ 的虚基类等复杂机制，MRO 让继承顺序可枚举、可验证——mro 就是权威文档。

31.19 属性冲突的解决 (Attribute Conflict Resolution)

英文原文

Though a useful pattern, multiple inheritance's chief downside is that it can pose a conflict when the same method (or other attribute) name is defined in more than one branch of the class tree. When this occurs, the conflict is resolved either automatically by the inheritance search order or manually in your code: Default (默认) — By default, inheritance chooses the first occurrence of an attribute it finds when an attribute is referenced normally (e.g., by `self.attr`). In this mode, Python chooses the first appearance located while scanning the MRO of an instance's class, from left to right. Explicit (显式) — In some class models, you may need to select an attribute explicitly by referencing it through its class name (e.g., by `superclass.attr`). Your code resolves the conflict this way and overrides the search's default to select an option other than the inheritance search's default. By default, multiple inherit

ance assumes that names on the left of a class tree should override the same names on the right, and the MRO further assumes that lower classes should override same-named attributes in common superclasses of diamonds. Of course, the problem with assumptions is that they assume things. In Example 31-9, what if you need some same-named attributes from B but also some others from C? Luckily, there is an inheritance escape hatch. If the default search order doesn't work, or if you simply want more control over the search process, you can always force the selection of an attribute from anywhere in the tree by assigning or otherwise naming the one you want at the place where classes are mixed together. In Example 31-9, for instance, any of the following selections are allowed, but the latter two make the choice explicit rather than relying on the implicit and subtle "magic" of MRO inheritance:

```
python class A(B, C): pass # Use the default MRO choice
(C) class A(B, C): attr = B.attr # Choose attr from the left branch
(D) class A(B, C): attr = C.attr # Choose attr from the right branch
(C) Naturally, attributes picked this way can also be methods—which are normal, assignable attributes that happen to reference callable function objects. Moreover, the choice can be made in a call:
```

```
python class A(B, C): def ...: self.method(...) # Use the default MRO choice
class A(B, C): def ...: B.method(self, ...) # Choose method from the left branch
class A(B, C): def ...: C.method(self, ...) #
```

Choose method from the right branch Such calls can be used to override the MRO, but also to kick calls up the tree when the class's own version must be skipped. As you'll see in the next chapter, such calls might also use `super().method()`, which selects the class following the call's host on self's MRO; though simple in single inheritance, this can be stunningly implicit in multiple inheritance and doesn't provide as much control as explicit class names. However they are coded, such manual overrides are required only when the same name appears in multiple superclasses, and you do not wish to use the first one inherited. For example, manual overrides allow you to unambiguously choose among a set of same names in both left and right branches, while inheritance would choose just names on the left. They can also be used to make choices explicit in general, though, even in nondiamonds. Because this isn't as common an issue in typical Python code as it may sound, we'll defer details on this topic until we study the `super` built-in in the next chapter and revisit this as a "gotcha" at the end of that chapter. First, though, the next section demonstrates a practical use case for the multiple-inheritance design pattern. NOTE — The inheritance finale: Despite the MRO's complexity, there are still a few inheritance hurdles left to clear. Its complete algorithm is more complex than the model sketched here, incorporating special cases for metaclasses, descriptors, and built-ins. In C

chapter 32, we'll expand inheritance for the metaclass tree and apply its MRO in the super built-in, and in Chapter 38, we'll study its special case for built-in operations, but we won't be able to formalize inheritance in full until Chapter 40 after we've studied all these tools in more depth. The good news is that the remaining hurdles almost never trip up application programs; for most coders, the "finale" is a fully optional read.

中文翻译

虽然多重继承是个有用的模式，但它最主要的缺点在于：当同一个方法（或其他属性）名在类树的多个分支中都有定义时，就会产生冲突。发生这种情况时，冲突要么由继承搜索顺序自动解决，要么在你的代码中手动解决：默认（Default）——默认情况下，属性被正常引用（如 `self.attr`）时，继承会选择找到的第一个出现。在这种模式下，Python 选择在从左到右扫描实例类的 MRO 时找到的第一次出现。显式（Explicit）——在某些类模型中，你可能需要通过类名引用属性来显式选择（如 `superclass.attr`）。你的代码以此方式解决冲突，覆盖搜索默认值，选出继承搜索默认值之外的选项。默认情况下，多重继承假定类树左侧的名字应该覆盖右侧的同名名字，MRO 还假定更低的类应该覆盖菱形共同超类中的同名属性。当然，假设的问题就在于“它们在假设”。在示例 31-9 里，如果你既需要 B 的某些同名属性、又需要 C 的另一些呢？幸运的是，继承有一个逃生舱口（escape hatch）。如果默认搜索顺序不适用，或者你只是想对搜索过程有更多控制，你总可以在类混合的地方赋值

或指名你想要的属性，从而强制从树中任何位置选取属性。以示例 31-9 为例，下面的任何选择都合法，但后两种把选择变成显式的，而不是依赖 MRO 继承那种隐晦的"魔法"：

```
python class A(B, C): pass # 用默认的 MRO 选择 (C)
class A(B, C): attr = B.attr # 从左分支选择 attr (D)
class A(B, C): attr = C.attr # 从右分支选择 attr (C)
```

当然，这样选出的属性也可以是方法——方法本来就是普通的、可赋值的属性，只不过引用的是可调用的函数对象。而且，选择也可以在调用中做出：

```
python class A(B, C): def ...: self
.method(...) # 用默认的 MRO 选择
class A(B, C): def ...: B
.method(self, ...) # 从左分支选择方法
class A(B, C): def ...: C
.method(self, ...) # 从右分支选择方法
```

这类调用既可用于覆盖 MRO，也可在必须跳过类自身版本时把调用踢向树的上方。下一章你会看到，这类调用也可以改用 `super().method()`，它会在 `self` 的 MRO 上选择调用宿主之后的类；这在单继承里很简单，但在多重继承里可能隐晦得惊人，而且控制力不如显式类名。无论怎么编码，这种手动覆盖只有在同一个名字出现在多个超类、而你又不希望使用继承到的第一个时才是必需的。例如，手动覆盖允许你在左右两个分支的一组同名中无歧义地选择，而继承只会选择左侧的名字。即使非菱形，它们也可以用来让选择变得显式。因为这在典型 Python 代码中并不像听起来那么常见，我们要把细节推迟到下一章研究 `super` 内建函数时再讲，并在那章末尾把它作为"陷阱 (gotcha)"重新审视。不过在那之前，下一节将演示多重继承设计模式的一个实际用例。注意——

继承的终章 (The inheritance finale)：尽管 MRO 已经够复杂，仍有一些继承的坎要过。它的完整算法比这里勾勒的模型复杂得多，还包含针对元类、描述符和内置类型的特

殊情况。在第 32 章，我们会把继承扩展到元类树，并在 `super` 内建函数中应用其 MRO；在第 38 章，我们会研究它对内置操作的特殊处理；但要到第 40 章，等我们深入学完所有这些工具之后，才能把继承完整地形式化。好消息是：剩下的这些坎几乎从来不会绊倒应用程序；对大多数编码者来说，“终章”完全是可以选读的。

深度理解

·核心概念：冲突解决有两个层次：默认（MRO 顺序自动决定，“左胜右、低胜高”）与显式（在混入处赋值选择，或调用时用 `B.method(self, ...)` 指名道姓）。显式是 MRO 的“逃生舱口”。

·底层实现：`attr = B.attr` 在 A 类体里执行时，右侧的 `B.attr` 在 B 的命名空间查找（不触发 MRO），再把结果绑定为 A 类自己的类属性——此后 A 的查找在 `A.dict` 即命中，完全绕开冲突。`B.method(self, ...)` 是“经类取方法”的普通函数调用，显式传入 `self`。

·设计原因：设计者（即你）比算法更懂意图。默认规则覆盖 95% 场景，剩下 5% 需要人拍板——Python 把选择权留给你，而不是发明更复杂的自动规则。

·实际问题：什么时候真的需要？混入多个框架类时，两侧各有一个同名工具方法，且都需要——用 `self` 调用默认的，用 `Other.method(self, ...)` 调用另一个。`super()` 在多重继承下的“隐晦”正是本章建议“优先显式类名”的原因。

·初学者误区：① 以为 `self.method()` 一定调用“最近的定义”——其实遵循 MRO 的确定性顺序，弄清楚 mro 就能

预测；② 以为 `super().method()` 一定调用"父类的"——在多重继承里它调用的是 MRO 下一个类，可能出乎意料；③ 把类属性赋值 `attr = B.attr` 与实例属性混淆——前者创建的是类级属性。

代码分析

```
class D: attr = 'D'          #
class C(D): attr = 'C'
class B(D): pass

#
class A1(B, C): pass         # MRO  C
class A2(B, C): attr = B.attr # B.attr D  'D'
class A3(B, C): attr = C.attr # C.attr C  'C'

#
class A4(B, C):
    def run(self):
        B.method(self)      #
        C.method(self)      #
```

·解释器如何处理：class A2 类体执行时，右侧 `B.attr` 先被求值（在 B 命名空间查得'D'，因为 B 没有同名属性，继承自 D），随后 `attr = 'D'` 被写入 `A2.dict`。此后任何 `a2.attr` 在 A2 本身即命中，MRO 无从插手。调用式选择同理：`B.method(self)` 通过 B 类直接取函数并手动传 `self`，跳过 MRO。

·设计思想："显式优于隐式"（Zen of Python）在多重继承冲突上的直接体现。默认 MRO 是方便，显式选择是掌控——两者都是同一套"名字绑定"机制的产物。

31.20 示例: "混入"式属性列出器 (Example: "Mix-in" Attribute Listers)

英文原文

Perhaps the most common way multiple inheritance is used is to "mix in" general-purpose methods from superclasses. Such superclasses are usually called mix-in classes—they provide methods you add to application classes by inheritance. In a sense, mix-in classes are similar to modules: they provide packages of methods for use in their client subclasses. Unlike simple functions in modules, though, methods in mix-in classes also can participate in inheritance hierarchies and have access to the self instance for using state information and other methods in their trees. For example, we've seen that Python's default way to print a class instance object isn't incredibly useful:

```
>>> class Hack:
def init(self, what): # No repr or str
    self.data1 = what
>>> X = Hack('code')
>>> print(X) # Default: class name + address (id)
<main.Hack object at 0x10ba464e0>
```

As you learned in both Chapter 28's case study and Chapter 30's operator-overloading coverage, classes can provide a str or repr method to implement custom displays. But rather than coding one of these in each class you wish to print, why not code it once in a general-purpose tool class and inherit it in all your other classes? That's what mix-ins are for. Defining a di

splay method in a mix-in superclass once enables us to reuse it anywhere we want to see a custom display format—even in classes that may already have another superclass. We've already explored tools that do related work: Chapter 28's `AttrDisplay` class, of Example 28-13, formatted instance attributes in a generic `repr` method, but it did not climb class trees and was utilized in single-inheritance mode only. Chapter 29's `classree.py` module, of Example 29-9, defined functions for climbing and sketching class trees, but it did not display object attributes along the way and was not architected as an inheritable class. Here, we're going to revisit these examples' techniques and expand upon them to code a set of three mix-in classes that serve as generic display tools for listing instance attributes, inherited attributes, and attributes on all objects in a class tree, respectively. We'll also use our tools in multiple-inheritance mode and deploy coding techniques that make classes better suited to use as generic tools. Unlike Chapter 28, we'll code this with a `str` instead of a `repr`. This is partially a style issue and limits their role to `print` and `str`, but the displays we'll be developing are meant to be user-friendly, not imitative of code. This policy also leaves client classes the option of coding an alternative lower-level display for interactive echoes and nested appearances with a `repr` and has built-in immunity from `repr` looping perils covered ahead.

中文翻译

多重继承最常见的用法也许是"混入" (mix in) 超类里的通用方法。这样的超类通常被称为混入类 (mix-in class) ——它们提供你通过继承添加给应用类的方法。从某种意义上说, 混入类类似模块: 它们为客户端子类提供方法包。但不同于模块里的简单函数, 混入类中的方法可以参与继承层次, 还能访问 self 实例, 以使用树中的状态信息和其它方法。例如, 我们见过 Python 默认打印类实例对象的方式并不怎么好用: >>> class Hack: def init(self, what): # 没有 repr 或 str self.data1 = what >>> X = Hack('code') >>> print(X) # 默认: 类名 + 地址 (id) <main.Hack object at 0x10ba464e0> 就像你在第 28 章的案例研究和第 30 章的运算符重载中学到的, 类可以提供 str 或 repr 方法来实现自定义显示。但是, 与其在每个想打印的类里都写一个, 为什么不把它写在一个通用工具类里, 然后让所有其它类继承它呢? 这就是混入类的用途。在混入超类里定义一次显示方法, 就能让我们在想看到自定义显示格式的任何地方复用它——甚至在已经有另一个超类的类里也行。我们已经探索过做相关工作的工具: 第 28 章示例 28-13 的 Attr Display 类, 在通用 repr 方法里格式化实例属性, 但它不爬类树, 而且只能在单继承模式下使用。第 29 章示例 29-9 的 classtree.py 模块定义了爬树和画树的函数, 但它不显示沿途对象的属性, 也没有被设计成可继承的类。在这里, 我们要重拾这些示例的技术并加以扩展, 编写三个混入类, 分别充当列出实例属性、列出继承属性、列出类树中所有对象属性的通用显示工具。我们还会在多重继承模式下使用这些工具, 并部署让类更适合充当通用工具的编码技巧。与

第 28 章不同，我们这次用 `str` 而不是 `repr` 编码。这部分是风格问题，把作用限定在 `print` 和 `str` 上；但我们开发这些显示格式是为了对人类友好，而不是模拟代码的样子。这个策略也把“为交互式回显和嵌套外观写一个更底层的 `repr`”的选项留给客户端类，并且天然免疫后面要讲的 `repr` 循环陷阱。

深度理解

·核心概念：混入类 = “方法包”的继承化。模块用 `import` 提供函数，混入类用继承提供方法——但混入类里的方法能访问 `self`，于是能读写客户端子类的实例状态。这是模块函数做不到的。

·底层实现：`class Sub(Super, ListInstance)` 把 `ListInstance.str` 混入查找链。任何 `Sub` 实例 `print` 时，解释器沿 MRO 找到 `ListInstance.str`；方法里的 `self` 是 `Sub` 实例，可以用 `self.dict`、`self.class` 等通用工具访问任意客户类的数据。

·设计原因：把“打印实例”这种横切关注点抽成可复用组件，避免在每个应用类里重复写 `str`。继承（自动获得）+ 组合（自由搭配）让这种“工具箱”可以一块块往里加。

·实际问题：混入类最常见的坑：方法名冲突（比如客户类也定义了 `str`）和属性名冲突。本章用伪私有 `attrnames` 预防后者，用“把混入类写在头行最左侧”控制前者。

·初学者误区：① 以为混入类是抽象类——不需要 `init`、不需要实例化，纯粹是方法提供者；② 以为混入类必须单独实例化使用——错，它是给别的类“借”的；③ 混淆 `str`

与 repr 的触发时机——print/str 用 str，REPL 回显和 repr() 用 repr。

31.21 用 dict 列出实例属性 (Listing instance attributes with dict)

英文原文

Let's get started with the simple case—listing attributes attached to an instance. Example 31-10, coded in the file listinstance.py, defines a mix-in called ListInstance that overloads the str method for all classes that include it in their header lines. Because this is coded as a class, ListInstance is a generic tool whose formatting logic can be used for instances of any subclass client.

Example 31-10. listinstance.py

```
python class ListInstance: """Mix-in class that provides a formatted print() or str() of instances via inheritance of str coded here. Displays instance attrs only; self is instance of lowest class; X naming avoids clashing with client's attrs. Works for classes with slots: a dict is ensured by lack of slots here.
```

```
"""
```

```
def attrnames(self): result = '\n'for attr in sorted(self.dict): # Slots okay result += f'\t{attr}={self.dict[attr]!r}\n' # Repr for quotes
```



```
return result def str(self): return (f'<Instance of {self.c  
lass.name},' # My class's name f'address {id(self):#x}:'  
# My address (hex) f'{self.attrnames()}>') # name=val  
ue list
```

```
if name == 'main': import testmixin testmixin.testter(Li  
stInstance) # Test class in this module
```

The `attrnames` method here exhibits a classic comprehension pattern, and you might save some program real estate by implementing it more concisely with a generator expression triggered by a `"".join()` call; we'll leave this as a suggested exercise. As coded, `ListInstance` uses some previously explored techniques to extract the instance's class name and attributes:

- It uses a `self.class.name` expression to fetch the name of an instance's class. Recall that each instance has a built-in class attribute that references the class from which it was created, and each class has a name attribute that references the class's name given in its header line.
- It does most of its work by simply scanning the instance's attribute dictionary (remember, it's available in `dict`) to build up a string showing the names and values of all instance attributes. The dictionary's keys are sorted by name to be human-friendly, instead of relying on the dictionary's insertion order.

In these respects, `ListInstance` is similar to Chapter 28's attribute display; in fact, it's largely just a variation on a theme. Our class here, though, uses two additio

nal techniques: - It displays the instance's memory address by calling the `id` built-in function, which returns any object's address. By definition, this is a unique object identifier, which will be useful in later mutations of this code. - It uses the pseudoprivate naming pattern for its worker method: `attrnames`. As we saw earlier in this chapter, Python automatically localizes any such name to its enclosing class by expanding the attribute name to include the class name; in this case, it becomes `ListInstanceattrnames`. This holds true for both class attributes like methods and instance attributes attached to `self`. As first noted in Chapter 28, this helps in a general tool like this, as it ensures that its names won't clash with any names used in its client subclasses.

Because `ListInstance` defines a `str` operator-overloading method, instances derived from this class display their attributes automatically when printed, giving a bit more information than a simple address. Here is this tool class in action in single-inheritance mode, mixed in to the previous section's class:

```
>> from listinstance import ListInstance >> class Hack(ListInstance):
```

```
# Inherit a str method
def init(self, what):
    self.data1 = what
>> X = Hack('code')
>> print(X) # print() and str()
run str <Instance of Hack, address 0x10c890b90:
```

```
data1='code'>
```

You can also get the listing display as a string with `str` without printing it (use `print` later to apply escapes), and interactive echoes and nesting still use the default format (we've left `repr` as an option for clients):

```
>> str(X) <Instance of Hack, address 0x10c890b90:\n\\tdata1='code'\\n>>> X
```

`<main.Hack object at 0x10c890b90>` The `ListInstance` class is useful for any classes you write—even classes that already have one or more superclasses. This is where multiple inheritance comes in handy: by adding `ListInstance` to the list of superclasses in a class header (i.e., mixing it in), you get its `str` "for free" while still inheriting from the existing superclass(es). The file `testmixin.py` in Example 31-11 codes test classes that prove the point.

Example 31-11. `testmixin.py`

```
python """Generic lister-mixin tester: similar to transi  
tive reloader in Chapter 25, but passes a class object  
to tester (not function), and testByNames adds loadi  
ng of both module and class by name strings here, in  
keeping with Chapter 31's factories pattern.
```

```
"""
```

```
import importlib def tester(listerclass, sept=False): "P  
ass any lister class to listerclass"
```

```

class Super: def init(self): # Superclass init self.data1
='code' # Create instance attrs def method1(self): pas
s class Sub(Super, listerclass): # Mix in method1 and
a str def init(self): # Listers have access to self Super.i
nit(self) # Or super().init()

```

```

self.data2 ='Python' # More instance attrs self.data3
= 3.12 def method2(self): # Define another method h
ere pass instance = Sub() # Build instance with lister's
str print(instance) # Run mixed-in str (or via str(x)) if
sept: print(f'\n{'-' 80}\n') # Python 3.12+ f-string def t
estByNames(modname, classname, sept=False): mod
object = importlib.importmodule(modname) # Import
mod by namestring listerclass = getattr(modobject,
classname) # Fetch attr by namestring tester(listercla
ss, sept)

```

```

if name == 'main': testByNames('listinstance', 'ListInst
ance', True) # Test all three here testByNames('listinh
erited', 'ListInherited', True) # See others ahead... test
ByNames('listtree', 'ListTree', False)

```

This file is instrumented to allow us to test a variety of listers. To this end, it is passed a lister class, listerclass, to be mixed in with test classes nested in a function. Again, classes are passable first-class objects, and we need to make the lister class a parameter to allow it to vary per test. This file also has tools to fetch classes by name strings, which we'll set aside for the moment.

More important here is the self-test code in Example 31-10: it imports the tester function in Example 31-11 and passes in `ListInstance` to be tested here. Hence, `Sub` here inherits names from both `Super` and `ListInstance`; it's a composite of its own names and names in both its superclasses. When you make a `Sub` instance and print it, you automatically get the custom representation mixed in from `ListInstance`:

```
$ python3 listinstance.py <Instance of Sub, address 0x10caae300:
```

```
data1='code'data2='Python'data3=3.12 >
```

The `ListInstance` class responsible for this display works in any class it's mixed into because `self` refers to an instance of the subclass that pulls this class in, whatever that may be. Again, in a sense, mix-in classes are the class equivalent of modules—packages of methods useful in a variety of clients.

Besides the utility they provide, mix-ins optimize code maintenance, as all classes do. For example, if you later decide to extend `ListInstance`'s `str` to also print all the class attributes that an instance inherits, you're covered; because it's an inherited method, changing `str` automatically updates the display of each subclass that imports the class and mixes it in. And since it's now officially "later," let's move on to the next section to see what such an extension might look like.

中文翻译

我们从头开始——列出附加在实例上的属性。示例 31-10（文件 listinstance.py）定义了一个叫 ListInstance 的混入类，它重载了所有把头行里包含它的类的 str 方法。因为它是按类编码的，ListInstance 是一个通用工具，其格式化逻辑可用于任何子类客户端的实例。示例 31-10. listinstance.py

```
python class ListInstance: """ Mix-in class that provides a formatted print() or str() of instances via inheritance of str coded here. Displays instance attrs only; self is instance of lowest class; X naming avoids clashing with client's attrs. Works for classes with slots : a dict is ensured by lack of slots here. """ def attrnames(self): result = '\n'for attr in sorted(self.dict): # 有 slots 也行result += f'\t{attr}={self.dict[attr]!r}\n' # 用 repr 加引号return result def str(self): return (f'<Instance of {self.class.name}, ' # 我的类名f'address {id(self):#x}: ' # 我的地址（十六进制）f'{self.attrnames()}>') # name=value 清单if name == 'main': import testmixin testmixin.testner(ListInstance) # 测试本模块中的这个类这里的 attrnames 方法展示了经典的循环模式，你可以用''.join() 触发的生成器表达式更精简地实现它，省点程序空间；我们把这留作建议练习。就目前编码而言，ListInstance 用一些此前探索过的技术获取实例的类名和属性：- 用 self.class.name 表达式取实例的类名。回忆一下：每个实例都有内置 class 属性，引用创建它的类；每个类都有 name 属性，引用它在头行里给出的类名。- 它的大部分工作就是扫描实例的属性字典（记住，就在 dict 里），拼出显示所有实例属性
```

名和值的字符串。字典的键按名字排序，对人更友好，而不是依赖字典的插入顺序。在这些方面，ListInstance 与第 28 章的属性显示类似；事实上，它大体只是旧主题的一个变体。不过我们这个类还用了两项额外技术：- 它调用内置 id 函数显示实例的内存地址，该函数返回任何对象的地址。按定义，这是唯一的对象标识符，对稍后这个代码的演变很有用。- 它的工作方法用了伪私有命名模式：attrnames。如本章前面所见，Python 自动把这种名字本地化到所在类：扩展属性名，包含类名；这里它变成 ListInstanceattrnames。这对方法之类的类属性和挂在 self 上的实例属性都成立。正如第 28 章最初提到的，这在这种通用工具里很有帮助，能确保它的名字不会与客户端子类用的任何名字冲突。因为 ListInstance 定义了 str 运算符重载方法，派生自这个类的实例被打印时会自动显示它们的属性，比一个单纯的地址多给一点信息。下面是这个工具类在单继承模式下的表现，混入到上一节的类里：

```
>>> from listinstance import ListInstance
>>> class Hack(ListInstance): # 继承一个 str 方法
def init(self, what): self.data1 = what
>>> X = Hack('code')
>>> print(X) # print() 和 str() 运行 str
<Instance of Hack, address 0x10c890b90: data1='code'>
你也可以用 str 把清单显示作为字符串取回而不打印（之后再 print 应用转义），而交互式回显和嵌套仍然用默认格式（我们把 repr 留作客户端选项）：
>>> str(X)
"<Instance of Hack, address 0x10c890b90:\n\tdata1='code'\n>"
>>> X
<main.Hack object at 0x10c890b90>
ListInstance 类对你编写的任何类都有用——即使已经有了一两个超类的类。这就是多重继承派上用场的地方：把 ListInstance 加进类头行的超类列表（即混入它），你就能“免费
```

"获得它的 str，同时仍然继承现有的超类。示例 31-11 的文件 testmixin.py 编写了证明这一点的测试类。示例 31-11.

```
testmixin.py python """通用 lister-mixin 测试器：类似第 25 章的传递式重载器，但这里把类对象（而非函数）传给 tester；testByNames 按名字字符串加载模块和类，与第 31 章的工厂模式保持一致。""" import importlib def tester(listerclass, sept=False): "把任意 lister 类传给 listerclass" class Super: def init(self): # 超类 init self.data1 = 'code' # 创建实例属性 def method1(self): pass class Sub(Super, listerclass): # 混入 method1 和一个 str def init(self): # lister 都能访问 self Super.init(self) # 或 super().init() self.data2 = 'Python' # 更多实例属性 self.data3 = 3.12 def method2(self): # 在这里再定义一个方法 pass instance = Sub() # 用 lister 的 str 构建实例 print(instance) # 运行混入的 str（或 str(x)） if sept: print(f'\n{'-' 80} \n') # Python 3.12+ f-string def testByNames(modname, classname, sept=False): modobject = importlib.importmodule(modname) # 按名字字符串导入模块 listerclass = getattr(modobject, classname) # 按名字字符串取属性 tester(listerclass, sept) if name == 'main': testByNames('listinstance', 'ListInstance', True) # 依次测试全部三个 testByNames('listinherited', 'ListInherited', True) # 看后面的其他... testByNames('listtree', 'ListTree', False) 这个文件被仪器化（instrumented），让我们能测试各色 lister。为此，它接收一个 lister 类参数 listerclass，与嵌套在函数里的测试类混合。再次强调，类是可传递的一等对象，我们需要把 lister 类做成参数，以便随测试而变化。这个文件还有按名字字符串取类的工具，我们先放在一边。
```


这里更重要的，是示例 31-10 里的自测代码：它导入示例 31-11 里的 `tester` 函数，传入待测试的 `ListInstance`。于是这里的 `Sub` 从 `Super` 和 `ListInstance` 两者继承名字；它是自己的名字和两个超类名字的复合体。当你创建 `Sub` 实例并打印它，就会自动得到从 `ListInstance` 混入的自定义表示：`$ python3 listinstance.py <Instance of Sub, address 0x10caae300: data1='code'data2='Python'data3=3.12 >` 负责这个显示的 `ListInstance` 类，在它混入的任何类里都能工作，因为 `self` 引用的是拉入这个类的子类的实例——无论那是谁。再次强调，从某种意义上说，混入类就是“类形式的模块”——对多种客户端有用的方法包。除了提供实用功能，混入类也和所有类一样优化了代码维护。比如，你以后决定扩展 `ListInstance` 的 `str`，让它也打印实例继承的所有类属性——没问题；因为它是继承来的方法，修改 `str` 会自动更新每个导入并混入它的子类的显示。既然现在“以后”已经正式到来，让我们进入下一节，看看那样一个扩展可能长什么样。

深度理解

·核心概念：`ListInstance` 是“方法包 + 状态访问”的完美范例：它无实例、无状态，只是提供一个访问 `self` 的 `str`。混入后，客户类立即获得自定义打印能力——这是横向复用（cross-cutting reuse）的典型。

·底层实现：`sorted(self.dict)` 对实例属性字典按键排序（Python 3.7+ 字典保序，但排序输出更友好）；`f'{value!r}'` 用 `repr` 形态嵌入值（给字符串加引号，区分 `'code'` 和数字）；`id(self):#x` 格式化十六进制地址。`attrnames` 被

改写为 `ListInstanceattrnames`，避免与客户类属性冲突

。

·设计原因：为什么用 `str` 不用 `repr`？① 展示面向人（友好），而 `repr` 面向解释器（要能重建对象）；② 免于 `repr` 的递归循环陷阱（后面 section 会看到：用 `repr` 时 `getattr` 返回绑定方法，会无限递归）。`str` 只在 `print/str` 触发，交互回显仍走 `repr`。

·实际问题：`testmixin.py` 展示了“测试工厂”：`tester(listerclass)` 把 `lister` 类当作参数混入测试子类；`testByNames` 用 `importlib.importmodule`（字符串→模块）和 `getattr`（字符串→类）组成反射式测试器——再次呼应本章工厂主题。`f'\n{'-' 80}\n'` 用了 Python 3.12 的复用引号 `f-string` 语法。

·初学者误区：① 混入类也要写 `init`？——不需要也不必调用（`ListInstance` 没有 `init`，`Sub` 调用 `Super.init` 即可）；② 以为混入类能单独 “new” 出来用——它设计上不是给客户端直接实例化的；③ 忘记 `!r` 与排序会让输出可读性天差地别。

代码分析

```
class ListInstance:
    """ __str__ print() str()
    self __X
    slots slots __dict__ """
    def __attrnames(self):          # _ListInstance_attr...
        result = '\n'
        for attr in sorted(self.__dict__):      #
            result += f'\t{attr}={self.__dict__[attr]!r}\n'    # !r...
```

```

        return result
    def __str__(self):
        return (f'<Instance of {self.__class__.__name__}, ' # ...
                f'address {id(self):#x}:' # id ...
                f'{self.__attrnames()}>') # ...

```

·解释器如何处理：print(instance) 时，print 内部调用 str(instance)，触发查找 str：沿 MRO (Sub, Super, ListInstance, object) 在 ListInstance.dict 命中。方法体里 self.class 是 Sub (实例的类)，self.class.name 是 'Sub'；self.dict 是实例命名空间 (data1/data2/data3)；attrnames() 调用时，attrnames 已在编译期被改写为 ListInstance.attrnames，在 self 上也找到。整个显示字符串一次拼装完成，无需格式化覆盖。

·设计思想：通用工具类通过 self 观察任意实例的结构 (class、dict、id)，这是"自省 (introspection)"在工具开发中的威力。一个纯逻辑类，就能服务所有客户类——复用的边界由"接口协议"而非"类型体系"定义。

31.22 用 dir 列出继承属性 (Listing inherited attributes with dir)

英文原文

As it is, our ListInstance mix-in displays instance attributes only—names attached to the instance object itself. It's nearly trivial, though, to extend the class to display all the attributes accessible from an instance—both its own and those it inherits from its classes. Th

e trick is to use the `dir` built-in function instead of scanning the instance's `dict` dictionary; the latter holds instance attributes only, but the former also collects all inherited attributes.

The mutation in Example 31-12, `listinherited.py`, codes this scheme. This is located in its own module to facilitate testing, but if existing clients were to use this version instead, they would pick up the new display automatically (and recall from Chapter 25 that a `from import's` `as` clause can rename a new version to a prior name being used).

Example 31-12. `listinherited.py`

```
python class ListInherited: """Use dir() to collect both
instance attrs and names inherited from its classes. This
includes default names inherited from the implied
'object' superclass above topmost classes. getattr() can
fetch inherited names not in self.dict. Caution: use
str, not repr, or else this loops when printing bound
methods that may be returned for some attributes by
getattr().
```

This will normally fail for class "slots" attributes not yet assigned.

```
"""
```

```
def attrnames(self, unders=False): result = '\n'
for attr in dir(self): # Instance dir() if attr[:2] == '_' and attr[-2:]
== '_': # Built-in names result += f'\t{attr}\n' if unders
```

```

else'' # Skip built-ins?

else: result += f'\t{attr}={getattr(self, attr)!r}\n'

return result
def str(self): return (f'<Instance of {self.class.name}, '
# My class's name
f'address {id(self):#x}: '
# My address (hex)
f'{self.attrnames()}>') # name=value list

if name == 'main': import testmixin
testmixin.testmixin(ListInherited) # Test class in this module

```

Notice that this code, by default, skips all X names for brevity. There are 27 such names in the test case, and most of these are internal names that we don't generally care about in a generic listing like this. Some are user-defined operator-overloading methods like our `str`, though most reflect defaults in the implicit object root class, and there's no easy way to determine an attribute's class of origin here (but stay tuned).

Because `dir` results may include names from anywhere in a class tree, this version also must use the `getattr` built-in function to fetch attributes by name string instead of indexing the instance's dict attribute dictionary. `getattr` runs inheritance search, and some of the names we're listing here are not stored on the instance itself.

To test the new version, run its file directly—it passes the `ListInherited` class it defines to the `testmixin.py` file's test function in Example 31-11 to be mixed in with

h a subclass in the function. Here's the output of this test and lister class; notice the extra names inherited from classes and the name mangling at work in the lister's method name:

```
$ python3 listinherited.py <Instance of Sub, address 0x101d76600: ListInheritedattrnames= <bound method ListInherited.attrnames of <...>>
```

```
data1='code'data2='Python'data3=3.12 method1= <bound method tester.<locals>.Super.method1 of <testmixin.tester...>> method2= <bound method tester.<locals>.Sub.method2 of <testmixin.tester...>> >
```

The display of bound methods in this was truncated to fit this page; here's what the first two look like with an added line break (as usual, run on your own for the full and up-to-date picture):

```
ListInheritedattrnames= <bound method ListInherited.attrnames of <testmixin.tester.<locals>.Sub object at 0x101d76600>>
```

```
method1= <bound method tester.<locals>.Super.method1 of <testmixin.tester.<locals>.Sub object at 0x101d76600>>
```

Display formatting is an open-ended task (e.g., Python's standard pprint "pretty printer" module may offer options here too), so we'll leave further polishing as a suggested exercise. The tree lister of the next section may be more useful in any event, so let's move o

n.

NOTE — Looping in repr: Now that we're displaying inherited methods too, we must use str instead of repr to overload printing. With repr, this code (and code like it) will fall into recursive loops—its getattro returns a bound method; whose display includes the instance; which triggers repr again to display the instance; and so on, quickly triggering a stack-overflow exception. Subtle, but true! Change str to repr to see this live. One way to avoid such repr loops is to skip getattro results for which isinstance comparisons to the standard library's types.MethodType are true. Using str instead is simpler.

中文翻译

就现状而言，我们的 ListInstance 混入类只显示实例属性——附加在实例对象本身上的名字。不过，把这个类扩展到显示实例所有可访问的属性（既有自己的，也有从类继承来的），几乎是小菜一碟。诀窍是用内置函数 dir 而不是扫描实例的 dict 字典；后者只装实例属性，而前者还会收集所有继承的属性。示例 31-12 中的变体 (listinherited.py) 编码了这个方案。它放在独立模块里以方便测试，但如果现有客户端改用这个版本，他们会自动获得新的显示（回想第 25 章，from 导入的 as 子句可以把新版本改名成正在使用的旧名字）。示例 31-12. listinherited.py

```
python class ListInherited: """ Use dir() to collect both instance attributes and names inherited from its classes. This includes default names inherited from the implied 'object' su
```

perclass above topmost classes. `getattr()` can fetch in herited names not in `self.dict`. Caution: use `str`, not `repr`, or else this loops when printing bound methods that may be returned for some attributes by `getattr()`. This will normally fail for class "slots" attributes not yet assigned.

```

"""
def attrnames(self, unders=False):
    result = '\n'
    for attr in dir(self):
        # 实例的 dir() if attr[:2] == '
        ' and attr[-2:] == ': # 内置名字
        result += f'\t{attr}\n' if
        unders else '' # 跳过内置名?
    else:
        result += f'\t{attr}={
        getattr(self, attr)!r}\n'
    return result
def str(self):
    return
    (f'<Instance of {self.class.name}, ' # 我的类名
    f'address {
    id(self):#x}: ' # 我的地址 (十六进制)
    f'{self.attrnames()}
    >') # name=value 清单
if name == 'main':
    import test
    mixin
    testmixin
    tester(ListInherited) # 测试本模块中的
    这个类
    注意这段代码默认跳过所有 X 名字以保持简洁。
    测试用例里有 27 个这样的名字，其中大部分是我们这种通用
    清单一般不在意的内部名字。有些是用户自定义的运算符重载
    方法（比如我们的 str），但大部分反映隐式 object 根
    类的默认实现，而且这里没有简便方法确定属性的起源类（
    但请继续看下去）。因为 dir 的结果可能包含来自类树任意
    位置的名字，这个版本还必须用内置函数 getattr 按名字字
    符串取属性，而不是索引实例的 dict 属性字典。getattr 会
    执行继承搜索，而我们这里列出的有些名字并不存储在实例
    本身。要测试新版本，直接运行它的文件——它把定义的 L
    istInherited 类传给示例 31-11 的 testmixin.py 文件里的
    测试函数，与函数中的子类混合。下面是这次测试和 lister
    类的输出；注意从类继承来的额外名字，以及 lister 方法名
    上正在生效的名称改写：
    $ python3 listinherited.py <In

```


stance of Sub, address 0x101d76600: ListInheritedattr
names=<bound method ListInherited.attrnames of <
...>> data1='code'data2='Python'data3=3.12 method
1=<bound method tester.<locals>.Super.method1 of
<testmixin.tester...>> method2=<bound method tes
ter.<locals>.Sub.method2 of <testmixin.tester...>> >
这里的绑定方法显示被截断以便排版；加上一个换行后，前
两个看起来是这样（照例，完整的、最新的样子请自己运行
）： ListInheritedattrnames=<bound method ListInh
erited.attrnames of <testmixin.tester.<locals>.Sub ob
ject at 0x101d76600>> method1=<bound method te
ster.<locals>.Super.method1 of <testmixin.tester.<lo
cals>.Sub object at 0x101d76600>> 显示格式化是个开
放式的任务（比如 Python 标准库的 pprint“漂亮打印”模
块在这里也可能提供选项），所以进一步的打磨我们留作建
议练习。无论如何，下一节的树形 lister 可能更有用，我们
继续。注意——repr 中的循环：既然我们开始显示继承的
方法，就必须用 str 而不是 repr 来重载打印。用 repr 时，
这段代码（及类似代码）会陷入递归循环——它的 getattr
返回一个绑定方法；而该方法的显示包括实例；这又触发 r
epr 来显示实例；如此往复，很快触发栈溢出异常。很微妙
，但千真万确！把 str 改成 repr 现场看看。避免这种 repr
循环的一种办法是：跳过那些与标准库 types.MethodTyp
e 做 isinstance 比较为真的 getattr 结果。改用 str 更简单
。

深度理解

·核心概念：从“实例属性”升级到“全部属性”只需换一个函

数: dict → dir。dir 返回的是"名字清单", 包含继承来的属性和 object 的内置默认名; 取值则必须改用 getattr (会做继承搜索)。

·底层实现: dir(self) 按 MRO 合并所有类的 dict 键与实例 dict 键, 去重后排序。getattr(self, attr) 对每个名字执行完整属性查找——这就是为什么输出里出现绑定方法 (显示时是 <bound method ...>)。

·设计原因: 这个示例埋了个重要的"坑教学"——为何必须用 str 而非 repr。用 repr 时: getattr 返回绑定方法 → 显示它需要调用它的 repr → 绑定方法的显示包含实例 → 又触发 repr (因为实例在另一个字符串的格式化上下文中会被 repr) → 无限递归 → RecursionError。用 str 则只在 print/str 触发, 避开链条。

·实际问题: dir 输出里 27 个 X 名字大多来自 object 的默认实现 (dict、doc、module.....)。用 attr[:2] == " and attr[-2:] == " 判断"双下划线首尾"以跳过。注意这也会跳过用户自己定义的 str 等特殊方法——是取舍。

·初学者误区: ① 用 getattr(self, attr) 却忘了它可能返回绑定方法, 导致意外的递归或格式爆炸; ② 以为 dir 只返回实例自己的属性; ③ 写 attr.startswith("") and attr.endswith("") 忘了还会误伤 module 之类——其实那正是"跳过"目标。

代码分析

```

class ListInherited:
    """ dir() 'object'
    getattr() self.__dict__
    __str__ __repr__ getattr()

    "slots" """
    def __attrnames(self, unders=False):
        result = '\n'
        for attr in dir(self):                # dir()
            if attr[:2] == '__' and attr[-2:] == '__':    # /
                result += f'\t{attr}\n' if unders else ' '    # ...
            else:
                result += f'\t{attr}={getattr(self, attr)!r}\n' #...
        return result

```

·解释器如何处理：dir(self) 是 C 层函数，调用对象的 dir（默认实现在 object），合并实例与各类的命名空间并去重排序。对每个普通名字执行 getattr(self, attr)：对实例属性走 dict；对类方法返回绑定方法对象；!r 又触发这些对象的 repr——绑定方法的 repr 里会包含其实例的地址信息（of <... object at 0x...>），但不递归打印实例属性，所以安全；若换成 repr 方案，实例自身的 repr 会再次进入 attrnames/格式化逻辑，造成无限循环。

·设计思想：注意 docstring 里诚实标注了局限：对“尚未赋值的 slots 属性”会失败（slots 描述的属性不在 dict，未赋值时 getattr 会抛 AttributeError）。工具代码写清边界是专业素养的体现。

31.23 在类树中按对象列出属性 (Listing attributes per object in class trees)

英文原文

Let's code one last extension. As it is, our latest lister includes inherited names but doesn't give any sort of designation of the classes from which the names are acquired. As we saw in the classtree.py example near the end of Chapter 29, though, it's straightforward to climb class inheritance trees in code.

The mix-in class in Example 31-13, coded in file listtree.py, makes use of this same technique to display attributes grouped by the classes in which they live—it sketches the full physical class tree, displaying attributes attached to each object along the way. The reader must still infer attribute inheritance (and we'll address this in the next section's final demo), but this version gives substantially more detail than a simple flat list of attributes, inherited or not.

Example 31-13. listtree.py

```
python class ListTree: """Mix-in that returns a str trace of the entire class tree and all its objects' attrs at and above the self instance. The display is run by print() automatically; use str() to fetch as a string. This: -Uses X pseudoprivate attr names to avoid conflicts with clients -Recurses to superclasses explicitly in DLFR (though not MRO) order -Uses dict instead of dir() because attrs are listed per object -Supports classes with slots: lack of slots here ensures a dict"""
```

```
def attrnames(self, obj, indent, unders=True): spaces
    = " (indent + 1) result = "for attr in sorted(obj.dict): if
    attr.startswith('') and attr.endswith(''): if unders: result
    += f'{spaces}{attr}\n'else: result += f'{spaces}{attr}={g
    etattr(obj, attr)!r}\n'
```

```
return result def listclass(self, aClass, indent): dots = '.
' indent preamble = (f'\n{dots}'f'<Class {aClass.name}
'f', address {id(aClass):#x}')
```

```
if aClass in self.visited: return preamble + ': (see abov
e)>\n' # Already listed elif aClass is object: self.visited
[aClass] = True return preamble + ': (see dir(object))>
\n' # Skip object's 24 else: self.visited[aClass] = True
here = self.attrnames(aClass, indent) # My attrs + su
pers above = "for Super in aClass.bases: above += sel
f.listclass(Super, indent + 4)
```

```
return preamble + f':\n{here}{above}{dots}>\n' def str(
self): self.visited = {} here = self.attrnames(self, 0) #
My attrs above = self.listclass(self.class, 4) # My supe
rs tree return (f'<Instance of {self.class.name}' # My cl
ass's name f', address {id(self):#x}' # My address (hex)
f':\n{here}{above}>') # attrs + supers
```

```
if name == 'main': import testmixin testmixin.testter(Li
stTree) # Test class in this module
```

This class achieves its goal by traversing the inheritance tree—from an instance's class to its class, and then from the class's bases to all superclasses recursively, scanning each object's attribute dict along the way.

y to enumerate attributes. Ultimately, it concatenates each tree portion's string as the recursion unwinds.

It can take a few moments to understand recursive programs like this, but given the arbitrary shape and depth of class trees, we really have no choice here (apart from explicit stack equivalents of the sorts we met in Chapters 19 and 25, which tend to be no simpler, and which we'll omit here for space and time). This class is coded to keep its business as explicit as possible, though, to maximize clarity.

To test, run this class's module file as before; it passes the ListTree class to testmixin.py of Example 31-11 again, to be mixed in with a subclass in the test function. The file's tree-sketcher output is as follows:

```
$ python3 listtree.py <Instance of Sub, address 0x10a45f980:
```

```
ListTreevisited={} data1='code'data2='Python'data3=3.12 ....<Class Sub, address 0x7fb26a448530: doc init module
```

```
method2= <function tester.<locals>.Sub.method2 at 0x10a482ac0> .....<Class Super, address 0x7fb26a445c00: dict doc init module weakref
```

```
method1= <function tester.<locals>.Super.method1 at 0x10a4823e0> .....<Class object, address 0x10a01b100: (see dir(object))> .....> .....<Class ListTree, address 0x7fb26a443d20: ListTreeattrnames= <functio
```

```
n ListTree.attrnames at 0x10a480f40> ListTreelistclas  
s=<function ListTree.listclass at 0x10a480fe0> dict d  
oc module str weakref
```

```
.....<Class object, address 0x10a01b100: (see above  
e)> .....> ....> >
```

Some points to notice about this example: - The visited table's name is mangled in the instance's attribute dictionary for pseudoprivacy; unless we're very unlucky, this won't clash with other data there. Some of the lister class's methods are mangled as well. - To minimize displays, X attributes are listed by name only, skipping their values.

The built-in object class implied above all topmost classes is also singled out to simply refer readers to its dir result; object comes with 24 attributes today, which wouldn't be useful to repeat in every display. - The attributes that were bound methods in the prior version are now plain functions.

This reflects the fact that this version fetches methods from classes instead of the instance—the getattro here is run on the current tree object whose dict is being scanned (getattro and dict indexing are equivalent in this context). Again, class methods are just functions, which are bound only when fetched from an instance.

To avoid listing a class object more than once, a table records classes visited so far. A dictionary works in

this role because class objects are hashable and thus may be dictionary keys; a set would work similarly.

On the last point, cycles are not generally possible in class inheritance trees—a class must already have been defined to be named as a superclass, and Python raises an exception as it should if you attempt to create a cycle later by bases changes:

```
>> class C: pass >> class B(C): pass >> C.bases = (B, )  
# Dark magic TypeError: a bases item causes an inheritance cycle
```

The visited mechanism here is still needed, though, to avoid relisting a class twice: For more fun, try mixing this class into something more substantial, like the Button class of Python's tkinter GUI-toolkit module. In general, you'll want to name ListTree first (leftmost) in a class header, so its str is picked up; Button has one, too, and the leftmost superclass is searched first in multiple inheritance's default:

```
>> from tkinter import Button >> from listtree import ListTree  
>> class ButtonPlus(ListTree, Button): pass  
# ListTree's str, not Button's >> print(ButtonPlus())  
...our class's display... >> class ButtonPlus(Button, ListTree): pass  
# Mix-in order can matter! >> print(ButtonPlus()).!buttonplus2
```

Order matters in multiple inheritance, though the manual overrides we explored earlier can force the issue

e:

```
>> class ButtonPlus(Button, ListTree): str = ListTree.str
```

...our class's display... You might also try running testmixin.py of Example 31-11 directly; its self-test code that we shelved earlier runs each of our three lister classes in turn, using their module and class name strings—a trivial class factory in action:

```
$ python3 testmixin.py ...all three listers' results... While our tree lister works as planned, it doesn't really follow Python's MRO ordering through class trees with diamonds. In fact, it really just sketches the DFLR order and leaves it to readers to determine from which class a given attribute is inherited. To do better, let's move on to a final example to close out this chapter.
```

中文翻译

让我们编写最后一个扩展。就现状而言，我们最新的 lister 包含了继承的名字，但没有给出这些名字来自哪个类的任何标示。不过，正如我们在第 29 章末尾附近的 classtree.py 示例中看到的，在代码中攀登类的继承树是直截了当的。示例 31-13 的混入类（文件 listtree.py）用了同样的技术，按属性所属的类分组显示——它画出完整的物理类树，沿途显示挂在每个对象上的属性。读者仍需自行推断属性继承（我们会在下一节的最终演示中处理这点），但这一版提供的信息比简单平铺的属性清单（无论是否继承）丰富得多。示例 31-13. listtree.py

```
python class ListTree: """ Mix-i
```

n that returns a str trace of the entire class tree and all its objects' attrs at and above the self instance. The display is run by print() automatically; use str() to fetch as a string. This:-

- Uses X pseudoprivate attr names to avoid conflicts with clients
- Recurses to superclasses explicitly in DLFR (though not MRO) order
- Uses dict instead of dir() because attrs are listed per object
- Supports classes with slots: lack of slots here ensures a dict

```
def attrnames(self, obj, indent, unders=True):
    spaces = " " * (indent + 1)
    result = ""
    for attr in sorted(obj.__dict__):
        if attr.startswith('_') and attr.endswith('__'):
            if unders:
                result += f'{spaces}{attr}\n'
            else:
                result += f'{spaces}{attr}={getattr(obj, attr)!r}\n'
    return result

def listclass(self, aClass, indent):
    dots = '.'
    indent_preamble = (f'\n{dots}' * f'<Class {aClass.name}' * f', address {id(aClass):#x}'
    )
    if aClass in self.visited:
        return indent_preamble + ': (see above)>\n'
    # 已列出过
    elif aClass is object:
        self.visited[aClass] = True
        return indent_preamble + ': (see dir(object))>\n'
    # 跳过 object 的 24 个属性
    else:
        self.visited[aClass] = True
        here = self.attrnames(aClass, indent)
        # 我的属性 + 超类
        above = ""
        for Super in aClass.bases:
            above += self.listclass(Super, indent + 4)
        return indent_preamble + f':\n{here}{above}{dots}>\n'

def str(self):
    self.visited = {}
    here = self.attrnames(self, 0)
    # 我的属性
    above = self.listclass(self.class, 4)
    # 我的超类树
    return (f'<Instance of {self.class.name}'
    # 我的类名
    f', address {id(self):#x}'
    # 我的地址 (十六进制)
    f':\n{here}{above}>')
    # 属性 + 超类

if name == 'main':
    import testmixin
    testmixin.testmixin().tester(List
```

Tree) # 测试本模块中的这个类这个类通过遍历继承树实现目标——从实例的 class 到它的类，再从类的 bases 递归到所有超类，沿途扫描每个对象的属性 dict 来枚举属性。最终，随着递归展开 (unwind)，把每部分树的字符串拼接起来。理解这种递归程序需要花点时间，但考虑到类树形状和深度的任意性，我们别无选择（除了我们第 19、25 章见过的显式栈等价物——那往往也不简单，出于篇幅和时间考虑这里省略）。不过，这个类被编码得尽量显式，以最大化可读性。要测试，照例运行这个类的模块文件；它再次把 ListTree 类传给示例 31-11 的 testmixin.py，与测试函数中的子类混合。文件画树器的输出如下：

```
$ python3 list
tree.py <Instance of Sub, address 0x10a45f980: ListTr
eevisited={} data1='code'data2='Python'data3=3.12 .
...<Class Sub, address 0x7fb26a448530: doc init mod
ule method2=<function tester.<locals>.Sub.method2
at 0x10a482ac0> .....<Class Super, address 0x7fb26a
445c00: dict doc init module weakref method1=<fun
ction tester.<locals>.Super.method1 at 0x10a4823e0
> .....<Class object, address 0x10a01b100: (see dir
(object))> .....> .....<Class ListTree, address 0x7fb26
a443d20: ListTreeattrnames=<function ListTree.attrn
ames at 0x10a480f40> ListTreelistclass=<function Lis
tTree.listclass at 0x10a480fe0> dict doc module str w
eakref .....<Class object, address 0x10a01b100: (se
e above)> .....> ....> > 关于这个示例要注意几点：- visi
ted 表的名字在实例属性字典里因伪私有而被改写；除非我
们运气极差，否则不会与那里的其它数据冲突。lister 类的一
些方法也被改写。- 为精简显示，X 属性只按名字列出，
```

跳过它们的值。在最顶层类之上隐含的内置 object 类也被单独拎出来，只是指引读者去看它的 dir 结果；object 如今自带 24 个属性，在每次显示里重复它们没意义。- 上一版里还是绑定方法的属性，现在变成了普通函数。这反映了一个事实：这一版是从类而不是实例取方法——这里的 get_attr 运行在当前正被扫描 dict 的树对象上（这个语境下 get_attr 与 dict 索引等价）。再说一次，类方法只是函数，只有从实例取回时才会被绑定。为避免同一个类对象被列出多次，一个表记录了截至目前访问过的类。字典胜任这个角色，因为类对象可哈希（hashable），因此可以作为字典键；用集合也类似可行。关于最后一点：类的继承树一般不可能有环——一个类必须先被定义才能被指名作超类，而如果你试图事后通过 bases 改动制造环，Python 会如它所应当地抛出异常：

```
>>> class C: pass
>>> class B(C): pass
>>> C.bases = (B,) # 黑暗魔法TypeError: a bases item causes an inheritance cycle
```

这里仍需 visited 机制，是为了避免一个类被列出两次：更有趣的是，试试把这个类混入更实在的东西，比如 Python 的 tkinter GUI 工具包模块的 Button 类。一般来说，你要把 ListTree 放在 class 头行的第一个（最左侧），这样它的 str 才会被选中；Button 也有一个 str，而多重继承默认先搜最左侧的超类：

```
>>> from tkinter import Button
>>> from listtree import ListTree
>>> class ButtonPlus(ListTree, Button): pass
# 用 ListTree 的 str，而非 Button 的
>>> print(ButtonPlus()) ...我们的类的显示...
>>> class ButtonPlus(Button, ListTree): pass
# 混入顺序可能影响结果！
>>> print(ButtonPlus())
```

!buttonplus2 在多重继承里顺序很重要，不过我们之前探讨的手动覆盖可以强制解决：

```
>>> class Butto
```

nPlus(Button, ListTree): str = ListTree.str ...我们的类的显示... 你也可以直接运行示例 31-11 的 testmixin.py; 我们先前搁置的自测代码会轮流运行我们三个 lister 类, 用到它们的模块和类名字字符串——一个简单类工厂的实际应用: \$ python3 testmixin.py ...三个 lister 的全部结果... 虽然我们的树形 lister 按计划工作, 但它在带有菱形的类树中并不真正遵循 Python 的 MRO 顺序。事实上, 它只是画出 DFLR 顺序, 把"某个属性到底从哪个类继承"留给读者判断。为了做得更好, 让我们进入本章收尾的最后一个示例。

深度理解

- 核心概念: ListTree 把混入工具提升到"递归遍历类树"的层次: 自底向上把实例、类、超类、再超类.....的属性按归属分组展示。它用 dict (按对象看) 而非 dir (按实例全景看), 并用点号缩进表示树深。
- 底层实现: 递归函数 listclass(aClass, indent): 对每个类打出 preamble 头, 然后 for Super in aClass.bases 对每个直接超类递归 (缩进加深 4)。visited 字典记录已访问类防止重复输出。str 先列实例自身属性, 再递归列出 self.class 的超类树。
- 设计原因: 为什么要 visited 表? 类树原则上无环, 但共享超类 (菱形) 会让同一个类被两条路径访问——需要去重。C.bases = (B,) 造环会被 Python 拒绝 (循环依赖保护), 但共享类仍然合法。
- 实际问题: 这个例子真实展示了混入顺序如何影响行为: Button 也有 str, ListTree 写在左边则 MRO 先找它; 写右边则用 Button 的。硬要控制可以 str = ListTree.str 显

式强制——正是 31.19 节的"显式覆盖"实战场。

·初学者误区：① 递归程序的"怎么控"困惑——建议跟一手纸笔推演 Sub → Super/ListTree → object 路径；② 以为 dict 和 dir 等价——前者是"该对象自己的"，后者是"全部可访问的"；③ 忘了绑定方法只在"从实例取"时产生——从类取是普通函数，输出里方法显示为 <function ...>。

代码分析

```
def __listclass(self, aClass, indent):
    dots = '.' * indent                                # '.'
    preamble = (f'\n{dots}'
                f'<Class {aClass.__name__}'
                f', address {id(aClass):#x}'))
    if aClass in self.__visited:                        #
        return preamble + ': (see above)>\n'
    elif aClass is object:                             # object 24 ...
        self.__visited[aClass] = True
        return preamble + ': (see dir(object))>\n'
    else:
        self.__visited[aClass] = True
        here = self.__attrnames(aClass, indent)        #
        above = ''
        for Super in aClass.__bases__:                  #
            above += self.__listclass(Super, indent + 4)
        return preamble + f'\n{here}{above}{dots}>\n'
```

·解释器如何处理：str 调 listclass(self.class, 4) (缩进 4，即 4 个点)。对 Sub：未访问 → 列出 Sub.dict (含 ListTreevisited 的实例键、init、method2 等)；再对 bases 里的 Super 和 ListTree 递归。每个递归层缩进 +4，形成树形嵌套。aClass in self.visited 用类对象做字典键

查找（类可哈希）。递归直到 object 分支或重复分支，返回后逐层拼接，dots> 作为每层的闭合标记。注意 get_attr(obj, attr) 的对象是本层类，故方法显示为函数而非绑定方法。

·设计思想：递归 + 备忘录（memoization）是"任意形状树"的标准处理手段；混入类 + self 自省让这个工具对任何客户类通用。示例结尾主动承认局限（按 DFLR 而非 MRO），为下一节 mapattrs 的"按 MRO 精确归属"埋下伏笔——这是严谨工程态度的范本。

31.24 示例：将属性映射到继承来源 (Example : Mapping Attributes to Inheritance Sources)

英文原文

This section wraps up with an example that demos an application for the MRO in programming tools. As coded, the preceding section's tree lister gave the physical locations of attributes in a class tree. However, by mapping the list of inherited attributes in a dir result to the linear MRO sequence, such tools can more directly associate attributes with the classes from which they are actually inherited—also a useful relationship for programmers.

We won't recode our tree lister in full here, but as a first major step, Example 31-14, file mapattrs.py, imple

ments tools that can be used to associate attributes with their inheritance source. As an added bonus, its `mapattrs` function demonstrates how inheritance actually searches for attributes in class tree objects—though the MRO is largely automated for us with the built-in `mro` class attribute we met earlier.

Example 31-14. `mapattrs.py`

```
python """Main tool: mapattrs() maps all attributes on or inherited by an instance to the instance or class from which they are inherited. Also here: assorted dictionary tools using comprehensions. Assumes dir() gives all attributes of an instance. To emulate inheritance, this uses the class's mro tuple, which gives the MRO search order for classes in Python 3.X. A recursive tree traversal for the DFLR order of classes is included but unused.
```

```
"""  
  
import pprint  
def trace(label, X, end='\n'): print(f'{label}\n{pprint.pformat(X)}{end}') # Print nicely  
def filterdictvals(D, V): """dict D with entries for value V removed. filterdictvals(dict(a=1, b=2, c=1), 1) => {'b': 2}"""  
    return {K: V2 for (K, V2) in D.items() if V2 != V}  
def invertdict(D): """dict D with values changed to keys (grouped by values). Values must all be hashable to work as dict/set keys. invertdict(dict(a=1, b=2, c=1)) => {1: ['a', 'c'], 2: ['b']}"""
```



```
def keysof(V): return sorted(K for K in D.keys() if D[K]
== V) return {V: keysof(V) for V in set(D.values())} def
dflr(cls): """Depth-first left-to-right order of class tree
at cls. Cycles not possible: Python disallows on bases
changes.
```

```
"""
```

```
here = [cls] for sup in cls.bases: here += dflr(sup)
```

```
return here def inheritance(instance): """Inheritance o
rder sequence: MRO or DFLR. DFLR alone is no longe
r used in Python 3.X.
```

```
"""
```

```
if hasattr(instance.class,'mro'): return (instance,) + ins
tance.class.mro else: return [instance] + dflr(instance.
class) def mapattrs(instance, withobject=False, bysou
rce=False): """dict with keys giving all inherited attrib
utes of instance,
```

```
with values giving the object that each is inherited fr
om. withobject: False=remove object built-in class att
ributes. bysource: True=group result by objects inste
ad of attributes. Supports classes with slots that precl
ude dict in instances.
```

```
"""
```

```
attr2obj = {} inherits = inheritance(instance) for attr i
n dir(instance): for obj in inherits: if hasattr(obj,'dict')
and attr in obj.dict: # Slots okay attr2obj[attr] = obj b
reak
```

```

if not withobject: attr2obj = filterdictvals(attr2obj, obj
ect) return attr2obj if not bysource else invertdict(attr
2obj) if name == 'main': class D: attr2 = 'D' class C(D):
attr2 = 'C' class B(D): attr1 = 'B' class A(B, C): pass l = A(
) l.attr0 = 'l' print(f'Py=>{l.attr0=}, {l.attr1=}, {l.attr2=}\
n') # Python's search trace('INHERITANCE', inheritanc
e(l)) # [Inheritance order] trace('ATTRIBUTES', mapattr
rs(l)) # {Attr => Source} trace('SOURCES', mapattrs(l,
bysource=True)) # {Source => [Attrs]}

```

This module's main `mapattrs` function uses `dir` to collect all the attributes that an instance inherits. For each, it maps the attribute to its source by scanning the MRO order available in the `mro` of the instance's class, searching each object's namespace dict along the way. The net effect replicates Python's true inheritance search for each attribute accessible from the instance passed in.

This file's self-test code applies its tools to a diamond multiple-inheritance tree similar to those we studied earlier. It uses Python's `pprint` standard-library module to display lists and dictionaries nicely—`pprint.pprint` is its basic call, and its `pformat` returns a print string. Notably, `attr2`, whose value is given on the first line and whose name appears in later function results, is inherited from class `C` per the MRO order we've studied:

```
$ python3 mapattrs.py Py=>l.attr0='l', l.attr1='B', l.a
```

```
ttr2='C'INHERITANCE (<main.A object at 0x10cc33e00>,
```

```
<class'main.A'>, <class'main.B'>, <class'main.C'>, <class'main.D'>, <class'object'>) ATTRIBUTES {'dict': <class'main.D'>,'doc': <class'main.A'>,'module': <class'main.A'>,'weakref': <class'main.D'>,'attr0': <main.A object at 0x10cc33e00>,'attr1': <class'main.B'>,'attr2': <class'main.C'>} SOURCES [<main.A object at 0x10cc33e00>: ['attr0'],
```

```
<class'main.D'>: ['dict','weakref'], <class'main.C'>: ['attr2'], <class'main.B'>: ['attr1'], <class'main.A'>: ['doc','module']] Although this module was not designed to be a mix-in class itself, listers may index its mapattrs function's dictionary results to obtain an attribute's source or a source's attributes. Moreover, it's easy to adapt this module's results to be a mix-in by wrapping them in a str. Here it is listing attributes' sources in the test classes of the prior section's Example 31-11:
```

```
$ python3 >> import pprint >> from mapattrs import mapattrs >> class ListAttr2Source:
```

```
def str(self): return pprint.pformat(mapattrs(self)) >> from testmixin import tester >> tester(ListAttr2Source) {'dict': <class'testmixin.tester.<locals>.Super'>,'doc': <class'testmixin.tester.<locals>.Sub'>,'init': <class'testmixin.tester.<locals>.Sub'>,'module': <class'testmixin.tester.<locals>.Sub'>,'str': <class'main.ListAttr2Source'>,'weakref': <class'testmixin.tester.<locals>.S
```

```
uper'>,'data1': <testmixin.testster.<locals>.Sub object  
at 0x102a8fa40>,'data2': <testmixin.testster.<locals>.S  
ub object at 0x102a8fa40>,'data3': <testmixin.testster.  
<locals>.Sub object at 0x102a8fa40>,'method1': <cla  
ss'testmixin.testster.<locals>.Super'>,'method2': <class  
'testmixin.testster.<locals>.Sub'>}
```

Listing sources' attributes is just as easy (see also Python's docs for pprint options like compact and width):

```
>> class ListSource2Attr:  
  
def str(self): return pprint.pformat(mapattrs(self, bysource=True))  
>> tester(ListSource2Attr) {<testmixin.testster.<locals>.Sub object at 0x102ad12e0>: ['data1','data2','data3'],  
  
<class'testmixin.testster.<locals>.Super'>: ['dict','weakref','method1'],  
  
<class'testmixin.testster.<locals>.Sub'>: ['doc','init','module','method2'],  
  
<class'main.ListSource2Attr'>: ['str']] Study this example's code for more insight. As callouts, notice how it uses hasattr to check whether an object has a dict attribute dictionary before trying to index it. Though rare, some instances may not have a dict if they use the class extension known as slots noted earlier. The prior section's slot story is varied: as mix-ins, ListTree and ListInstance work as is for classes with slots because t
```

their lack of slots ensures an instance dict, but `ListInherited` can fail for slots not yet assigned—findings to be clarified in the next chapter.

Additionally, both slots and other "virtual" instance attributes like properties and descriptors live at the class instead of the instance and hence may require generic handling—`dir` enumeration, and either `getattr` fetches, tree climbs, or MRO scans. This example and the prior section's listers accommodate this, but unevenly: some such names will be associated with the classes in which their implementations live, not the instance through which they are accessed.

Moreover, no lister can show attribute names dynamically computed in full by methods like `getattr` because these names have no physical basis. Classes implementing such dynamic names can also define a `dir` method to provide an attribute result list for `dir` calls, but general tools like our listers and mapper cannot depend on this optional and relatively uncommon interface being present.

Finally, all the attribute listers and mappers in this chapter work in full for normal instances but don't support classes. For the latter, `prints` run a default display instead of any of the three listers, and `mapattrs` strangely attributes most names to a mystery class called "type." The lister skips `stem` from the fact that built-ins like `print` skip the "instance," as we've noted before.

The mapattrs oddity reflects the fact that classes acquire names from both their own superclass tree (and MRO), and a secondary tree (and MRO) formed by "metaclasses" that we have yet to meet.

But to understand both the inheritance bifurcation of metaclasses, as well as ethereal attributes like slots, properties, and descriptors, we need to move on to the next chapter.

中文翻译

本节用一个示例收尾，演示 MRO 在编程工具中的应用。如前所码，上一节的树形 lister 给出的是属性在类树中的物理位置。然而，把 dir 结果中的继承属性列表映射到线性的 MRO 序列，这类工具就能更直接地把属性与它们实际继承自的类关联起来——这对程序员也是一种有用的关系。我们不在这里完整重写树形 lister，但作为重要第一步，示例 31-14（文件 mapattrs.py）实现了可用于“把属性关联到其继承来源”的工具。额外福利：它的 mapattrs 函数演示了继承在类树对象中实际如何搜索属性——不过 MRO 很大程度上由我们早前见过的内置 mro 类属性自动提供。示例 31-14. mapattrs.py

```
python """ Main tool: mapattrs(
) maps all attributes on or inherited by an instance to
the instance or class from which they are inherited. A
lso here: assorted dictionary tools using comprehensi
ons. Assumes dir() gives all attributes of an instance.
To emulate inheritance, this uses the class's mro tupl
e, which gives the MRO search order for classes in Py
thon 3.X. A recursive tree traversal for the DFLR order
```

```

of classes is included but unused. """ import pprint
def trace(label, X, end='\n'): print(f'{label}\n{pprint.pformat(X)}{end}') # 漂亮打印
def filterdictvals(D, V): """ dict D with entries for value V removed. filterdictvals(dict(a=1, b=2, c=1), 1) => {'b': 2} """
return {K: V2 for (K, V2) in D.items() if V2 != V}
def invertdict(D): """ dict D with values changed to keys (grouped by values). Values must all be hashable to work as dict/set keys. invertdict(dict(a=1, b=2, c=1)) => {1: ['a', 'c'], 2: ['b']} """
def keysof(V): return sorted(K for K in D.keys() if D[K] == V)
return {V: keysof(V) for V in set(D.values())}
def dflr(cls): """ Depth-first left-to-right order of class tree at cls. Cycles not possible: Python disallows on bases changes. """
here = [cls]
for sup in cls.bases: here += dflr(sup)
return here
def inheritance(instance): """ Inheritance order sequence: MRO or DFLR. DFLR alone is no longer used in Python 3.X. """
if hasattr(instance, 'mro'): return (instance,) + instance.class.mro
else: return [instance] + dflr(instance.class)
def mapattrs(instance, withobject=False, bysource=False): """ dict with keys giving all inherited attributes of instance, with values giving the object that each is inherited from. withobject: False=remove object built-in class attributes. bysource: True=group result by objects instead of attributes. Supports classes with slots that preclude dict in instances. """
attr2obj = {}
for attr in dir(instance):
    for obj in inheritance(instance):
        if hasattr(obj, 'dict') and attr in obj.dict: #

```

有 slots 也行

```

attr2obj[attr] = obj
break if not withobject:
    attr2obj = filterdictvals(attr2obj, object)
return attr2obj if not bysource else invertdict(attr2obj)
if name == 'main':
    class D:
        attr2 = 'D'
    class C(D):
        attr2 = 'C'
    class B(D):
        attr1 = 'B'
    class A(B, C):
        pass
    l = A()
    l.attr0 = 'l'
    print(f'Py=>{l.attr0=}, {l.attr1=}, {l.attr2=}\n')
# Python 的真实搜索
trace('INHERITANCE', inheritance(l)) # [继承顺序]
trace('ATTRIBUTES', mapattrs(l)) # {属性 => 来源}
trace('SOURCES', mapattrs(l, bysource=True)) # {来源 => [属性列表]}

```

这个模块的主函数 `mapattrs` 用 `dir` 收集实例继承的所有属性。对每个属性，它通过扫描实例类的 `mro` 中的 `MRO` 顺序，沿途搜索每个对象的命名空间 `dict`，把属性映射到它的来源。净效果是：针对从传入实例可访问的每个属性，复刻出 Python 真实的继承搜索。这个文件的自测代码把它的工具应用到与前面研究过的类似的菱形多重继承树上。它用 Python 标准库 `pprint` 模块来漂亮地显示列表和字典——`pprint.pprint` 是它的基本调用，`pformat` 返回打印字符串。值得注意的是，`attr2`（第一行给出其值、后面函数结果里也出现其名字）按我们学过的 `MRO` 顺序继承自类 `C`：

```

$ python3 mapattrs.py
Py=>l.attr0='l', l.attr1='B', l.attr2='C'
INHERITANCE (<main.A object at 0x10cc33e00>, <class'main.A'>, <class'main.B'>, <class'main.C'>, <class'main.D'>, <class'object'>)
ATTRIBUTES {'dict': <class'main.D'>,'doc': <class'main.A'>,'module': <class'main.A'>,'weakref': <class'main.D'>,'attr0': <main.A object at 0x10cc33e00>,'attr1': <class'main.B'>,'attr2': <class'main.C'>}
SOURCES {<main.A object at 0x10cc33e00>: ['attr0'], <class'main.D'>: ['di

```


ct','weakref'], <class'main.C': ['attr2'], <class'main.B': ['attr1'], <class'main.A': ['doc','module']] 虽然这个模块本身并非设计成混入类，但 lister 可以索引它的 mapattrs 函数返回的字典结果，得到"属性的来源"或"来源的属性"。而且，把这些结果包进一个 str，就能轻松把这个模块的结果改造成混入类。下面是它在上一节示例 31-11 的测试类中列出"属性的来源"：

```
$ python3 >>> import pprint
>>> from mapattrs import mapattrs
>>> class ListAttr2Source:
def str(self):
    return pprint.pformat(mapattrs(self))
>>> from testmixin import tester
>>> tester(ListAttr2Source)
{'dict': <class'testmixin.tester.<locals>.Super'>,'doc': <class'testmixin.tester.<locals>.Sub'>,'init': <class'testmixin.tester.<locals>.Sub'>,'module': <class'testmixin.tester.<locals>.Sub'>,'str': <class'main.ListAttr2Source'>,'weakref': <class'testmixin.tester.<locals>.Super'>,'data1': <testmixin.tester.<locals>.Sub object at 0x102a8fa40>,'data2': <testmixin.tester.<locals>.Sub object at 0x102a8fa40>,'data3': <testmixin.tester.<locals>.Sub object at 0x102a8fa40>,'method1': <class'testmixin.tester.<locals>.Super'>,'method2': <class'testmixin.tester.<locals>.Sub'>}
```

列出"来源的属性"同样容易（关于 pprint 的 compact、width 等选项，也请参阅 Python 文档）：

```
>>> class ListSource2Attr:
def str(self):
    return pprint.pformat(mapattrs(self, bysource=True))
>>> tester(ListSource2Attr)
{<testmixin.tester.<locals>.Sub object at 0x102ad12e0>: ['data1','data2','data3'], <class'testmixin.tester.<locals>.Super'>: ['dict','weakref','method1'], <class'testmix
```

`in.testers.<locals>.Sub': ['doc','init','module','method 2'], <class'main.ListSource2Attr': ['str']]` 仔细阅读这个示例的代码以获得更多洞见。作为要点提示，注意它如何在索引对象之前用 `hasattr` 检查对象是否有 `dict` 属性字典。虽然少见，但如果实例使用前面提过的叫 `slots` 的类扩展，它们可能没有 `dict`。上一节的 `slots` 故事很曲折：作为混入类使用，`ListTree` 和 `ListInstance` 对带 `slots` 的类开箱即用，因为它们的缺乏 `slots` 保证了实例有 `dict`，但 `ListInherited` 可能对尚未赋值的 `slots` 失败——这些结论将在下一章澄清。此外，`slots` 以及 `properties`、`descriptors` 这类“虚拟”实例属性，都活在类而非实例里，因此可能需要通用处理——`dir` 枚举，配合 `getattr` 取值、爬树或 MRO 扫描。这个示例和上一节的各种 `lister` 都兼容了这一点，但兼容得并不均匀：有些名字会被关联到它们实现所在的类，而不是访问它们的实例。再者，没有任何 `lister` 能显示出诸如 `getattr` 方法动态计算出整名的属性，因为这些名字没有物理依据。实现这种动态名字的类也可以定义 `dir` 方法为 `dir` 调用提供属性结果列表，但像我们的 `lister` 和 `mapper` 这样的通用工具，不能依赖这个可选且相对少见的接口一定存在。最后，本章所有属性 `lister` 和 `mapper` 对普通实例工作完整，但不支持类。对后者，打印运行默认显示而不是三个 `lister` 中的任何一个，而 `mapattrs` 则奇怪地把大多数名字归给一个叫“`type`”的神秘类。`lister` 的跳过源于 `print` 这类内置操作跳过了“实例”，正如我们之前提到的。`mapattrs` 的怪象反映了一个事实：类既从自己的超类树（和 MRO）获取名字，也从由“元类（`metaclass`）”构成的第二棵树（和 MRO）获取名字——元类我们还没讲到。但要理解元类造成的继承分歧（`bifurcation`），以及 `slots`、`properti`

es、descriptors 这些飘渺属性，我们需要进入下一章。

深度理解

·核心概念：mapattrs 把"继承搜索"反向工程化：对 dir 里的每个属性名，沿 mro 逐类查 dict，找到第一个藏有该名字类（或实例本身），记为 {属性名: 来源对象}。bysource=True 再反转成 {来源: [属性列表]}——两个方向都可用。

·底层实现：inheritance() 返回 (instance,) + instance.class.mro（实例在前，随后是 MRO 序列）。mapattrs 双重循环：外层 for attr in dir(instance)，内层沿继承序列找来源，命中即 break——这精确复刻了 X.attr 的查找：从实例开始、按 MRO 顺序、取第一个含该名字的对象。filterdictvals(attr2obj, object) 用字典推导删掉 object 提供的默认属性。

·设计原因：dir 只给名字、不给来源；而"这个属性到底从哪个类来"恰是理解继承的关键。把 MRO（已是线性元组）当作查询顺序，一举实现属性来源审计。工具设计上区分 dict（物理存储）与 dir（逻辑可见），并诚实处理 slots/properties 等"虚拟属性"。

·实际问题：输出揭示了细节：'attr0' 来源是实例 I；'attr1' 来自 B；'attr2' 来自 C（菱形中 MRO 让 C 先于 D）；'dict' 记录了 D（因为 object 的 dict 是描述符，实际存储由普通类的 D 定义）.....这些正是 31.18、31.19 节理论的实证。

·初学者误区：① 以为"类的所有属性都来自父类"——实例自己、元类、甚至描述符都可能贡献；② 对 `mapattrs` 作用于类对象时结果怪异（多数属性被归到 "type" 元类）感到困惑——因为类对象本身又 type 的实例，有两棵继承树；③ 忘记 `hasattr(obj,'dict')` 保护——slots 类实例抛 `AttributeError` 的隐患。

代码分析

```
def mapattrs(instance, withobject=False, bysource=False):
    attr2obj = {}
    inherits = inheritance(instance)           # (,) + __mro__ ...
    for attr in dir(instance):                  #
        for obj in inherits:                    # MRO
            if hasattr(obj, '__dict__') and attr in obj.__dict__:
                attr2obj[attr] = obj            #
                break                            #
    if not withobject:
        attr2obj = filterdictvals(attr2obj, object) # object ...
    return attr2obj if not bysource else invertdict(attr2obj)
```

·解释器如何处理：`dir(instance)` 返回按 MRO 合并去重的所有可访问名字。对每个名字，内层循环按 `inherits` 顺序检查：先实例 dict，再 A、B、C、D 各自的 dict，最后 object。 `attr in obj.dict` 是纯键查找（ $O(1)$ 字典操作），须先 `hasattr(obj,'dict')` 防 slots。 `attr2obj` 建立完成后决定是否过滤 object、是否反转分组——`invertdict` 用集合推导取所有不同值，再对每个值收集对应键并排序。

·设计思想：这是"自省 + 线性化继承"的巅峰示例：把语言的运行时机制当作可观测、可复制的对象。理解 `mapat`

trs 的每行，就等于理解了 Python 属性查找的全部秘密；同时它也为下一章的元类、描述符埋下伏笔——“继承有两棵树”的说法再次升华本章开头“OOP 三支柱”之外的深度。

31.25 其他设计相关主题 (Other Design-Related Topics)

英文原文

In this chapter, we've studied an assortment of design patterns used to combine classes in Python programs, along with the mechanism behind some of them. We've really only scratched the surface here in the design patterns domain, though. Elsewhere in this book, you'll find coverage of other design-related topics, such as: - Abstract superclasses (Chapter 29) - Decorators (Chapters 32 and 39) - Type subclasses (Chapter 32) - Static and class methods (Chapter 32) - Managed attributes (Chapters 32 and 38) - Metaclasses (Chapters 32 and 40) For even more details on design patterns, this book must delegate to other resources on OOP at large. Although patterns are important in OOP work and are often more natural in Python than other languages, they are not specific to Python itself and a subject that's often best acquired by experience.

中文翻译

在本章，我们研究了一套用于在 Python 程序中组合类的设计模式，以及其中一些模式背后的机制。不过，在设计模式这个领域，我们真的只是触及了皮毛。在本节的其他地方，你会找到其它设计相关主题的讲解，比如：- 抽象超类（第 29 章）- 装饰器（第 32 章和第 39 章）- 类型子类（第 32 章）- 静态方法与类方法（第 32 章）- 受管属性（第 32 章和第 38 章）- 元类（第 32 章和第 40 章）至于更多设计模式的细节，本书必须把读者引荐给更宏观的 OOP 资料。虽然模式在 OOP 工作中很重要，而且在 Python 中往往比在其它语言里更自然，但它们并非 Python 所特有，而且这个主题通常最好靠经验来获取。

深度理解

- 核心概念：设计模式是"语言无关"的软件工程方法论，Python 只是让它更容易落地。本章讲的是它的子集：继承/组合/委托/工厂/混入。作者明确区分"语言机制"（Python 提供）与"设计模式"（人类总结）。
- 底层实现：模式在 Python 里往往坍缩为库级或语法级极小技巧——工厂是函数，代理是 `getattr`，混入是多重继承，装饰器在 3.x 里是语法糖。
- 设计原因：Lutz 把"模式的细节"委托给专门的 OOP 书籍，是为了不稀释本书语言教学的主题——同时诚实承认"模式需靠经验习得"，避免给读者"读一遍就会设计"的错觉。
- 实际问题：学习路线建议：先掌握语言机制（本章 + 后

续各章)，再看 GoF 《设计模式》或其他 Python 模式专著练习对应，最后在真实项目里打磨直觉。

·初学者误区：① 背诵 GoF 模式的类图而不理解"这个模式解决了什么约束"；② 以为 Python 里必须照搬 Java 模式的样板代码——多数时候 Python 里更简；③ 过早担心"设计"，在玩具项目里过度设计。

31.26 章节小结 (Chapter Summary)

英文原文

In this chapter, we sampled common ways to use and combine classes to optimize their reusability and factoring benefits—what are usually considered design issues, which are often independent of any particular programming language (though Python can make them easier to implement). We studied inheritance (acquiring behavior from other classes), composition (controlling embedded objects), and delegation (wrapping objects in proxy classes), as well as the related topics of pseudoprivate attributes, bound methods, factories, multiple inheritance, and the MRO. The next chapter concludes our look at classes and OOP by surveying class-related topics that are more esoteric than most of what we've already seen. Some of its material may be of more interest to tool writers than application programmers, but it still merits a review by most people who will do OOP in Python—if not for your co

de, then for others' code you may need to understand and reuse. First, though, here's another quick chapter quiz to review.

中文翻译

在本章，我们品尝了使用和组合类的常见方式，以优化它们的可复用性与分解（factoring）收益——这些通常被视为设计问题，往往与任何特定编程语言无关（尽管 Python 会让它们更容易实现）。我们研究了继承（从其它类获取行为）、组合（控制嵌入对象）、委托（把对象包进代理类），以及相关的伪私有属性、绑定方法、工厂、多重继承和 MRO。下一章将结束我们对类和 OOP 的观察，调查一些比我们目前所见更冷僻的类相关主题。它的部分内容对工具编写者的吸引力可能大于对应用程序员，但对大多数将用 Python 做 OOP 的人来说，仍值得一读——如果不是为了你自己的代码，也是为了你可能需要理解并复用的别人的代码。不过在那之前，这里还有一个快速章节测验来回顾。

深度理解

·核心概念：本章主线即“三大关系 + 三件机制”：继承（is-a）、组合（has-a）、委托（like-a）是设计骨架；伪私有属性、绑定方法、工厂、多重继承与 MRO 是它们的实现机制。设计问题与语言机制被分开界定。

·底层实现：注意作者策略——先讲机制（第 26-30 章），再讲设计（本章），最后讲冷僻主题（第 32 章起：装饰器、元类、描述符）。下一章是“类工厂内的高级工具”，适合工具作者但与每个 OOP 使用者相关。

·设计原因：章节结构的用心：把"能直接用的"放在前面，把"锦上添花的"放在后面，保证学习者先建立可工作的心智模型。

·实际问题：作为复习自查：能否不看笔记说清 is-a / has-a / like-a 的取舍？能否口算出任意菱形继承的 mro？

·初学者误区：把这些概念当"知识点"背诵而不是当"判断工具"使用——设计的本质是"在约束下做权衡判断"。

31.27 测试你的知识：小测验 (Test Your Knowledge: Quiz)

英文原文

1. What is multiple inheritance? 2. What is composition? 3. What is delegation? 4. What are bound methods? 5. What are pseudoprivate attributes used for? 6. How does the MRO inheritance search order differ from DFLR?

中文翻译

1. 什么是多重继承？2. 什么是组合？3. 什么是委托？4. 什么是绑定方法？5. 伪私有属性是用来做什么的？6. MRO 继承搜索顺序与 DFLR 有何不同？

31.28 测试你的知识：答案 (Test Your Knowledge: Answers)

英文原文

1. Multiple inheritance occurs when a class inherits from more than one superclass; it's useful for mixing together multiple packages of class-based code. The left-to-right order in class statement headers determines the general order of attribute searches, and the MRO specializes search for diamonds with common superclasses. 2. Composition is a technique whereby a controller class embeds and directs a number of objects and provides an interface all its own; it's a way to build up larger structures with classes. 3. Delegation involves wrapping an object in a proxy class, which adds extra behavior and passes other operations to the wrapped object. The proxy generally retains the interface of the wrapped object. 4. Bound methods combine an instance and a method function; you can call them without passing in an instance object explicitly because the original instance is still available in the instance+function pair. 5. Pseudoprivate attributes, whose names begin but do not end with two leading underscores (e.g., `_X`), are used to localize names to the enclosing class. This includes both class attributes, like methods defined inside the class statement, and self instance attributes assigned inside the class's methods. Such names are expanded to include the class name, which makes them generally unique among all classes in an inheritance tree. 6. The MRO selects same-na

med attributes in a lower subclass over those in a higher common superclass in multiple-inheritance "diamond" trees—effectively searching across before up in this specific case. The DFLR is otherwise the same; in fact, the MRO is defined by starting with the DFLR order and then removing all but the last (rightmost) appearances of classes that are visited more than once. This differs from DFLR only when there are duplicates, which arise only in diamonds that have common superclasses. That said, the built-in object makes every multiple-inheritance tree a diamond, so this is a common, if implicit, occurrence.

中文翻译

1. 多重继承：指一个类从多于一个超类继承的情况；它适合把多套基于类的代码包混合在一起。class 语句头行里从左到右的顺序决定属性搜索的大体顺序，而 MRO 针对具有共同超类的菱形专门优化了搜索。2. 组合：一种控制器类嵌入并指挥一批对象、并提供一套完全属于自己的接口的技术；它是用类搭建更大结构的方式。3. 委托：把对象包进代理类，代理添加额外行为并把其它操作转交给被包装对象。代理通常会保留被包装对象的接口。4. 绑定方法：把实例和方法函数组合在一起；调用它们时无需显式传入实例对象，因为原实例仍保留在“实例 + 函数”这一对里。5. 伪私有属性：名字以两个前导下划线开头但不以双下划线结尾（如 `_X`），用于把名字本地化到所在类。这既包括类属性（如 class 语句内定义的方法），也包括在类方法内赋给 `self` 的实例属性。这些名字会被扩展以包含类名，使它们通常在所

有继承树类中唯一。6. MRO 在多重继承的"菱形"树中会选择更低子类中的同名属性，而不是更高共同超类中的——也就是在这种特定情况下先横向搜索再向上搜索。DFLR 其余情况下都一样；事实上，MRO 的定义就是：从 DFLR 顺序出发，然后删除被多次访问的类的所有重复出现（只保留最后、最右侧那次）。只有当存在重复——即只发生在有共同超类的菱形中——时才与 DFLR 不同。话虽如此，内置 object 使每一棵多重继承树都成为菱形，所以这其实是常见（尽管是隐式的）情形。

本章总结

技术拓展 (Technical Expansion)

实际项目中的应用场景

- 继承 (is-a)：ORM 模型继承（如 Django 的多表继承/抽象基类）、异常体系（Exception → ValueError → 自定义异常）、测试基类。适用前提：子类确实"是一种"父类，且父类的公共接口稳定。
- 组合 (has-a)：绝大多数"聚合根-子对象"场景——订单包含明细、店铺包含员工、组件化 UI（一个窗口组合多个控件）。现代工程实践强烈建议先选组合：类边界清晰、依赖注入容易、单测方便，且避免了深继承树。
- 委托 (like-a)：日志/性能监控包装、访问控制代理、数据库游标包装、缓存代理、装饰器工厂。getattr 让代理对"未来方法"自动兼容。

·工厂：插件架构、配置驱动的动态实例化（importlib + getattr + 类对象参数）、依赖注入容器（如 injector、fastapi 的 Depends 就是运行时解析的工厂在框架层的体现）。

·混入类：Django/DRF 的 Mixin、list/dict 的能力混入、通用序列化器 mixin、日志 mixin——凡“横切关注点”（打印、序列化、验证、日志）都适合做成 mixin。

·MRO 意识：多继承的排列顺序即优先级；动态语言的调试神器类.mro、inspect.getmro()。

与其他语言（Java/C++）的区别

设计维度	Python	Java	C++
继承实现	运行时属性查找 + mro 线性化	编译期类型系统 + 虚方法表	编译期 + 虚表，多重继承靠虚
多态	鸭子类型（接口即约定）	接口（interface）显式声明	函数重载 + 虚函数 + 模板
封装	约定（X）+ 改写（X），无强制私有	private/protected/public 语言级	private/protected/public 语言级
多重继承	内建，C3 线性化，冲突可显式	不支持（接口继承替代）	支持但有菱形/二义性风险
工厂	函数工厂 3 行搞定	工厂类/抽象工厂样板繁多	工厂类 + 虚构造器复杂
委托/代理	getattr 几行实现	Proxy/Decorator 模式类	需要手写大量样板

Python 发展历史背景

·非绑定方法→普通函数：Python 2 中经类取方法得到“非绑定方法”（调用仍需实例），3.x 中统一为普通函数——作者在认证注释里戏称“非绑定方法已死，普通函数万岁”。

- MRO 的演进：Python 2.2 之前只有 DFLR；2.2 引入新式类时改为 C3 线性化，修复了"右侧低类无法覆盖共同超类"的问题；3.0 之后所有类都是新式类，object 统一置于根顶——这使得"每棵多继承树都是菱形"成为现代 Python 的默认事实。

- f-string 现代写法：示例里 `f'\n{' - ' 80}\n'` 用了 Python 3.12 的同型引号（PEP 701），说明书中代码已随语言版本更新。

高级开发者需要掌握的相关知识

- C3 线性化算法：理解"局部优先 + 单调性"两条约束，就能预判 mro 结果与 `TypeError: Cannot create a consistent MRO`（不一致继承）错误来源。

- `super()` 的 MRO 语义：`super()` 不是"父类"而是"MRO 中当前类的下一个类"，多继承下手写显式类名常常更可读（作者明确表态）。

- 描述符与 `getattr/getattribute` 的协同：绑定方法本质就是描述符协议（`get`）的实现；理解它等于理解属性查找全栈。

- 元类（metaclass）第二棵继承树：`mapattrs` 的 "type" 怪象正是元类影响类对象属性的实证——第 32、40 章深入。

- 类型提示与协议：现代 Python 用 `typing.Protocol` 把"鸭子接口"显式化，供静态检查器与 IDE 使用，但运行时依然纯动态。

学习建议 (Learning Advice)

·重要程度：★★★★★ (5/5)

·本章是"从抄代码到设计与设计评审"的底层能力：继承/组合/委托的选择、MRO 心智、工厂与混入，几乎出现在每一个 middle 以上级别的 Python 面试与代码评审里。

·应该掌握到什么程度：

1. 【必须】能口述 is-a / has-a / like-a 三关系的判断标准，并为每个场景举一个自己的例子；
2. 【必须】能手算出常见菱形继承的 mro（先用"DFLR 列出 + 去重留右"的直觉法验证）；
3. 【必须】理解绑定方法 $\neq$ 普通函数，能在回调、map/filter、装饰器场景正确使用；
4. 【建议】能独立写出 ListInstance 级别的混入工具与 mapattrs 级别的自省工具；
5. 【进阶】分析 Django 的 QuerySet/mixin、标准库 functools 装饰器，思考它们如何落实本章的模式。

·特别提醒——"优先组合而非继承"：这是本章真正的灵魂。判断顺序：先问"有没有 has-a"→"有没有 like-a（委托可行？）"→最后才考虑 is-a。当继承深度超过 2-3 层、或子类只需要父类一小部分能力时，几乎可以确定组合/委托更优。多重继承只应作为"能力混入"（方法包）使用，避免在其中定义沉重的 init 状态。

·后续应该学习哪些相关内容：

- 下一章 32 (高级类主题) : `super()` 完整语义、静态/类方法、装饰器、受管属性、元类——补全本章所有"伏笔" ;
- 第 38 章 (受管属性)、第 39 章 (装饰器)、第 40 章 (元类) : 属性的更深层机制;
- 若从事框架开发, 辅以 GoF 《设计模式》或 Python 版模式书籍, 重点对照 Decorator/Proxy/Factory/Composite 模式与本能力对照;
- 用 `inspect`、`pprint`、`pdb` 把自己写的混入工具武装起来, 练习"自省式调试"思维。